

**SVEUČILIŠTE JOSIPA JURJA STROSSMAYERA U OSIJEKU
FAKULTET ELEKTROTEHNIKE, RAČUNARSTVA I
INFORMACIJSKIH TEHNOLOGIJA OSIJEK**

Sveučilišni diplomski studij Računarstva

**IMPLEMENTACIJA BLOCKCHAIN SUSTAVA ZA
KUPOPRODAJU NEKRETNINA POMOĆU PAMETNIH
UGOVORA**

Diplomski rad

Mislav Čačić

Osijek, 2026.

SADRŽAJ

1. UVOD	1
1.1. Zadatak diplomskog rada	2
2. PREGLED POSTOJEĆIH SUSTAVA ZA PRAĆENJE VLASNIŠTVA	2
2.1. Prijenos vlasništva nad nekretninama	3
2.2. Tradicionalni sustavi za praćenje vlasništva	3
2.3. Digitalni sustavi za evidenciju vlasništva	4
2.4. Blockchain sustavi za praćenje vlasništva	5
2.5. Usporedna analiza sustava za praćenje vlasništva	6
2.5.1. Sigurnost i pouzdanost	6
2.5.2. Transparentnost i dostupnost podataka	6
2.5.3. Automatizacija i potreba za posrednicima	7
2.5.4. Sažetak usporedbe	7
3. PRAVNI OKVIR PRIJENOSA VLASNIŠTVA NEKRETNINA U REPUBLICI HRVATSKOJ	8
3.1. Postupak kupoprodaje nekretnina	8
3.2. Uloga zemljišnih knjiga i katastra	8
3.3. Uvjeti za prijenos vlasništva	9
3.4. Ograničenja postojećeg pravnog okvira	9
4. TEHNOLOGIJE I ARHITEKTURA SUSTAVA	10
4.1. Odabir tehnologija	10
4.2. Arhitektura sustava	11
5. IMPLEMENTACIJA SUSTAVA	14
5.1. Implementacija pametnih ugovora	14
5.1.1. Pametni ugovor <i>PropertyRegistry</i>	15
5.1.2. Pametni ugovor <i>RealEstateEscrow</i>	28
5.1.3. Pametni ugovor <i>MockEUR</i>	39
5.2. Pohrana dokumentacije izvan blockchaina	41
5.2.1. Document Storage Server	42
5.2.2. Hashiranje i povezivanje dokumenta s blockchain zapisom	44

5.3. Implementacija korisničkog sučelja.....	48
5.3.1. Povezivanje s blockchain mrežom i MetaMaskom	49
5.3.2. Korisnički profili i višekorisnički način rada.....	51
5.3.3. Registracija i pregled nekretnina	55
5.3.4. Predaja i verifikacija dokumentacije.....	58
5.3.5. Kreiranje prodaje i kupnja nekretnine.....	63
5.3.6. Prikaz povijesti transakcija	68
6. PRIKAZ I ISPITIVANJE RADA SUSTAVA	69
6.1. Povezivanje korisnika i prikaz korisničkog profila	69
6.2. Registracija nekretnine i predaja dokumentacije.....	71
6.3. Verifikacija dokumentacije.....	75
6.4. Kreiranje prodaje	79
6.5. Dodjela simuliranih sredstava kupcu	81
6.6. Kupnja nekretnine.....	83
6.7. Rezultat kupoprodaje i povijest transakcija	85
6.8. Ispitivanje sustava	87
7. ZAKLJUČAK.....	90
LITERATURA	93
POPIS KRATICA	95
SAŽETAK.....	96
ABSTRACT	97
ŽIVOTOPIS.....	98
PRILOZI	99

1. UVOD

Kupovina, prijepis i prijenos vlasništva nad nekretninama jedan je od administrativnih i pravnih procesa u svakom društvu i državi. Naime, radi se o postupku koji ima velik gospodarski, pravni i društveni značaj zbog uključivanja visokih financijskih iznosa, dugoročnih posljedica za sve sudionike te potrebe za visokom razinom pravne sigurnosti. U većini država, pa tako i u Republici Hrvatskoj, evidencija vlasništva nad nekretninama temelji se na centraliziranim sustavima kao što su zemljišne knjige i katastri, pri čemu je prijenos vlasništva povezan s velikim brojem formalnih koraka i uključivanjem velikog broja posrednika poput javnih bilježnika, sudova, državnih institucija itd. Iako takvi sustavi osiguravaju pravnu valjanost i institucionalnu kontrolu, u praksi često dolazi do problema kao što su složenost postupka i njegova dugotrajnost, administrativni troškovi, ograničena transparentnost i slično tomu. Dodatni su izazovi i mogući problemi mogućnost pogreške u evidencijama, preciznije rečeno – pogreške prilikom unosa podataka. Isto tako, problem se može stvoriti zbog neusklađenosti podataka između različitih registara te zbog ograničene mogućnosti modernizacije i digitalizacije sustava za praćenje vlasništva nad nekretninama.

Razvojem informacijskih i komunikacijskih tehnologija razvili su se i različiti digitalni sustavi koji nastoje unaprijediti postojeće modele evidencije vlasništva nekretnina. Ta rješenja najčešće se temelje na bazama podataka i *web*-aplikacijama koje uz to što omogućuju brži pristup informacijama, omogućuju i djelomičnu automatizaciju. Međutim, problem kod ovih sustava jest što i dalje zadržavaju centraliziranu arhitekturu te se oslanjaju na povjerenje u jednu instituciju ili skup institucija čime se u potpunosti ne uklanjaju temeljni problemi tradicionalnih sustava.

Blockchain tehnologija, kao distribuirana i decentralizirana knjiga zapisa, u posljednjih desetak godina privukla je pozornost u različitim područjima primjene. Najčešće je riječ o financijama, logistici, javnoj uporabi, kupoprodaji, nabavi i brojnim drugim primjenama. Blockchain, zbog svojih značajki, poput nepromjenjivosti zapisa, transparentnosti, distribuiranog konsenzusa i mogućnosti automatizacije s pomoću pametnih ugovora, ostvaruje nove mogućnosti za unapređenje sustava za praćenje vlasništva nad nekretninama. Na temelju svega iznesenog blockchain se sve češće razmatra kao potencijalna tehnologija koja bi mogla i trebala smanjiti potrebu za posrednicima, povećati sigurnost samih transakcija, ubrzati postupak prijenosa vlasništva, smanjiti mogućnost pogreške u tim procesima te smanjiti troškove prijenosa.

1.1. Zadatak diplomskog rada

U ovom radu daje se pregled i usporedba postojećih sustava za praćenje vlasništva nad nekretninama, uključujući tradicionalne, digitalne i blockchainom temeljene pristupe. Analiziraju se njihove osnovne značajke, prednosti i ograničenja s posebnim naglaskom na sigurnost, transparentnost, automatizaciju i potrebu za posrednicima. Uz sve navedeno, provedena usporedba predstavlja teorijsku i analitičku podlogu za razvoj prototipa decentraliziranog sustava za kupoprodaju nekretnina. Riječ je o prototipu koji se temelji na blockchain tehnologiji i pametnim ugovorima, a sam će prototip biti razrađen u nastavku diplomskog rada.

U drugom poglavlju prikazani su tradicionalni, digitalni i blockchainom temeljeni sustavi za praćenje vlasništva nad nekretninama te je provedena njihova usporedna analiza. U trećem poglavlju analiziran je pravni okvir prijenosa vlasništva nad nekretninama u Republici Hrvatskoj. Četvrto poglavlje opisuje odabrane tehnologije i arhitekturu razvijenog sustava. U petom poglavlju detaljno je prikazana implementacija pametnih ugovora, sustava za pohranu dokumentacije i korisničkog sučelja. U šestom poglavlju prikazan je rad sustava kroz praktične scenarije korištenja i provedena ispitivanja, dok su u sedmom poglavlju izneseni zaključci rada.

2. PREGLED POSTOJEĆIH SUSTAVA ZA PRAĆENJE VLASNIŠTVA

Sustavi za praćenje vlasništva nad nekretninama od velike su važnosti za osiguravanje pravne sigurnosti, zaštite imovinskih prava te provedbe pravnih i gospodarskih transakcija. Njihova je osnovna svrha pouzdano evidentiranje podataka o vlasnicima nekretnina, pravima koja na njima postoje te promjenama tih prava. U Republici Hrvatskoj, kao i u većini država, taj je sustav tijekom povijesti razvijen kao centralizirani registri kojima upravljaju državne institucije. Razvojem društva i povećanjem broja pravnih i tržišnih transakcija, sustavi za praćenje evidencija vlasništva postajali su sve složeniji. Uz tradicionalne papirnate registre postupno su se uvodili digitalni sustavi koji su trebali ubrzati procese i povećati dostupnost podataka. Unatoč digitalizaciji temeljna struktura većine postojećih sustava ostala je nepromijenjena, odnosno ostala je centralizirana i ovisna o povjerenju nadležnih državnih tijela.

S druge strane, u novije vrijeme, razvojem blockchain tehnologije, pojavljuju se alternativni pristupi praćenju vlasništva koji se temelje na distribuiranim sustavima zapisa i automatizaciji

putem pametnih ugovora. Takvi pristupi nastoje riješiti ograničenja tradicionalnih i digitalnih centraliziranih sustava, posebice u transparentnosti, sigurnosti i potrebi za posrednicima.

U ovom poglavlju dan je pregled i analiza već postojećih sustava za praćenje vlasništva nad nekretninama. Sustavi su podijeljeni na tradicionalne, digitalne i blockchain temeljene, pri čemu se za svaki pristup razmatraju njegove osnovne značajke, prednosti i ograničenja.

2.1. Prijenos vlasništva nad nekretninama

Prijenos vlasništva nad nekretninama predstavlja pravni postupak u kojem se pravo vlasništva nad nekretninom prenosi s jedne osobe na drugu, najčešće temeljem kupoprodajnog ugovora, darovanja ili nasljeđivanja. U pravnom sustavu Republike Hrvatske prijenos vlasništva nad nekretninama ima pravni učinak tek upisom u zemljišne knjige, čime se osigurava javnost i pravna sigurnost tog prava [1].

Zemljišne knjige predstavljaju javni registar u kojem se evidentiraju stvarna prava na nekretninama, uključujući pravo vlasništva, založna prava i druga prava i ograničenja. Upis u zemljišne knjige ima konstitutivni učinak, što znači da se vlasništvo smatra stečenim tek nakon provedbe upisa. Time se osigurava povjerenje trećih osoba u podatke sadržane u registru te se smanjuje mogućnost sporova oko vlasništva.

Postupak prijenosa vlasništva u praksi uključuje više koraka i sudionika, kao što su ugovorne strane, javni bilježnici, sudovi, nadležna tijela i drugi. Iako takav sustav osigurava visoku razinu pravne kontrole, često se ističu problemi poput dugotrajnosti postupaka, administrativnih troškova i ograničene transparentnosti za krajnje korisnike te mogući problemi i pravni sporovi ako dođe do pogreške prilikom upisivanja podataka. Upravo su ti nedostaci jedan od glavnih razloga za razmatranje novih tehnoloških rješenja u području evidencije i prijenosa vlasništva.

2.2. Tradicionalni sustavi za praćenje vlasništva

Tradicionalni sustavi za praćenje vlasništva nad nekretninama temelje se na centraliziranim javnim registrima koje vode državne institucije. U Republici Hrvatskoj temelj takvog sustava čine zemljišne knjige i katastar, koji zajedno osiguravaju evidenciju pravnog i faktičnog stanja nekretnina. Ti su sustavi razvijeni s ciljem osiguravanja pravne sigurnosti, zaštite vlasničkih prava i javnosti podataka [1], [2].

Zemljišne knjige predstavljaju javni registar u kojem se evidentiraju stvarna prava na nekretninama, uključujući pravo vlasništva, založna prava i druga ograničenja. Naime, budući da upis u zemljišne knjige ima konstitutivni učinak, njihova osnovna funkcija jest osigurati pravnu sigurnost u pravnom prometu nekretnina. Time se omogućuje trećim osobama pouzdanje u stanje upisa, što je ključno za kupoprodaju i druge pravne poslove vezane uz nekretnine [1].

Katastar nekretnina vodi evidenciju o položaju, obliku, površini i namjeni nekretnina. Za razliku od zemljišnih knjiga, katastar ne evidentira vlasnička prava, već tehničke i prostorne podatke. U praksi, nesklad između podataka u katastru i zemljišnim knjigama može dovesti do dodatnih administrativnih postupaka i usporavanja prijenosa vlasništva [2].

U tradicionalnom sustavu značajnu ulogu imaju nadležna državna tijela koja osiguravaju zakonitost i valjanost pravnog prometa nekretnina. Iako takav model osigurava visoku razinu pravne kontrole, istodobno povećava složenost postupka, trajanje procesa i ukupne troškove za sudionike [1].

Potrebno je naglasiti i prednosti tradicionalnih sustava koji se očituju u njihovoj dugoj pravnoj tradiciji, jasno definiranom zakonodavnom okviru i visokoj razini institucionalnog povjerenja. Međutim, njihova glavna ograničenja uključuju centraliziranu strukturu, sporost postupaka, ograničenu transparentnost i dostupnost informacija krajnjim korisnicima te povećanu mogućnost pogrešaka u administrativnim postupcima [1].

2.3. Digitalni sustavi za evidenciju vlasništva

Digitalni sustavi za evidenciju vlasništva nad nekretninama razvijeni su u okviru šire digitalne transformacije javne uprave u Republici Hrvatskoj, s ciljem modernizacije administrativnih postupaka, povećanja učinkovitosti rada javnih tijela i poboljšanja dostupnosti javnih usluga građanima [3]. Digitalizacija zemljišnih knjiga i katastra predstavlja značajan korak u tom procesu jer omogućuje elektroničku obradu podataka i postupno napuštanje isključivo papirnatih evidencija.

U digitalnim sustavima podaci o nekretninama pohranjuju se u centralizirane informacijske sustave kojima upravljaju nadležna državna tijela. Takav pristup omogućuje standardizaciju podataka, elektroničko podnošenje zahtjeva te bržu razmjenu informacija između institucija, čime se pojednostavljaju administrativni postupci povezani s evidencijom i prijenosom vlasništva [4].

U konačnici se mogu istaknuti prednosti digitalnih rješenja koje se očituju u sve većoj dostupnosti podataka, smanjenju administrativnog opterećenja korisnika i skraćivanju trajanja pojedinih postupaka. Digitalni sustavi također doprinose boljoj kontroli i praćenju promjena u evidencijama, što jača pravnu sigurnost i transparentnost pravnog prometa nekretnina [3].

Unatoč navedenim prednostima digitalni sustavi za evidenciju vlasništva u Republici Hrvatskoj u pravilu zadržavaju centraliziranu arhitekturu i oslanjaju se na postojeći institucionalni okvir. Time se ne uklanjaju u potpunosti ograničenja tradicionalnih sustava, poput ovisnosti o nadležnim tijelima i administrativnim postupcima, već se oni prvenstveno moderniziraju i ubrzavaju korištenjem informacijskih tehnologija [5].

2.4. Blockchain sustavi za praćenje vlasništva

Sustavi zemljišnih knjiga i katastara u Republici Hrvatskoj za sada nemaju implementiranu blockchain tehnologiju. Međutim, ona je prisutna u stručnim i regulatornim tehnologijama kao potencijalna mogućnost za buduću modernizaciju javnih registara. Primjenu distribuiranih knjiga zapisa (DLT) u administrativnom i financijskom kontekstu razmatraju Hrvatska narodna banka i slične institucije radi prednosti poput transparentnosti, nepromjenjivosti zapisa i smanjenja potrebe za posrednicima [6]. Zbog manjka domaćih implementacija u literaturi se većinom navode i analiziraju primjeri drugih država [7], [8].

Navedene su države provele pilot-projekte ili djelomične implementacije blockchaine u području evidencije vlasništva nad nekretninama, a najpoznatiji primjer dolazi iz Švedske. Naime, švedska nacionalna geodetska služba Lantmäteriet provela je testiranje blockchain rješenja za prijenos vlasništva nad nekretninama u kojem se blockchain koristi za automatizaciju procesa i bilježenje transakcija, a državne institucije imaju nadzornu ulogu [7], [9].

Gruzija se također navodi kao država u kojoj je blockchain tehnologija integrirana u sustav zemljišne registracije uz podršku Svjetske banke, a u navedenom sustavu blockchain služi kao dodatni sloj za osiguranje integriteta i nepromjenjivosti zapisa o vlasništvu. Rezultat toga jest povećanje povjerenja u sustav te smanjenje mogućnosti manipulacije podacima [9].

Bitno je naglasiti da uz državne projekte postoje još i komercijalna rješenja koja koriste blockchain tehnologiju za kupoprodaju nekretnina. Primjer je platforma Propy koja omogućuje provođenje transakcija nekretninama uz korištenje pametnih ugovora. Pri tome se dio procesa automatizira, a

zapisi o transakcijama pohranjuju se na blockchainu [8], [10]. Navedeni sustavi pokazuju da je primjena blockchaina tehnički izvediva u ovom području, ali njihova pravna valjanost ovisi prije svega o zakonodavnom okviru pojedine države [8].

Na temelju analiza i preglednih znanstvenih radova može se zaključiti da blockchain sustavi za praćenje vlasništva mogu doprinijeti povećanju transparentnosti, sigurnosti i očuvanju postupaka [6], [7]. Također, potrebno je naglasiti i ograničenja poput pravnih prepreka, potrebe za pouzdanim unosom podataka te integracije s postojećim institucionalnim sustavima [6], [7]. Zbog navedenog blockchain se u ovom području promatra kao dopuna, a ne potpuna zamjena tradicionalnih registara [6].

2.5. Usporedna analiza sustava za praćenje vlasništva

Usporedba sustava za praćenje vlasništva nad nekretninama provodi se s ciljem isticanja razlika među tradicionalnim, digitalnim i blockchainom temeljenim pristupima. Kriteriji koji se uzimaju za usporedbu jesu sigurnost i pouzdanost sustava, transparentnost podataka i njihova dostupnost te razina automatizacije i potrebe za posrednicima.

2.5.1. Sigurnost i pouzdanost

Tradicionalni sustavi koji se temelje na zemljišnim knjigama i katastru osiguravaju visoku razinu pravne sigurnosti zahvaljujući jasno definiranim zakonodavnim okvirima i institucionalnoj kontroli. Pouzdanost ovog sustava proizlazi iz dugogodišnje prakse i povjerenja u državna tijela, no centralizirana struktura predstavlja potencijalnu ranjivost u slučaju pogrešaka ili zlouporaba.

Digitalni sustavi dodatno unaprjeđuju pouzdanost elektroničkom obradom i pohranom podataka smanjujući tako rizik od fizičkog gubitka dokumentacije. Ipak, i dalje se radi o centraliziranom sustavu koji ovisi o integritetu upravljačkih institucija.

Blockchain sustavi nude drugačiji pristup sigurnosti temeljen na nepromjenjivosti zapisa i distribuiranoj pohrani podataka. Jednom zapisane transakcije ne mogu se jednostavno izmijeniti, čime se smanjuje mogućnost manipulacije i zlouporabe. Međutim, sigurnost tog sustava uvelike ovisi o ispravnosti početnog unosa podataka i pravnoj integraciji s postojećim sustavima.

2.5.2. Transparentnost i dostupnost podataka

Kod tradicionalnih sustava transparentnost je ograničena administrativnim podacima i potrebom za fizičkim ili formalnim pristupom podacima. Dostupnost informacija ovisi o radnom vremenu institucija i propisanim procedurama.

Suprotno tomu, digitalni sustavi znatno povećavaju dostupnost podataka omogućujući elektronički uvid u svakom trenutku i bržu obradu zahtjeva. Zbog toga se poboljšava transparentnost, no i dalje je ograničena pravilima pristupa koje definiraju nadležna tijela.

Blockchain rješenja omogućuju iznimno visoku razinu transparentnosti jer su zapisi dostupni svim ovlaštenim sudionicima mreže u stvarnom vremenu što pridonosi povećanju povjerenja korisnika u sustav. Međutim, otvara se pitanje zaštite osobnih podataka i usklađenosti s važećim propisima.

2.5.3. Automatizacija i potreba za posrednicima

U tradicionalnim sustavima prijenos vlasništva značajno ovisi o posrednicima poput sudova, javnih bilježnika i drugih nadležnih tijela, što povećava složenost i trajanje postupka. Prema tome, automatizacija je minimalna, a većina procesa provodi se ručno ili poluručno.

Kod digitalnih sustava situacija je nešto drugačija. Oni djelomično automatiziraju administrativne korake, primjerice podnošenje zahtjeva i obradu dokumentacije, no posrednici i dalje imaju ključnu ulogu u donošenju odluka i potvrdi pravne valjanosti.

Promatrajući blockchain sustave, jasno se uviđa da omogućuju višu razinu automatizacije putem pametnih ugovora koji mogu automatski izvršavati određene radnje kada su ispunjeni unaprijed definirani uvjeti. Time se potencijalno smanjuje potreba za posrednicima i ubrzava postupak prijenosa vlasništva. Ipak, u stvarnosti potpuna eliminacija posrednika zasad nije realna zbog pravnih i regulatornih ograničenja.

2.5.4. Sažetak usporedbe

Na temelju provedene analize može se zaključiti da tradicionalni sustavi pružaju visoku pravnu sigurnost, ali su sporiji i manje fleksibilni. Digitalni sustavi predstavljaju evolucijski korak koji poboljšava učinkovitost i dostupnost, dok blockchain rješenja nude potencijal za dodatnu automatizaciju i transparentnost. Međutim, njihova primjena u području evidencije vlasništva trenutačno je ograničena pravnim, tehničkim i institucionalnim izazovima.

3. PRAVNI OKVIR PRIJENOSA VLASNIŠTVA NEKRETNINA U REPUBLICI HRVATSKOJ

Pravni okvir prijenosa vlasništva nad nekretninama u Republici Hrvatskoj temelji se na pravnim i zemljišnoknjižnim propisima, kao i na pravilima koja uređuju oporezivanje i digitalno podnošenje zahtjeva za upis. Za prijenos vlasništva osobito važi *Zakon o vlasništvu i drugim stvarnim pravima*, *Zakon o zemljišnim knjigama*, službene informacije sa sustava *gov.hr* o upisu prava vlasništva te podaci *Državne geodetske uprave* i sustava *Uređena zemlja* o katastru i *Zajedničkom informacijskom sustavu zemljišnih knjiga i katastra (ZIS)* [11].

3.1. Postupak kupoprodaje nekretnina

U Republici Hrvatskoj postupak kupoprodaje nekretnina najčešće započinje dogovorom prodavača i kupca o predmetu kupnje, cijeni, načinima plaćanja i drugim bitnim stvarima za nekretninu koja se prodaje. Nakon dogovora popunjava se i potpisuje kupoprodajni ugovor koji sadrži osobne podatke od obje strane te dogovorene podatke nekretnine. Međutim, samim sklapanjem ugovora kupac još ne postaje vlasnik nekretnine, jer se pravo vlasništva u pravilu stječe tek upisom u zemljišnu knjigu. Službene informacije portala *gov.hr* izričito navode da se prijedlog za upis prava vlasništva podnosi mjesno nadležnom općinskom sudu koji vodi zemljišnu knjigu, i to elektronički putem javnog bilježnika ili odvjetnika kao ovlaštenih korisnika sustava [12].

3.2. Uloga zemljišnih knjiga i katastra

Zemljišne knjige predstavljaju javni registar o pravnom stanju nekretnina mjerodavnom za pravni promet u Republici Hrvatskoj. To proizlazi iz Zakona o zemljišnim knjigama, koji uređuje ustrojstvo, vođenje i vrste upisa u zemljišne knjige. Njihova je temeljna funkcija osigurati pravnu sigurnost u prometu nekretnina, jer se upravo u njima evidentiraju vlasništvo i druga stvarna prava relevantna za pravni promet [11].

Katastar vodi evidenciju o tehničkim i prostornim podacima nekretnina poput položaja i oblika zemljišnih čestica, površini i namjeni zemljišnih čestica, vlasnicima i korisnicima itd. Državna geodetska uprava navodi da u okviru katastarskog sustava obavlja poslove vezane uz katastar zemljišta, katastar nekretnina, katastarske izmjere, održavanje katastarskih operata i povezivanje s drugim registrima. Zbog toga zemljišne knjige i katastar imaju različite, ali međusobno povezane

funkcije: zemljišne knjige evidentiraju pravno stanje, a katastar tehničko i prostorno stanje nekretnine [2].

Za suvremeni sustav prijenosa vlasništva posebno je važan i Zajednički informacijski sustav zemljišnih knjiga i katastra (ZIS), dostupan kroz sustav Uređena zemlja. Taj sustav omogućuje pregled zemljišnoknjižnih i katastarskih podataka te elektroničko podnošenje prijedloga, čime se ubrzava administrativni dio postupka [14].

3.3. Uvjeti za prijenos vlasništva

Kako bi prijenos vlasništva nad nekretninom bio valjan, nije dovoljan samo dogovor stranaka, nego se moraju ispuniti određeni materijalni i formalni uvjeti. U Hrvatskom pravnom sustavu bitno je postojanje valjanog pravnog temelja kojeg najčešće predstavlja kupoprodajni ugovor i provedba odgovarajućeg upisa u zemljišnu knjigu. Upravo zato službeni državni portal naglašava da se pravo vlasništva upisuje na temelju prijedloga podnesenog zemljišnoknjižnom sudu za nekretninu koja je bila predmet pravnog posla [12].

Također, vrlo je važna identifikacija nekretnine, usklađenost zemljišnoknjižnih i katastarskih podataka i postojanje sve potrebne dokumentacije za provedbu upisa. Važan je i porezni aspekt zbog plaćanja poreza na promet nekretnina po stopi od 3%, odnosno PDV u slučajevima propisanim zakonom [15].

3.4. Ograničenja postojećeg pravnog okvira

Pravni okvir Republike Hrvatske osigurava vrlo visoku razinu sigurnosti, međutim postojeći sustav i dalje je u značajnoj mjeri formalan i institucionalno složen. U cijelom postupku kupoprodaje nekretnine i prijepisa vlasništva sudjeluju različiti i brojni akteri poput sudova, javnih bilježnika, nadležnih katastarskih i poreznih tijela. Elektronička obrada i dalje nije uklonila potrebu za formalnim provjerama i pravnim koracima. Djelomični elektronički upis i obrada preko ovlaštenih korisnika pokazuje da je sustav digitaliziran, ali i dalje ne potpuno automatiziran za krajnjeg korisnika [12].

Takva pravna struktura ostavlja prostor za razmatranje tehnoloških rješenja koja bi dodatno smanjila administrativno opterećenje, ubrzala provjere, povećala transparentnost procesa i smanjila troškove svim sudionicima, naročito kupcu nekretnine. U tom kontekstu i kao rješenje

problema mogu se razmatrati pametni ugovori i blockchain tehnologija kao nadogradnja postojećeg sustava, ali s potrebnim usklađivanjem pravnih i institucionalnih okvira. To je posebno važno za daljnji dio rada u kojem će se razraditi prototip decentraliziranog sustava kupoprodaje nekretnina.

4. TEHNOLOGIJE I ARHITEKTURA SUSTAVA

4.1. Odabir tehnologija

U svrhu izrade sustava za kupoprodaju nekretnina pomoću pametnih ugovora odabrane su tehnologije koje omogućuju sigurnu, decentraliziranu i učinkovitu obradu transakcija i podataka. Prilikom odabira tehnologija vodilo se računa o performansama, sigurnosti, dostupnosti alata i primjeni u realnim sustavima.

Blockchain tehnologija predstavlja temelj sustava jer omogućuje nepromjenjivu pohranu podataka i eliminaciju potrebe za centraliziranim posrednicima. Korištenje blockchain platformi značajno povećava sigurnost i transparentnost transakcija te smanjuje mogućnost manipulacije podacima [16].

Zbog svoje široke primjene, visoke razine razvijenosti ekosustava i podrške za pametne ugovore za implementaciju sustava odabrana je Ethereum platforma. Ethereum platforma omogućuje izvođenje poslovne logike kroz pametne ugovore koji se automatski izvršavaju kada se zadovolje definirani uvjeti, što je i glavni razlog visoke razine automatizacije procesa kupoprodaje.

Postoje i druge alternative, poput Solane, koja omogućuje bržu obradu podataka uz niže troškove. Solana koristi kombinaciju mehanizama *Proof of Stake* i *Proof of History*, čime postiže visoku stabilnost i učinkovitost [17] [18]. Međutim, zbog većeg ekosustava i većoj dostupnosti razvojnih alata odabrana je Ethereum platforma.

Za implementaciju pametnih ugovora koristi se programski jezik Solidity što je standard za razvoj na Ethereum platformi. On omogućuje definiranje pravila kupoprodaje, odnosno provjeru vlasništva, dostupnost sredstava, valjanost dokumentacije i automatskog prijenosa vlasništva nakon ispunjenja uvjeta.

Za razvoj i testiranje pametnih ugovora koristi se Hardhat koje omogućuje lokalno testiranje, simulaciju blockchain mreže i deployment ugovora.

Za izradu korisničkog sučelja koristi se React koji omogućuje razvoj interaktivne web-aplikacije i komunikaciju s pametnim ugovorima na blockchain mreži. Za komunikaciju između frontenda i blockchain mreže koristi se *ethers.js* biblioteka koja obavlja slanje transakcija i dohvat podataka s blockchaina.

Za autentifikaciju i upravljanje korisničkim računima koristi se MetaMask koji omogućuje sigurno potpisivanje transakcija i povezivanje korisnika s blockchain mrežom.

Kombinacija navedenih tehnologija omogućuje izradu sustava koji je transparentan, siguran, automatiziran i koji pruža osnovu za daljnji razvoj i primjenu u svakodnevnici za kupoprodaju nekretnina.

4.2. Arhitektura sustava

Razvijeni prototip sustava organiziran je u nekoliko međusobno povezanih slojeva: korisničko sučelje, sloj za komunikaciju s blockchain mrežom, digitalni novčanik korisnika, sloj pametnih ugovora i pomoćni off-chain sloj za pohranu dokumentacije. Razlika između klasične web stranice je što sustav nema zaseban aplikacijski poslužitelj ni relacijsku bazu podataka u kojoj bi se čuvala sva stanja i podaci. Ključni podaci i pravila poslovne logike nalaze se u pametnim ugovorima na Ethereum kompatibilnoj blockchain mreži. Iznimka tome je pomoćni lokalni off-chain servis koji se u prototipu koristi isključivo za pohranu i pregled izvornih datoteka dokumentacije. U okviru ovog rada i prototipa mreža je pokrenuta lokalno pomoću razvojnog okruženja Hardhat čime je omogućeno kontrolirano testiranje cjelokupnog procesa bez korištenja stvarnih financijskih sredstava i mreže.

Korisnički sloj, odnosno sučelje, izrađeno je pomoću React-a i TypeScript-a. Aplikacija razlikuje tri osnovna profila korisnika: administratora, verifikatora i korisnika. Profil korisnika nije trajno vezan uz ulogu kupca ili prodavatelja već ista blockchain adresa može, ovisno o konkretnoj transakciji, nastupiti i kao prodavatelj vlastite nekretnine i kao kupac nekretnine drugog korisnika. Time je omogućen višekorisnički način rada u kojem više korisnika može registrirati nekretnine, kreirati prodaje te sudjelovati u kupoprodajnim transakcijama. Implementirane su komponente koje omogućuju pregled i registraciju nekretnina, predaju i verifikaciju dokumenata, kreiranje i pregled prodaja, kupnju nekretnina, upravljanje simuliranim sredstvima te prikaz povijesti transakcija nekretninama. Verifikatoru su dostupne funkcionalnosti provjere dokumentacije, a administratoru nadzorne i administrativne mogućnosti. Prikaz pojedinih funkcionalnosti u

korisničkom sučelju prilagođava se profilu i odnosu povezanog računa prema određenoj nekretnini ili prodaji, dok se sva stvarna kontrola dopuštenih radnji provodi unutar pametnih ugovora.

Komunikacija između blockchain mreže i korisničkog sučelja ostvarena je bibliotekom ethers.js. Sve operacije koje služe isključivo za čitanje blockchain stanja izvršavaju se izravno prema lokalnom Hardhat JSON-RPC poslužitelju usluge. Na taj se način dohvaćaju podaci o nekretninama, vlasnicima, dokumentaciji, prodajama, stanju simuliranih sredstava i drugim podacima potrebnim za prikaz korisničkog sučelja. Sve operacije kojima se mijenja blockchain stanje, poput prodaje, odobravanja sredstava za kupnju ili izvršavanje kupnje zahtijevaju potpis korisnika putem digitalnog novčanika MetaMask. Sam MetaMask je povezan s lokalnom Hardhat mrežom te omogućuje odabir blockchain računa i potvrđivanje transakcije prije njihova slanja na mrežu.

Središnji dio arhitekture predstavljaju tri pametna ugovora: *PropertyRegistry*, *RealEstateEscrow* i *MockEUR*. Pametni ugovor *PropertyRegistry* predstavlja blockchain registar nekretnina. U njemu se evidentiraju osnovni podaci o nekretnini, njezini trenutni digitalni vlasnik i pripadajuća dokumentacija. Za svaku nekretninu u prototipu definirana su tri obvezna dokumenta: zemljišnoknjižni izvadak, katastarski dokument i dokaz o vlasništvu nad nekretninom. Svaki dokument ima podatke o tome je li predan te status verifikacije koji može biti: *Pending*, *Verified* ili *Rejected*. Nekretnina se smatra spremnom za prodaju tek kada su sva tri obvezna dokumenta predana i potvrđena, odnosno u statusu *Verified*.

Izvorne datoteke dokumentacije se ne pohranjuju izravno na blockchain. Prilikom predaje dokumenata korisničko sučelje izračunava njezinu kriptografsku *keccak256* hash vrijednost, dok se izvorna datoteka sprema u lokalnom off-chain spremištu. Pomoćni servis za pohranu dokumentacije vraća URI link dokumenta. Nakon toga u pametni ugovor *PropertyRegistry* se zapisuju *documentHash* i *documentURI*. Hash vrijednost omogućuje provjeru identiteta i integriteta predane dokumentacije, dok URI omogućuje dohvat njezinog izvornog sadržaja. Na taj način verifikator prije potvrde ili odbijanja dokumenta može otvoriti stvarnu datoteku i pregledati ju, dok blockchain zadržava nepromjenjiv zapis podataka povezanih s predanim dokumentom. Lokalna baza koristi se isključivo za potrebe prototipa, dok se isti pristup u stvarnom sustavu može povezati s drugim rješenjima za off-chain pohranu.

Pametni ugovor *RealEstateEscrow* upravlja procesom kupoprodaje nekretnine. On povezuje podatke iz registra nekretnina sa simuliranim financijskim sredstvima te provjerava sve uvjete koji moraju biti zadovoljeni prije izvršenja transakcije. Prodaju može kreirati samo korisnik koji je

trenutni vlasnik nekretnine i čija je dokumentacija valjana i potvrđena. Prije same kupnje sustav provjerava postoji li prodaja i je li aktivna, jesu li svi obvezni dokumenti potvrđeni, je li prodavatelj i dalje trenutačni digitalni vlasnik nekretnine, razlikuju li se adrese kupca i prodavatelja te raspolaže li kupac dovoljnim iznosom simuliranih sredstava i odgovarajućim odobrenjem za njihovo korištenje. Svi rezultati tih provjera dostupni su u korisničkom sučelju kako bi kupac prije pokretanja transakcije vidio i potvrdio kako su svi preduvjeti zadovoljeni.

Za simulaciju financijskog dijela kupoprodaje koristi se pametni ugovor *MockEUR* koji je implementiran prema ERC-20 standardu. Token nema stvarnu novčanu vrijednost, nego služi isključivo za demonstraciju prijenosa sredstava između sudionika, odnosno korisnika. Kupac prije izvršenja kupnje mora raspolagati s dovoljnim iznosom tokena te pametnom ugovoru *RealEstateEscrow* odobriti korištenje potrebnog iznosa. Kada su svi uvjeti zadovoljeni kupnja se izvršava kao jedna blockchain transakcija. Pritom se simulirana sredstva prenose računa kupca na račun prodavatelja, digitalno vlasništvo se prenosi na kupca, a prodaja nekretnine se označava kao završena. Ako bilo koji dio postupka ne može biti uspješno proveden, transakcija se poništava u cijelosti čime se osigurava atomsko izvršenje kupoprodaje.

Za upravljanje posebnim ovlastima koristi se sustav uloga unutar pametnih ugovora. Uloga verifikatora je potvrđivanje ili odbijanje predane dokumentacije, dok je pravo prijenosa digitalnog vlasništva nad nekretninom dodijeljeno *escrow* pametnom ugovoru. Administrator upravlja potrebnim ulogama i simuliranim sredstvima, ali se poslovna pravila kupoprodaje ne oslanjaju na prikaz u korisničkom računu. Ako bi korisnik pokušao izravno pozvati nedopuštenu funkciju mimo korisničkog sučelja, pametni ugovor provodi vlastite provjere i odbija transakciju ako nisu zadovoljeni svi definirani osnovni uvjeti.

Cjelokupni tijek rada započinje registracijom nekretnine od strane korisnika koji postaje njezin početni digitalni vlasnik. Nakon registracije korisnik predaje sva tri obvezna dokumenta. Izvorne datoteke spremaju se u off-chain spremište, a njihove hash vrijednosti i URI adrese zapisuju se na blockchain. Verifikator putem korisničkog sučelja može otvoriti i pregledati predane dokumente te svaki od njih posebno prihvatiti ili odbiti. Nakon potvrde dokumenata nekretnina postaje spremna za prodaju te njezin vlasnik može kreirati prodaju i odrediti cijenu nekretnine. Drugi korisnik, odnosno kupac, može pregledati ponudu, provjeriti uvjete kupoprodaje, osigurati potreban iznos *MockEUR* tokena i odobriti njihovo korištenje *escrow* ugovoru. Kada su svi uvjeti zadovoljeni, pametni ugovor automatski izvršava prijenos sredstava i digitalnog vlasništva te evidentira završetak prodaje.

Opisani model predstavlja prototip tehničkog prijenosa digitalnog vlasništva unutar razvijenog blockchain sustava. Promjena podataka o digitalnom vlasniku na blockchainu u okviru ovog rada ne predstavlja sama po sebi pravno valjan prijenos vlasništva nad nekretninom u Republici Hrvatskoj, budući da se stvarno pravno vlasništva stječe u skladu s važećim pravnim propisima i odgovarajućim upisom u zemljišne knjige. Razvijena arhitektura služi za demonstraciju mogućnosti automatizacije pojedinih dijelova postupka kupoprodaje primjenom blockchain tehnologija i pametnih ugovora.

5. IMPLEMENTACIJA SUSTAVA

U ovom poglavlju opisana je implementacija razvijenog prototipa sustava za kupoprodaju nekretnina pomoću blockchain tehnologije i pametnih ugovora. Programsko rješenje sastoji se od pametnih ugovora napisanih u programskom jeziku *Solidity*, korisničkog sučelja razvijenog pomoću *React*-a i *TypeScript*-a te pomoćnog servisa za pohranu izvornih datoteka dokumentacije izvan blockchain mreže. Za razvoj, pokretanje lokalne blockchain mreže, implementaciju i ispitivanje pametnih ugovora korišteno je razvojno okruženje *Hardhat*.

Glavni dio poslovne logike sustava nalazi se u pametnim ugovorima. Njihova je zadaća evidentirati nekretnine i pripadajuću dokumentaciju, provjeriti sve uvjete potrebne za prodaju i kupnju, upravljati simuliranim financijskim sredstvima te nakon ispunjenja svih uvjeta provesti prijenos digitalnog vlasništva. Korisničko sučelje omogućuje jednostavnu interakciju s navedenim funkcionalnostima, dok se konačna provjera dopuštenih radnji provodi unutar pametnih ugovora.

5.1. Implementacija pametnih ugovora

Blockchain dio sustava sastoji se od tri glavna pametna ugovora: *PropertyRegistry*, *RealEstateEscrow* i *MockEUR*. Svaki od njih ima zasebnu odgovornost. *PropertyRegistry* služi za registraciju nekretnina, pohranu podataka o njihovim digitalnim vlasnicima te upravljanje dokumentacijom. *RealEstateEscrow* upravlja procesom prodaje i kupnje nekretnine te provjerava preduvjete potrebne za izvršenje transakcije. *MockEUR* predstavlja ERC-20 token kojim se u prototipu simuliraju financijska sredstva potrebna za kupnju nekretnine.

Razdvajanjem funkcionalnosti u više pametnih ugovora pojedini dijelovi poslovne logike ostaju jasno odvojeni. Registar nekretnina nije izravno zadužen za plaćanje, dok escrow ugovor ne pohranjuje cjelokupne podatke o nekretninama, nego ih prema potrebi dohvaća iz registra.

5.1.1. Pametni ugovor *PropertyRegistry*

Pametni ugovor *PropertyRegistry* predstavlja središnji registar nekretnina unutar razvijenog prototipa. U njemu se pohranjuju osnovni podaci o registriranim nekretninama, trenutni digitalni vlasnik te podaci povezani s obveznom dokumentacijom. Ugovor također sadrži pravila kojima se određuje tko smije verificirati dokumentaciju i tko smije izvršiti prijenos digitalnog vlasništva.

Za upravljanje ovlastima korišten je OpenZeppelinov ugovor *AccessControl*. Uz administratorsku ulogu definirane su uloge *VERIFIER_ROLE* i *TRANSFER_ROLE*. Uloga *VERIFIER_ROLE* namijenjena je računu koji provodi provjeru dokumentacije, dok *TRANSFER_ROLE* omogućuje promjenu digitalnog vlasnika nekretnine. Pravo prijenosa u konačnoj konfiguraciji sustava dodjeljuje se pametnom ugovoru *RealEstateEscrow*, čime se sprječava proizvoljna promjena vlasnika mimo definiranog postupka kupoprodaje.

```
PropertyRegistry.sol X
blockchain > contracts > PropertyRegistry.sol > ...
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.28;
3
4 import {AccessControl} from "@openzeppelin/contracts/access/AccessControl.sol";
5
6 /**
7  * @title PropertyRegistry
8  * @notice Blockchain registar nekretnina, pripadajuće dokumentacije
9  * i digitalnih vlasnika.
10  *
11  * Ovaj pametni ugovor predstavlja prototip i ne zamjenjuje
12  * službeni upis vlasništva u zemljišne knjige Republike Hrvatske.
13  */
14 contract PropertyRegistry is AccessControl {
15     bytes32 public constant VERIFIER_ROLE = keccak256("VERIFIER_ROLE");
16     bytes32 public constant TRANSFER_ROLE = keccak256("TRANSFER_ROLE");
17
18     /**
19     * @notice Broj obveznih vrsta dokumenata u prototipu.
20     */
21     uint8 public constant REQUIRED_DOCUMENT_COUNT = 3;
22
23     /**
24     * @notice Status provjere dokumenta ili ukupne dokumentacije nekretnine.
25     */
26     enum VerificationStatus {
27         Pending,
28         Verified,
29         Rejected
30     }
31
32     /**
33     * @notice Vrste dokumenata koje su obvezne prije prodaje nekretnine.
34     *
35     * Popis predstavlja prototip sustava.
36     * Konačna interpretacija dokumentacije obrađuje se
37     * u pravnom dijelu diplomskog rada.
38     */
39     enum DocumentType {
40         LandRegistryExtract,
41         CadastralDocument,
42         OwnershipDocument
43     }
44 }
```

Slika 5.1. Definicija uloga i enumeracija pametnog ugovora PropertyRegistry.sol

Status provjere dokumentacije definiran je enumeracijom *VerificationStatus*. Dokument može biti u stanju *Pending*, *Verified* i *Rejected*. Vrijednost *Pending* označava da je dokument predan, ali još nije provjeren. *Verified* označava da je dokument uspješno potvrđen, dok status *Rejected* označava dokument koji verifikator nije prihvatio.

Vrste obvezne dokumentacije definirane su enumeracijom *DocumentType*. U razvijenom prototipu za svaku nekretninu potrebno je predati tri dokumenta: zemljišnoknjižni izvadak, katastarski dokument te dokaz o vlasništvu nad nekretninom. Time je omogućeno da se svaki dokument evidentira i provjerava zasebno, umjesto da nekretnina ima samo jedan zajednički podatak o dokumentaciji.

Podaci o samoj nekretnini pohranjuju se u strukturi *Property*. Struktura sadrži jedinstveni identifikator nekretnine, katastarsku općinu, broj katastarske čestice, adresu nekretnine, adresu trenutnog digitalnog vlasnika, ukupni status dokumentacije i podatak kojim se određuje postoji li nekretnina u registru.

Za pojedinačni dokument koristi se zasebna struktura *PropertyDocument*. U njoj se pohranjuje kriptografska hash vrijednost dokumenta, URI adresa izvorne datoteke pohranjene izvan blockchaina, status njegove verifikacije i podatak o tome je li dokument predan.

```
45      /**
46       * @notice Dokument povezan s određenom nekretninom.
47       *
48       * Sadržaj dokumenta ne sprema se izravno na blockchain.
49       *
50       * documentHash predstavlja kriptografski hash sadržaja dokumenta
51       * i koristi se za provjeru integriteta.
52       *
53       * documentURI predstavlja referencu na dokument pohranjen
54       * izvan blockchaina, primjerice IPFS URI.
55       */
56      struct PropertyDocument {
57          bytes32 documentHash;
58          string documentURI;
59          VerificationStatus verificationStatus;
60          bool submitted;
61      }
62
63      /**
64       * @notice Podaci o nekretnini spremljenoj u registru.
65       */
66      struct Property {
67          uint256 id;
68          string cadastralMunicipality;
69          string parcelNumber;
70          string propertyAddress;
71          address digitalOwner;
72          VerificationStatus verificationStatus;
73          bool exists;
74      }
```

Slika 5.2. Strukture za pohranu podataka o nekretnini i dokumentaciji

Razdvajanjem osnovnih podataka o nekretnini i podataka o pojedinačnim dokumentima omogućeno je jednostavnije upravljanje dokumentacijom. Svaka registrirana nekretnina može imati zaseban zapis za svaki od tri obvezna dokumenta, pri čemu se njihov status može mijenjati neovisno o ostalim dokumentima.

Registracija nove nekretnine provodi se funkcijom *registerProperty*. Funkcija kao ulazne podatke prima katastarsku općinu, broj katastarske čestice i adresu nekretnine. Prije stvaranja novog zapisa provjerava jesu li uneseni svi obvezni podaci. Ako se bilo koji podatak ne popuni i ostane prazan izvršavanje transakcije se prekida pomoću naredbe *require*.

Osim provjere ulaznih podataka, implementirana je i zaštita od višestruke registracije iste nekretnine. U tu svrhu od katastarske općine i broja katastarske čestice stvara se jedinstveni ključ *parcelKey*. Vrijednosti se kodiraju pomoću funkcije *abi.encode*, nakon čega se nad njima izračunava *keccak256* hash vrijednost. Dobiveni ključ koristi se u strukturi *registeredParcels* kako bi se provjerilo postoji li ista katastarska čestica već upisana u registar. Ako je nekretnina prethodno registrirana, nova registracija se odbija.

```

174  /**
175   * @notice Registrira novu nekretninu u blockchain registru.
176   *
177   * Registracijom se još ne smatra da nekretnina ima
178   * valjanu dokumentaciju. Dokumenti se dostavljaju zasebno.
179   *
180   * @param cadastralMunicipality Katastarska općina.
181   * @param parcelNumber Broj katastarske čestice.
182   * @param propertyAddress Adresa nekretnine.
183   *
184   * @return propertyId Jedinstveni identifikator nekretnine.
185   */
186  function registerProperty(
187      string calldata cadastralMunicipality,
188      string calldata parcelNumber,
189      string calldata propertyAddress
190  ) external returns (uint256 propertyId) {
191      require(
192          bytes(cadastralMunicipality).length > 0,
193          "Katastarska općina je obavezna"
194      );
195
196      require(bytes(parcelNumber).length > 0, "Broj cestice je obavezan");
197
198      require(
199          bytes(propertyAddress).length > 0,
200          "Adresa nekretnine je obavezna"
201      );
202
203      bytes32 parcelKey = keccak256(
204          abi.encode(cadastralMunicipality, parcelNumber)
205      );
206
207      require(
208          !registeredParcels[parcelKey],
209          "Nekretnina je vec registrirana"
210      );
211
212      propertyId = nextPropertyId;
213
214      properties[propertyId] = Property({
215          id: propertyId,
216          cadastralMunicipality: cadastralMunicipality,
217          parcelNumber: parcelNumber,
218          propertyAddress: propertyAddress,
219          digitalOwner: msg.sender,
220          verificationStatus: VerificationStatus.Pending,
221          exists: true
222      });
223
224      registeredParcels[parcelKey] = true;
225
226      nextPropertyId++;
227
228      emit PropertyRegistered(
229          propertyId,
230          msg.sender,
231          cadastralMunicipality,
232          parcelNumber
233      );
234
235      return propertyId;
236  }

```

Slika 5.3. Funkcija za registraciju nekretnine

Kao što je prikazano na slici 5.3., nakon uspješnih provjera novoj nekretnini se dodjeljuje identifikator koji se sprema u varijablu *nextPropertyId*. Podaci se zatim zapisuju u strukturu *Property*, pri čemu blockchain adresa koja je pozvala funkciju (odnosno *msg.sender*), postaje početni digitalni vlasnik nekretnine. Ukupni status dokumentacije postavlja se na *Pending* budući da je u trenutku registracije nekretnine obvezna dokumentacija koja je predana nije potvrđena.

Nakon spremanja nekretnine njezin se *parcelKey* označava registriranim, a vrijednost *nextPropertyId* povećava se kako bi sljedeća nekretnina dobila novi jedinstveni identifikator. Na kraju funkcije emitira se događaj *PropertyRegistered* koji sadrži identifikator nekretnine, adresu digitalnog vlasnika, katastarsku općinu i broj katastarske čestice. Nakon toga registracija nekretnine ostaje evidentirana u zapisima blockchain transakcije.

Nakon registracije nekretnine njezin digitalni vlasnik može predati obveznu dokumentaciju pomoću funkcije *submitPropertyDocument*. Funkcija prima identifikator nekretnine, vrstu dokumenta, kriptografsku hash vrijednost dokumenta i URI adresu na kojoj je spremljena njegova izvorna datoteka. Na taj način sadržaj dokumenta se ne sprema izravno u blockchain stanje već se samo evidentiraju podaci koji omogućuju njegovo povezivanje s konkretnom nekretninom i naknadnom provjerom ispravnosti dokumenata.

Prije spremanja dokumenta pametni ugovor provodi nekoliko provjera. Prvo se provjerava postoji li nekretnina s navedenim identifikatorom i je li račun koji poziva funkciju njezin trenutni digitalni vlasnik. Nakon toga provjerava se je li predana hash vrijednost različita od nulte vrijednosti te je li URI dokumenta postavljen. Time se onemogućuje stvaranje nepotpunog zapisa o dokumentu.

```

237
238  /**
239   * @notice Predaje jedan od obveznih dokumenata nekretnine.
240   *
241   * Sadržaj dokumenta ne sprema se izravno na blockchain.
242   *
243   * Na blockchain se sprema:
244   *   kriptografski hash dokumenta
245   *   URI dokumenta pohranjenog izvan blockchaina
246   *
247   * Dokument može predati samo trenutni digitalni vlasnik.
248   *
249   * Odbijeni dokument moguće je ponovno predati.
250   * Već potvrđeni dokument nije moguće mijenjati.
251   *
252   * @param propertyId ID nekretnine.
253   * @param documentType Vrsta dokumenta.
254   * @param documentHash Kriptografski hash dokumenta.
255   * @param documentURI URI dokumenta pohranjenog izvan blockchaina.
256   */
257  function submitPropertyDocument(
258      uint256 propertyId,
259      DocumentType documentType,
260      bytes32 documentHash,
261      string calldata documentURI
262  ) external {
263      require(properties[propertyId].exists, "Nekretnina ne postoji");
264
265      require(
266          properties[propertyId].digitalOwner == msg.sender,
267          "Samo vlasnik može predati dokument"
268      );
269
270      require(documentHash != bytes32(0), "Hash dokumenta nije valjan");
271
272      require(bytes(documentURI).length > 0, "URI dokumenta je obavezan");
273
274      PropertyDocument storage document = propertyDocuments[propertyId][
275          documentType
276      ];
277
278      require(
279          document.verificationStatus != VerificationStatus.Verified,
280          "Potvrđeni dokument nije moguće mijenjati"
281      );
282
283      document.documentHash = documentHash;
284      document.documentURI = documentURI;
285      document.verificationStatus = VerificationStatus.Pending;
286      document.submitted = true;
287
288      _refreshPropertyVerificationStatus(propertyId);
289
290      emit PropertyDocumentSubmitted(
291          propertyId,
292          documentType,
293          documentHash,
294          documentURI,
295          msg.sender
296      );
297  }

```

Slika 5.4. Funkcija za predaju dokumentacije

Na slici 5.4. možemo vidjeti da se za odabranu vrstu dokumenta dohvaća pripadajuća struktura *PropertyDocument*. Ako je dokument već prethodno potvrđen, njegova izmjena više nije dopuštena. Ovim ograničenjem spriječena je izmjena sadržaja dokumenta nakon što je verifikator već potvrdio njegovu valjanost. Dokument koji je prethodno odbijen može se ponovo predati čime vlasnik nekretnine ima mogućnost dostavljanja ispravne ili novije verzije dokumenta.

Nakon uspješne predaje spremaju se *documentHash* i *documentURI*, status dokumenta postavlja se na *Pending*, a vrijednost *submitted* na *true*. Zatim se poziva interna funkcija *_refreshPropertyVerificationStatus* koja ponovno izračunava ukupni status dokumentacije nekretnine. Na kraju kako bi se na blockchainu evidentirala predaja određenog dokumenta emitira se *PropertyDocumentSubmitted*.

Na slici 5.5. prikazana je funkcija *verifyPropertyDocument*, dok je na slici 5.6. prikazana funkcija *rejectPropertyDocument*. Obje funkcije prije promjene statusa provjeravaju postoji li nekretnina, je li dokument prethodno predan i nalazi li se još uvijek u stanju *Pending*. Ako dokument više nije na čekanju, ponovna potvrda ili odbijanje nisu dopušteni. Time se osigurava jasno definiran tijek promjene statusa pojedinačnog dokumenta.

Pozivom funkcije *verifyPropertyDocument* status dokumenta mijenja se u *Verified*, nakon čega se emitira događaj *PropertyDocumentVerified*. Pozivom funkcije *rejectPropertyDocument* status dokumenta mijenja se u *Rejected*, nakon čega se emitira događaj *PropertyDocumentRejected*. Nakon jedne od operacija poziva se funkcija *_refreshPropertyVerificationStatus* kako bi se ukupni status dokumentacije nekretnine uskladio sa statusima svih pojedinačnih dokumenata.

Ako potvrđivanje pojedinačnog dokumenta uzrokuje da sva obvezna dokumentacija nekretnine postane valjana, emitira se događaj *PropertyVerified*. Isto tako, ako odbijanje dokumenta dovede do promjene ukupnog statusa dokumentacije na *Rejected*, emitira se događaj *PropertyRejected*. Time se osim statusa svakog pojedinačnog dokumenta evidentira i promjena ukupnog stanja dokumentacije nekretnine.

```

299      /**
300       * @notice Potvrđuje pojedinačni dokument nekretnine.
301       *
302       * Funkciju može pozvati samo korisnik s VERIFIER_ROLE ulogom.
303       *
304       * @param propertyId ID nekretnine.
305       * @param documentType Vrsta dokumenta.
306       */
307     function verifyPropertyDocument(
308         uint256 propertyId,
309         DocumentType documentType
310     ) external onlyRole(VERIFIER_ROLE) {
311         require(properties[propertyId].exists, "Nekretnina ne postoji");
312
313         PropertyDocument storage document = propertyDocuments[propertyId][
314             documentType
315         ];
316
317         require(document.submitted, "Dokument nije predan");
318
319         require(
320             document.verificationStatus == VerificationStatus.Pending,
321             "Dokument više nije na cekanju"
322         );
323
324         document.verificationStatus = VerificationStatus.Verified;
325
326         emit PropertyDocumentVerified(propertyId, documentType, msg.sender);
327
328         VerificationStatus previousStatus = properties[propertyId]
329             .verificationStatus;
330
331         _refreshPropertyVerificationStatus(propertyId);
332
333         if (
334             previousStatus != VerificationStatus.Verified &&
335             properties[propertyId].verificationStatus ==
336             VerificationStatus.Verified
337         ) {
338             emit PropertyVerified(propertyId, msg.sender);
339         }
340     }

```

Slika 5.5. Funkcija za potvrđivanje dokumentacije

```

342  /**
343   * @notice Odbija pojedinačni dokument nekretnine.
344   *
345   * Funkciju može pozvati samo korisnik s VERIFIER_ROLE ulogom.
346   *
347   * @param propertyId ID nekretnine.
348   * @param documentType Vrsta dokumenta.
349   */
350  function rejectPropertyDocument(
351      uint256 propertyId,
352      DocumentType documentType
353  ) external onlyRole(VERIFIER_ROLE) {
354      require(properties[propertyId].exists, "Nekretnina ne postoji");
355
356      PropertyDocument storage document = propertyDocuments[propertyId][
357          documentType
358      ];
359
360      require(document.submitted, "Dokument nije predan");
361
362      require(
363          document.verificationStatus == VerificationStatus.Pending,
364          "Dokument više nije na cekanju"
365      );
366
367      document.verificationStatus = VerificationStatus.Rejected;
368
369      emit PropertyDocumentRejected(propertyId, documentType, msg.sender);
370
371      VerificationStatus previousStatus = properties[propertyId]
372          .verificationStatus;
373
374      _refreshPropertyVerificationStatus(propertyId);
375
376      if (
377          previousStatus != VerificationStatus.Rejected &&
378          properties[propertyId].verificationStatus ==
379          VerificationStatus.Rejected
380      ) {
381          emit PropertyRejected(propertyId, msg.sender);
382      }
383  }

```

Slika 5.6. Funkcija za odbijanje dokumentacije

Osim statusa pojedinačnih dokumenata, pametni ugovor mora moći automatski odrediti je li dokumentacija nekretnine u potpunosti spremna za daljnji postupak prodaje. U tome pomažu dvije funkcije: *hasAllRequiredDocuments* i *hasValidDocuments*. Obje funkcije prolaze kroz sve vrste dokumenata definirane kao obveze u prototipu.

Funkcija *hasAllRequiredDocuments* provjerava je li za svaku od tri obvezne vrste dokumenta vrijednost *submitted* na vrijednosti *true*. Isključivo ako su predana sva tri dokumenta funkcija vraća vrijednost *true*. Ukoliko nije zadovoljen taj uvjet funkcija vraća vrijednost *false*.

Funkcija *hasValidDocuments* provodi strožu provjeru od funkcije *hasAllRequiredDocuments*. Osim što svi dokumenti moraju biti predani, također njihov status mora biti *Verified*. Isključivo ako su predana sva tri dokumenta funkcija vraća vrijednost *true*. Ukoliko nije zadovoljen taj uvjet funkcija vraća vrijednost *false*. Na taj način sama činjenica da je vlasnik predao dokumentaciju nije dovoljna za dopuštanje prodaje, nego svaki dokument mora zasebno proći postupak verifikacije.

Na slici 5.7. prikazane su funkcije *hasAllRequiredDocuments* i *hasValidDocuments*.

```

385  /**
386   * @notice Provjerava jesu li sva obvezna dokumenta predana.
387   *
388   * @param propertyId ID nekretnine.
389   *
390   * @return true ako su sva obvezna dokumenta predana.
391   */
392  function hasAllRequiredDocuments(
393      uint256 propertyId
394  ) public view returns (bool) {
395      require(properties[propertyId].exists, "Nekretnina ne postoji");
396
397      for (uint8 i = 0; i < REQUIRED_DOCUMENT_COUNT; i++) {
398          DocumentType documentType = DocumentType(i);
399
400          if (!propertyDocuments[propertyId][documentType].submitted) {
401              return false;
402          }
403      }
404
405      return true;
406  }
407
408  /**
409   * @notice Provjerava jesu li sva obvezna dokumenta
410   * predana i potvrđena.
411   *
412   * Ovo je ključna automatska provjera prije dopuštanja prodaje.
413   *
414   * @param propertyId ID nekretnine.
415   *
416   * @return true ako sva obvezna dokumentacija postoji
417   * i svaki dokument ima status Verified.
418   */
419  function hasValidDocuments(uint256 propertyId) public view returns (bool) {
420      require(properties[propertyId].exists, "Nekretnina ne postoji");
421
422      for (uint8 i = 0; i < REQUIRED_DOCUMENT_COUNT; i++) {
423          DocumentType documentType = DocumentType(i);
424
425          PropertyDocument storage document = propertyDocuments[propertyId][
426              documentType
427          ];
428
429          if (
430              !document.submitted ||
431              document.verificationStatus != VerificationStatus.Verified
432          ) {
433              return false;
434          }
435      }
436
437      return true;
438  }

```

Slika 5.7. Funkcije za provjeru potpunosti i valjanosti dokumentacije

Objekti funkcije predstavljaju važan dio poslovne logike sustava jer odluka o tome je li dokumentacija nekretnine valjana nije prepuštena korisničkom sučelju. Korisničko sučelje samo dohvaća i prikazuje rezultat provjere s blockchaina, dok pametni ugovori donose konačnu odluku na temelju spremljenog stanja svih obveznih dokumenata.

Uz navedene funkcije implementirana je i privatna funkcija `_refreshPropertyVerificationStatus` koja je prikazana na slici 5.8. čija je zadaća automatski odrediti ukupni status dokumentacije nekretnine. Funkcija se poziva nakon predaje, potvrđivanja ili odbijanja pojedinačnog dokumenta, odnosno svaki put kada bilo kakva promjena stanja jednog dokumenta može utjecati na ukupno stanje nekretnine.

```
534  /**
535   * @dev Automatski određuje ukupni status dokumentacije nekretnine.
536   *
537   * Pravila:
538   * ako je barem jedan predani dokument odbijen -> Rejected
539   * ako sva obvezna dokumenta postoje i potvrđena su -> Verified
540   * u svim ostalim slučajevima -> Pending
541   */
542  function _refreshPropertyVerificationStatus(uint256 propertyId) private {
543      VerificationStatus previousStatus = properties[propertyId]
544          .verificationStatus;
545
546      bool allDocumentsVerified = true;
547      bool hasRejectedDocument = false;
548
549      for (uint8 i = 0; i < REQUIRED_DOCUMENT_COUNT; i++) {
550          DocumentType documentType = DocumentType(i);
551
552          PropertyDocument storage document = propertyDocuments[propertyId][
553              documentType
554          ];
555
556          if (
557              document.submitted &&
558              document.verificationStatus == VerificationStatus.Rejected
559          ) {
560              hasRejectedDocument = true;
561          }
562
563          if (
564              !document.submitted ||
565              document.verificationStatus != VerificationStatus.Verified
566          ) {
567              allDocumentsVerified = false;
568          }
569      }
570
571      VerificationStatus newStatus;
572
573      if (hasRejectedDocument) {
574          newStatus = VerificationStatus.Rejected;
575      } else if (allDocumentsVerified) {
576          newStatus = VerificationStatus.Verified;
577      } else {
578          newStatus = VerificationStatus.Pending;
579      }
580
581      properties[propertyId].verificationStatus = newStatus;
582
583      if (previousStatus != newStatus) {
584          emit PropertyVerificationStatusChanged(
585              propertyId,
586              previousStatus,
587              newStatus
588          );
589      }
590  }
```

Slika 5.8. Automatsko određivanje ukupnog statusa dokumentacije nekretnine

Funkcija prolazi kroz sva tri obvezna dokumenta i određuje dvije informacije: jesu li svi dokumenti potvrđeni i postoji li među njima barem jedan odbijeni dokument. Ako postoji odbijeni dokument ukupni status dokumentacije postavlja se na *Rejected*. Ako nema odbijenih dokumenata, a svi dokumenti su predani i potvrđeni status postaje *Verified*. U svim ostalim slučajevima dokumentacija ostaje u stanju *Pending*.

Time se omogućuje automatska promjena ukupnog stanja ovisno o stanju pojedinačnih dokumenata. Primjer: predaja svih triju dokumenata nije dovoljna za dobivanje statusa *Verified* sve dok ih verifikator pojedinačno ne potvrdi. Isto tako odbijanje jednog dokumenta dovoljno je da ukupni status postane *Rejected*. Ako vlasnik nakon toga ponovo preda ispravljenu verziju dokumenta koji je odbijen, njegov status ponovno postaje *Pending* i ukupno stanje dokumentacije se ponovno izračunava.

Ako se novodobiveni ukupni status razlikuje od prethodnog statusa, emitira se događaj *PropertyVerificationStatusChanged*. Time se promjene ukupnog statusa mogu pratiti putem blockchain zapisa.

Na slici 5.9. prikazana je funkcija *transferPropertyOwnership*. Njezin poziv nije moguć s bilo kojeg računa, nego je za njezino izvršavanje potrebna uloga *TRANSFER_ROLE*. U razvijenom sustavu ta se uloga dodjeljuje pametnom ugovoru *RealEstateEscrow* koji prijenos pokreće kao dio uspješno izvršene kupoprodaje.

```
458      /**
459       * @notice Mijenja digitalnog vlasnika nekretnine.
460       *
461       * Prijenos je moguć samo ako su svi obvezni dokumenti
462       * predani i potvrđeni.
463       *
464       * Funkciju može pozvati samo adresa s TRANSFER_ROLE ulogom.
465       * U stvarnom toku prototipa tu ulogu ima RealEstateEscrow pametni ugovor.
466       *
467       * @param propertyId Jedinstveni identifikator nekretnine.
468       * @param newOwner Adresa novog digitalnog vlasnika.
469       */
470      function transferPropertyOwnership(
471          uint256 propertyId,
472          address newOwner
473      ) external onlyRole(TRANSFER_ROLE) {
474          require(properties[propertyId].exists, "Nekretnina ne postoji");
475
476          require(
477              isValidDocuments(propertyId),
478              "Dokumentacija nekretnine nije valjana"
479          );
480
481          require(newOwner != address(0), "Adresa novog vlasnika nije valjana");
482
483          address previousOwner = properties[propertyId].digitalOwner;
484
485          require(
486              previousOwner != newOwner,
487              "Novi vlasnik je vec trenutni vlasnik"
488          );
489
490          properties[propertyId].digitalOwner = newOwner;
491
492          emit PropertyOwnershipTransferred(propertyId, previousOwner, newOwner);
493      }
```

Slika 5.9. Funkcija za prijenos digitalnog vlasništva nad nekretninom

Prije promjene vlasnika ugovor ponovno provjerava postoji li nekretnina i ima li potpuno potvrđenu svu dokumentaciju. Time se prijenos ne može izvršiti nad nekretninom čiji dokumenti nisu predani ili nisu potvrđeni. Također se provjerava da adresa novog vlasnika nije nulta adresa i da se razlikuje od adrese trenutnog digitalnog vlasnika.

Nakon zadovoljenja svih uvjeta adresa trenutnog vlasnika sprema se u varijablu *previousOwner*, a vrijednost *digitalOwner* mijenja se na adresu novog vlasnika. Promjena se zatim evidentira pomoću *PropertyOwnershipTransferred*, koji sadrži identifikator nekretnine te adresu prethodnog i novog digitalnog vlasnika.

Ograničavanjem funkcije ulogom *TRANSFER_ROLE* osigurava se da korisnik ne može samostalno promijeniti digitalnog vlasnika nekretnine mimo definiranog tijeka kupoprodaje. U razvijenom prototipu prijenos zato predstavlja završni dio procesa kojim upravlja pametni ugovor *RealEstateEscrow* čija je implementacija objašnjena u potpoglavlju 5.1.2.

5.1.2. Pametni ugovor *RealEstateEscrow*

Pametni ugovor *RealEstateEscrow* upravlja procesom prodaje i kupnje registriranih nekretnina. Za razliku od ugovora *PropertyRegistry*, koji predstavlja registar nekretnina i njihove dokumentacije, *RealEstateEscrow* zadužen je za upravljanje pojedinačnim prodajama, provjeru uvjeta kupoprodaje, prijenos simuliranih financijskih sredstava i pokretanje procesa prijenosa digitalnog vlasništva.

Prilikom postavljanja ugovora povezuju se adresa pametnog ugovora *PropertyRegistry* i adresa ERC-20 tokena *MockEUR*. Na taj način escrow ugovor može tijekom izvršavanja provjeriti podatke o nekretninama i njihovim vlasnicima te pristupiti simuliranim financijskim sredstvima koja se koriste za kupoprodaju.

Stanje pojedine prodaje definirano je enumeracijom *SaleStatus*. Prodaja može imati status *Created*, *Funded*, *Completed* ili *Cancelled*. Status *Created* označava aktivno kreiranu prodaju koja još nije financirana. *Funded* predstavlja prijelazno stanje tijekom izvršenja kupoprodajne transakcije nakon čega prelazi u stanje *Completed*. Status *Cancelled* koristi se kada prodavatelj odustane od prodaje prije nego je kupoprodaja izvršena.

Podaci o prodaji spremljeni su u strukturi *Sale*. Za svaku prodaju evidentiraju se njezin jedinstveni identifikator, identifikator nekretnine koja se prodaje, adresa prodavatelja, adresa kupca, prodajna

cijena, trenutni status i podatak postoji li prodaja. U trenutku kreiranja prodaje kupac još nije upoznat pa se za njegovu adresu postavlja adresa nulte blockchain adrese.

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.28;
3
4 import {IERC20} from "@openzeppelin/contracts/token/ERC20/IERC20.sol";
5 import {SafeERC20} from "@openzeppelin/contracts/token/ERC20/Utils/SafeERC20.sol";
6 import {ReentrancyGuard} from "@openzeppelin/contracts/Utils/ReentrancyGuard.sol";
7
8 import {IPropertyRegistry} from "../interfaces/IPropertyRegistry.sol";
9
10 /**
11  * @title RealEstateEscrow
12  * @notice Escrow pametni ugovor za simuliranu kupoprodaju nekretnina.
13  *
14  * Ugovor provjerava jesu li zadovoljeni uvjeti kupoprodaje,
15  * zaprima sredstva kupca i automatski izvršava prijenos
16  * digitalnog vlasništva i sredstava prodavatelju.
17  *
18  * Digitalni prijenos vlasništva ne predstavlja stvarni
19  * zemljišnoknjižni upis u Republici Hrvatskoj.
20  */
21 contract RealEstateEscrow is ReentrancyGuard {
22     using SafeERC20 for IERC20;
23
24     /**
25      * @notice Status kupoprodajne transakcije.
26      */
27     enum SaleStatus {
28         Created,
29         Funded,
30         Completed,
31         Cancelled
32     }
33
34     /**
35      * @notice Podaci o jednoj kupoprodajnoj transakciji.
36      */
37     struct Sale {
38         uint256 id;
39         uint256 propertyId;
40         address seller;
41         address buyer;
42         uint256 price;
43         SaleStatus status;
44         bool exists;
45     }
46
47     /**
48      * @notice Rezultat automatske provjere svih uvjeta
49      * potrebnih za izvršenje kupoprodaje.
50      *
51      * Struktura se može dohvatiti s frontenda prije pokušaja
52      * kupnje kako bi korisnik jasno vidio koji su uvjeti
53      * zadovoljeni, a koji nisu.
54      */
55     struct PurchaseConditions {
56         bool saleExists;
57         bool saleActive;
58         bool documentsValid;
59         bool sellerIsOwner;
60         bool buyerIsNotSeller;
61         bool buyerHasSufficientBalance;
62         bool buyerHasSufficientAllowance;
63         bool readyForPurchase;
64     }
```

Slika 5.10. Definicija statusa i strukture prodaje u pametnom ugovoru *RealEstateEscrow* – 1. dio

```

66      /**
67       * @notice PropertyRegistry pametni ugovor.
68       */
69      IPropertyRegistry public immutable propertyRegistry;
70
71      /**
72       * @notice ERC-20 token koji se koristi za simulirano plaćanje.
73       */
74      IERC20 public immutable paymentToken;
75
76      /**
77       * @notice ID sljedeće kupoprodajne transakcije.
78       */
79      uint256 private nextSaleId = 1;
80
81      /**
82       * @notice Kupoprodajne transakcije spremljene prema njihovu ID-u.
83       */
84      mapping(uint256 => Sale) private sales;
85
86      /**
87       * @notice Aktivna kupoprodaja za pojedinu nekretninu.
88       *
89       * Vrijednost 0 znači da nekretnina trenutno nema aktivnu prodaju.
90       */
91      mapping(uint256 => uint256) private activeSaleByProperty;

```

Slika 5.11. Definicija statusa i strukture prodaje u pametnom ugovoru RealEstateEscrow – 2. dio

Za svaku nekretninu potrebno je dodatno pratiti postoji li već aktivna prodaja. U tu svrhu koristi se evidencija *activeSaleByProperty*, koja povezuje identifikator nekretnine s identifikatorom njezine trenutno aktivne prodaje. Vrijednost nula označava da nekretnina u tom trenutku nema aktivnu prodaju. Ovakav pristup sprječava istodobno stvaranje više aktivnih prodaja za istu nekretninu.

Kreiranje nove prodaje provodi se funkcijom *createSale* kojoj se predaju identifikator nekretnine i prodajna cijena. Pametni ugovor prije stvaranja prodaje provodi više provjera kako bi spriječio stvaranje neispravne ponude. Prodajna cijena mora biti veća od nule, korisnik koji pokreće transakciju mora biti trenutni vlasnik nekretnine, dokumentacija nekretnine mora biti potpuno potvrđena i ista nekretnina ne smije imati već aktivnu drugu prodaju.

```

151  /**
152  * @notice Kreira novu ponudu za prodaju nekretnine.
153  *
154  * Prodaja se može kreirati samo ako:
155  *   cijena je veća od nule
156  *   dokumentacija nekretnine je valjana
157  *   pozivatelj je trenutni digitalni vlasnik
158  *   nekretnina nema drugu aktivnu prodaju
159  *
160  * @param propertyId Jedinstveni identifikator nekretnine.
161  * @param price Prodajna cijena u najmanjim jedinicama mEUR tokena.
162  *
163  * @return saleId Jedinstveni identifikator kupoprodaje.
164  */
165  function createSale(
166      uint256 propertyId,
167      uint256 price
168  ) external returns (uint256 saleId) {
169      require(price > 0, "Cijena mora biti veka od nule");
170
171      require(
172          propertyRegistry.isPropertyVerified(propertyId),
173          "Dokumentacija nekretnine nije valjana"
174      );
175
176      address currentOwner = propertyRegistry.getDigitalOwner(propertyId);
177
178      require(
179          currentOwner == msg.sender,
180          "Samo vlasnik može kreirati prodaju"
181      );
182
183      require(
184          activeSaleByProperty[propertyId] == 0,
185          "Nekretnina već ima aktivnu prodaju"
186      );
187
188      saleId = nextSaleId;
189
190      sales[saleId] = Sale({
191          id: saleId,
192          propertyId: propertyId,
193          seller: msg.sender,
194          buyer: address(0),
195          price: price,
196          status: SaleStatus.Created,
197          exists: true
198      });
199
200      activeSaleByProperty[propertyId] = saleId;
201
202      nextSaleId++;
203
204      emit SaleCreated(saleId, propertyId, msg.sender, price);
205
206      return saleId;
207  }

```

Slika 5.12. Funkcija za kreiranje prodaje nekretnine

Nakon što su sve provjere uspješno zadovoljene, prodaji se dodjeljuje novi jedinstveni identifikator. Adresa računa koji je pokrenuo akciju sprema se kao prodavatelj, dok se adresa kupca inicijalno postavlja na nultu adresu jer u trenutku kreiranja ponude kupac još nije poznat. Prodaja se sprema sa statusom *Created* čime postaje aktivna i dostupna drugim korisnicima sustava.

Identifikator novostvorene prodaje zatim se povezuje s nekretninom pomoću *activeSaleByProperty*. Time pametni ugovor može izravno utvrditi ima li određena nekretnina već aktivnu prodaju i odbiti pokušaj stvaranja nove ponude sve dok se postojeća ne završi ili ne otkáže. Na kraju funkcije emitira se događaj *SaleCreated* kojim se stvaranje prodaje evidentira u blockchain zapisima.

Prodavatelju je omogućeno i otkazivanje aktivne prodaje prije izvršenja kupoprodaje pomoću funkcije *cancelSale*. Otkazivanje može provesti samo račun koji je evidentiran kao prodavatelj konkretne prodaje, a prodaja se pritom mora i dalje nalaziti u statusu *Created*.

```
308      /**
309       * @notice Otkazuje prodaju prije polaganja sredstava.
310       *
311       * Prodaju može otkazati samo prodavatelj dok je
312       * transakcija u statusu Created.
313       *
314       * @param saleId Jedinstveni identifikator kupoprodaje.
315       */
316      function cancelSale(uint256 saleId) external {
317          Sale storage sale = sales[saleId];
318
319          require(sale.exists, "Prodaja ne postoji");
320
321          require(
322              sale.seller == msg.sender,
323              "Samo prodavatelj može otkazati prodaju"
324          );
325
326          require(
327              sale.status == SaleStatus.Created,
328              "Prodaju više nije moguće otkazati"
329          );
330
331          sale.status = SaleStatus.Cancelled;
332
333          activeSaleByProperty[sale.propertyId] = 0;
334
335          emit SaleCancelled(saleId, sale.propertyId, msg.sender);
336      }
337
338      /**
339       * @notice Vraća ukupan broj kreiranih kupoprodajnih transakcija.
340       */
341      function getSaleCount() external view returns (uint256) {
342          return nextSaleId - 1;
343      }
```

Slika 5.13. Funkcija za otkazivanje aktivne prodaje

Nakon uspješnog otkazivanja prodaje status iste mijenja se u *Cancelled*, ali zapis o samoj prodaji ostaje pohranjen u blockchainu. Time se zadržava povijest prethodno kreiranih i otkazanih prodaja. U isto vrijeme se uklanja veza aktivne prodaje s nekretninom, čime se njezinu digitalnom vlasniku ponovo omogućuje stvaranje nove prodaje.

Ovakvim načinom upravljanja prodajama osigurava se da samo trenutačni digitalni vlasnik nekretnine s potpunom potvrđenom dokumentacijom može ponuditi nekretninu na prodaju te da za istu nekretninu u jednom trenutku ne može postojati više od jedne aktivne ponude.

Prije izvršavanja kupoprodaje potrebno je provjeriti ispunjavaju li prodaja, nekretnina i kupac sve definirane uvjete. Provjeru izvršava struktura *PurchaseConditions* koja sadrži rezultate pojedinačnih provjera te završnu vrijednost *readyForPurchase* koja označava jesu li svi uvjeti za izvršenje kupnje istodobno zadovoljeni.

Provjerava se više stvari: postojanje prodaje, njezin trenutni status, valjanost dokumentacije nekretnine, podudaranje evidentiranog prodavatelja s trenutnim digitalnim vlasnikom, različitost adresa kupca i prodavatelja te raspolaže li kupac dovoljnim stanjem i odobrenjem *MockEUR* tokena. Na taj način uvjeti kupnje definirani su na jednom mjestu unutar pametnog ugovora. Na slici 5.14. je prikazano opisano.


```

47      /**
48       * @notice Rezultat automatske provjere svih uvjeta
49       * potrebnih za izvršenje kupoprodaje.
50       *
51       * Struktura se može dohvatiti s frontenda prije pokušaja
52       * kupnje kako bi korisnik jasno vidio koji su uvjeti
53       * zadovoljeni, a koji nisu.
54       */
55      struct PurchaseConditions {
56          bool saleExists;
57          bool saleActive;
58          bool documentsValid;
59          bool sellerIsOwner;
60          bool buyerIsNotSeller;
61          bool buyerHasSufficientBalance;
62          bool buyerHasSufficientAllowance;
63          bool readyForPurchase;
64      }

```

```

354      /**
355       * @dev Interna funkcija koja centralno provjerava
356       * sve uvjete potrebne za izvršenje kupoprodaje.
357       *
358       * Ova funkcija predstavlja jedinstveni izvor istine
359       * za provjeru preduvjeta kupoprodaje.
360       */
361      function _getPurchaseConditions(
362          uint256 saleId,
363          address buyer
364      ) private view returns (PurchaseConditions memory conditions) {
365          Sale storage sale = sales[saleId];
366
367          /**
368           * 1. Prodaja mora postojati.
369           */
370          conditions.saleExists = sale.exists;
371
372          /**
373           * Ako prodaja ne postoji, ostali uvjeti nemaju smisla
374           * i vraća se struktura s false vrijednostima.
375           */
376          if (!conditions.saleExists) {
377              return conditions;
378          }
379
380          /**
381           * 2. Prodaja mora biti u statusu Created
382           * i mora biti evidentirana kao aktivna prodaja
383           * za navedenu nekretninu.
384           */
385          conditions.saleActive =
386              sale.status == SaleStatus.Created &&
387              activeSaleByProperty[sale.propertyId] == saleId;
388
389          /**
390           * 3. Sva obvezna dokumentacija nekretnine
391           * mora biti predana i potvrđena.
392           */
393          conditions.documentsValid = propertyRegistry.isPropertyVerified(
394              sale.propertyId
395          );
396
397          /**
398           * 4. Prodavatelj mora i dalje biti trenutačni
399           * digitalni vlasnik nekretnine.
400           */
401          address currentOwner = propertyRegistry.getDigitalOwner(
402              sale.propertyId
403          );
404
405          conditions.sellerIsOwner = currentOwner == sale.seller;
406
407          /**
408           * 5. Kupac ne smije biti prodavatelj.
409           */
410          conditions.buyerIsNotSeller =
411              buyer != address(0) &&
412              buyer != sale.seller;

```

```

414      /**
415       * 6. Kupac mora raspolagati dovoljnim
416       * brojem mEUR tokena.
417       */
418      uint256 buyerBalance = paymentToken.balanceOf(buyer);
419
420      conditions.buyerHasSufficientBalance = buyerBalance >= sale.price;
421
422      /**
423       * 7. Kupac mora escrow ugovoru odobriti
424       * dovoljan iznos tokena.
425       */
426      uint256 buyerAllowance = paymentToken.allowance(buyer, address(this));
427
428      conditions.buyerHasSufficientAllowance = buyerAllowance >= sale.price;
429
430      /**
431       * Konačna odluka.
432       *
433       * Kupoprodaja je spremna za izvršenje
434       * samo ako su SVI uvjeti zadovoljeni.
435       */
436      conditions.readyForPurchase =
437          conditions.saleExists &&
438          conditions.saleActive &&
439          conditions.documentsValid &&
440          conditions.sellerIsOwner &&
441          conditions.buyerIsNotSeller &&
442          conditions.buyerHasSufficientBalance &&
443          conditions.buyerHasSufficientAllowance;
444
445      return conditions;
446      }

```

Slika 5.14. Automatska provjera uvjeta potrebnih za kupnju nekretnine

Funkcija *getPurchaseConditions* najprije provjerava postoji li prodaja s predanim identifikatorom i ako ne postoji vrijednost *saleExists* ostaje netočna, a ostali uvjeti ne mogu dovesti do dopuštenja kupnje. Za postojeću prodaju provjerava se nalazi li se ona u statusu *Created*, odnosno je li ona još uvijek aktivna i dostupna za prodaju.

Valjanost dokumentacije provjerava se pozivom funkcije *hasValidDocuments* pametnog ugovora *PropertyRegistry*. Time se koristi prethodno implementirana provjera prema kojoj sva tri obvezna dokumenta moraju biti predana i imati status *Verified*. Ako dokumentacija više nije valjana kupnja se ne može izvršiti.

Dodatno se provjerava je li adresa spremljena kao prodavatelj i dalje trenutni digitalni vlasnik nekretnine. Ova provjera je bitna jer se stanje vlasništva može promijeniti između trenutka stvaranja prodaje i pokušaja izvršenja kupnje. Na taj način nije dovoljno da je korisnik bio vlasnik u trenutku stvaranja ponude, nego to mora biti i neposredno prije kupoprodaje.

Sljedeći uvjet koji mora biti zadovoljen je da kupac i prodavatelj nisu ista blockchain adresa. Time se sprječava da korisnik kupuje nekretninu sam od sebe.

Financijski uvjeti provjeravaju se pomoću ERC-20 tokena *MockEUR*. Vrijednost *buyerHasSufficientBalance* određuje ima li kupac na svojoj adresi najmanje dovoljno tokena koliko iznosi cijena nekretnine koju je postavio prodavatelj. Međutim, samo imanje dovoljnog broja tokena nije dovoljno za izvršavanje kupnje jer pametni ugovor ne smije samostalno raspolagati tokenima korisnika. Zbog toga postoji dodatna provjera *allowance*, odnosno iznosa koji je kupac prethodno odobrio pametnom ugovoru *RealEstateEscrow* na korištenje.

Završna vrijednost *readyForPurchase* postavlja se na *true* samo isključivo onda kada su svi navedeni uvjeti istodobno zadovoljeni. Ako samo jedan uvjet nije ispunjen vrijednost ostaje *false* te kupoprodajna transakcija nije spremna za izvršenje.

Implementacija ove funkcije omogućuje da ista pravila koristi i korisničko sučelje prije pokretanja kupnje. Kupcu se tako može prikazati koji je uvjet zadovoljen, a koji nije. Međutim, prikaz rezultata u korisničkom sučelju ima samo informativnu ulogu, dok se stvarna provjera ponovno provodi unutar pametnog ugovora prilikom izvršenja kupnje. Time se sigurnost sustava ne oslanja na ispravnost korisničkog sučelja.

Nakon što su svi preduvjeti zadovoljeni kupac može pokrenuti kupoprodaju pozivom funkcije *fundSale*. Funkcija predstavlja glavnu ulaznu točku za izvršenje kupnje nekretnine te mijenja stanje blockchaina zbog čega transakciju mora potpisati kupac svojim blockchain računom.

Iako korisničko sučelje prije kupnje prikazuje rezultate funkcije *getPurchaseConditions*, pametni ugovor se ne oslanja na prethodno stanje prikazano u korisničkom sučelju. Funkcija *fundSale* ponovo provjerava uvjete u trenutku izvršenja transakcije čime se postiže mogućnost da korisnik zaobiđe ograničenja manipulacijom korisničkog sučelja ili da se kupnja izvrši na temelju zastarjelih podataka.

Posebno se provjerava raspolaže li kupac dovoljnim brojem *MockEUR* tokena za plaćanje prodajne cijene te je li pametnom ugovoru *RealEstateEscrow* prethodno odobrio korištenje dovoljnog iznosa tokena. Ako stanje kupca nije dovoljno, transakcija se prekida. Ukoliko dodijeljeni *allowance* ne pokriva cjelokupnu prodajnu cijenu, kupnja se odbija.

Cijela funkcija *fundSale* prikazana je na slici 5.15.

```

229  /**
230   * @notice Kupac polaže puni iznos kupoprodajne cijene.
231   *
232   * Prije prijenosa sredstava pametni ugovor automatski
233   * provjerava sve uvjete potrebne za izvršenje kupoprodaje.
234   *
235   * Kada su svi uvjeti zadovoljeni, sredstva se prebacuju
236   * u escrow te se kupoprodaja automatski završava.
237   *
238   * @param saleId Jedinstveni identifikator kupoprodaje.
239   */
240  function fundSale(uint256 saleId) external nonReentrant {
241      PurchaseConditions memory conditions = _getPurchaseConditions(
242          saleId,
243          msg.sender
244      );
245
246      /**
247       * Svaki uvjet ima vlastitu poruku greške kako bi bilo
248       * potpuno jasno zašto transakcija nije dopuštena.
249       */
250
251      require(conditions.saleExists, "Prodaja ne postoji");
252
253      require(conditions.saleActive, "Prodaja nije dostupna za uplatu");
254
255      require(conditions.buyerIsNotSeller, "Prodavatelj ne može biti kupac");
256
257      require(
258          conditions.documentsValid,
259          "Dokumentacija nekretnine nije valjana"
260      );
261
262      require(
263          conditions.sellerIsOwner,
264          "Prodavatelj više nije vlasnik nekretnine"
265      );
266
267      require(
268          conditions.buyerHasSufficientBalance,
269          "Kupac nema dovoljno sredstava"
270      );
271
272      require(
273          conditions.buyerHasSufficientAllowance,
274          "Kupac nije odobrio dovoljan iznos sredstava"
275      );
276
277      /**
278       * Ako smo došli do ove točke,
279       * svi uvjeti moraju biti zadovoljeni.
280       */
281      require(
282          conditions.readyForPurchase,
283          "Uvjeti za kupoprodaju nisu zadovoljeni"
284      );
285
286      Sale storage sale = sales[saleId];
287
288      /**
289       * Tek nakon potvrde svih uvjeta sredstva
290       * se prenose s kupca u escrow.
291       */
292      paymentToken.safeTransferFrom(msg.sender, address(this), sale.price);
293
294      sale.buyer = msg.sender;
295      sale.status = SaleStatus.Funded;
296
297      emit SaleFunded(saleId, msg.sender, sale.price);
298
299      /**
300       * Nema ručnog posrednika koji završava kupoprodaju.
301       *
302       * Smart contract odmah i automatski izvršava
303       * završetak kupoprodaje.
304       */
305      _completeSale(saleId);
306  }

```

Slika 5.15. Funkcija za financiranje i pokretanje kupoprodaje nekretnine

Kada su sve provjere zadovoljene, adresa računa koji je pozvao funkciju evidentira se kao kupac određene nekretnine koja je na prodaju. Status prodaje privremeno se mijenja iz *Created* u *Funded*. U prototipu sustava status *Funded* nije dugotrajno stanje u kojem sredstva ostaju zaključana u *escrow* ugovoru već predstavlja prijelazni korak unutar iste blockchain transakcije kojom se kupoprodaja dovršava.

Za prijenos ERC-20 sredstava koristi se mehanizam prethodnog odobrenja. Kupac najprije funkcijom *approve* dopušta *escrow* ugovoru korištenje određenog iznosa svojih tokena. Nakon toga *RealEstateEscrow* može tijekom izvršenja kupoprodaje prenijeti iznos jednak prodajnoj cijeni. Na taj način pametni ugovor ne može samostalno pristupiti neograničenom broju tokena korisnika, odnosno kupca, nego samo onom iznosu koji mu je korisnik, odnosno kupac, prethodno odobrio.

Nakon financijskog dijela provodi se završetak kupoprodaje. U sklopu istog postupka pametni ugovor pokreće prijenos digitalnog vlasništva nad nekretninom putem ugovora *PropertyRegistry*, prenosi ugovorena sredstva prodavatelju, prodaju označava završenom i uklanja nekretninu iz evidencije aktivnih prodaja što je prikazano slikom 5.16.

```
448      /**
449       * @dev Automatski završava financiranu kupoprodaju.
450       *
451       * Funkcija se poziva isključivo iz fundSale nakon što su
452       * svi uvjeti provjereni i puni iznos uspješno prenesen u escrow.
453       *
454       * Ako prijenos vlasništva ili isplata ne uspiju,
455       * revertira se cijela blockchain transakcija.
456       */
457     function _completeSale(uint256 saleId) private {
458         Sale storage sale = sales[saleId];
459
460         require(
461             sale.status == SaleStatus.Funded,
462             "Prodaja nije spremna za završetak"
463         );
464
465         sale.status = SaleStatus.Completed;
466
467         activeSaleByProperty[sale.propertyId] = 0;
468
469         /**
470          * Automatski digitalni prijenos vlasništva.
471          */
472         propertyRegistry.transferPropertyOwnership(sale.propertyId, sale.buyer);
473
474         /**
475          * Automatska isplata punog iznosa prodavatelju.
476          */
477         paymentToken.safeTransfer(sale.seller, sale.price);
478
479         emit SaleCompleted(
480             saleId,
481             sale.propertyId,
482             sale.buyer,
483             sale.seller,
484             sale.price
485         );
486     }
```

Slika 5.16. Završetak kupoprodaje i prijenos digitalnog vlasništva

Nakon uspješnog izvršenja prodavatelj prima ugovoreni iznos *MockEUR* tokena, dok kupac postaje novi *digitalOwner* nekretnine u ugovoru *PropertyRegistry*. Prodaja dobiva status *Completed*, a zapis aktivne prodaje za pripadajuću nekretninu se uklanja iz *activeSaleByProperty*. Time kupljena nekretnina više nije dostupna u popisu aktivnih prodaja, ali završena prodaja ostaje trajno evidentirana u strukturi prodaja i prikazuje se u povijesti transakcija.

Važno svojstvo ovakvog načina izvršavanja je atomskost blockchain transakcije. To znači da se sve operacije koje čine kupoprodaju izvršavaju kao jedna nedjeljiva cjelina. Prijenos sredstava i prijenos digitalnog vlasništva stoga ne predstavljaju dvije neovisne radnje koje mogu završiti različitim rezultatima. Ako bilo koji korak kupoprodaje ne uspije, izvršavanje cijele transakcije se poništava, a stanje blockchaina vraća se na stanje prije njezina pokretanja.

Primjer takvog slučaja je situacija u kojoj pametnom ugovoru *RealEstateEscrow* nije dodijeljena uloga *TRANSFER_ROLE*. U tom slučaju *PropertyRegistry* odbija zahtjev za promjenom digitalnog vlasnika. Budući da se sve radnje izvršavaju u okviru iste transakcije, poništavaju se i prethodne promjene stanja. Kupac zadržava svoja sredstva, prodavatelj ne prima tokene, digitalni vlasnik ostaje nepromijenjen, a prodaja ostaje u statusu *Created*.

Takav način rada sprječava pojavu djelomično provedene kupoprodaje, primjerice gdje bi prodavatelj dobio sredstva, ali kupac ne bi postao novi digitalni vlasnik nekretnine. Time pametni ugovor osigurava da se kupoprodaja izvrši u cijelosti ili se u cijelosti poništi.

5.1.3. Pametni ugovor *MockEUR*

Za simulaciju financijskog dijela kupoprodaje razvijen je pametni ugovor *MockEUR*. Radi se o ERC-20 tokenu koji unutar prototipa predstavlja simulirana novčana sredstva potrebna za kupnju nekretnine. Token nema stvarnu novčanu vrijednost i koristi se isključivo kako bi se mogao prikazati i ispitati cjelokupan postupak od dodjele sredstava kupcu do njihove automatske isplate prodavatelju.

Token je nazvan *Mock Euro* s oznakom *mEUR*. Za prikaz iznosa koriste se dvije decimalne znamenke čime se način prikaza približava prikazu stvarne valute. Primjer, vrijednost od 150 000,00 mEUR unutar pametnog ugovora predstavljena je vrijednošću od 15 000 000 najmanjih jedinica tokena. Pri postavljanju ugovora ukupna početna količina iznosi 0,00 mEUR, a potrebna sredstva stvaraju se naknadno preko administratora za potrebe simulacije.

Stvaranje novih tokena nije omogućeno svakom korisniku sustava, a funkcija za stvaranje tokena ograničena je posebnom ovlasti kako običan korisnik ne bi mogao sam sebi proizvoljno dodjeljivati sredstva. U praktičnom dijelu prototipa administrator koristi navedenu funkcionalnost za dodjelu simuliranih *mEUR* sredstava blockchain računu koji će biti kupac. Testiranjem je potvrđeno kako pokušaj stvaranja tokena s računom bez odgovarajuće ovlasti završava odbijanjem transakcije i ne mijenja ukupnu količinu tokena.

```
MockEUR.sol X
blockchain > contracts > MockEUR.sol > ...
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.28;
3
4 import {ERC20} from "@openzeppelin/contracts/token/ERC20/ERC20.sol";
5 import {AccessControl} from "@openzeppelin/contracts/access/AccessControl.sol";
6
7 /**
8  * @title MockEUR
9  * @notice Simulirani ERC-20 token koji predstavlja euro u prototipu.
10  *
11  * Token nema stvarnu novčanu vrijednost i koristi se isključivo
12  * za demonstraciju kupoprodaje nekretnina pomoću pametnih ugovora.
13  */
14 contract MockEUR is ERC20, AccessControl {
15     bytes32 public constant MINTER_ROLE = keccak256("MINTER_ROLE");
16
17     constructor() ERC20("Mock Euro", "mEUR") {
18         _grantRole(DEFAULT_ADMIN_ROLE, msg.sender);
19         _grantRole(MINTER_ROLE, msg.sender);
20     }
21
22     /**
23     * @notice Token koristi dvije decimale, poput iznosa u eurima i centima.
24     */
25     function decimals() public pure override returns (uint8) {
26         return 2;
27     }
28
29     /**
30     * @notice Stvara nove mEUR tokene i dodjeljuje ih određenoj adresi.
31     *
32     * Funkciju može pozvati samo korisnik s MINTER_ROLE ulogom.
33     *
34     * @param recipient Adresa primatelja tokena.
35     * @param amount Količina tokena izražena u najmanjim jedinicama.
36     */
37     function mint(
38         address recipient,
39         uint256 amount
40     ) external onlyRole(MINTER_ROLE) {
41         require(
42             recipient != address(0),
43             "Adresa primatelja nije valjana"
44         );
45
46         require(
47             amount > 0,
48             "Kolicina mora biti veca od nule"
49         );
50
51         _mint(recipient, amount);
52     }
53 }
```

Slika 5.17. Implementacija pametnog ugovora MockEUR

Funkcija *mint* omogućuje stvaranje novih tokena i njihovu dodjelu odabranoj blockchain adresi. U razvijenom prototipu administrator pomoću ove funkcije kupcu dodjeljuje proizvoljni iznos simuliranih sredstava prije pokušaja kupnje nekretnine. Time je omogućeno jednostavno pripremanje različitih scenarija testiranja, primjerice s dovoljnim ili nedovoljnim stanjem računa.

Kupac može posjedovati dovoljan iznos *mEUR* tokena, ali samo stanje računa nije dovoljno za izvršavanje kupoprodaje. ERC-20 standard zahtijeva da vlasnik tokena prethodno odobri drugom računu ili pametnom ugovoru korištenje određenog iznosa svojih sredstava. U ovom sustavu kupac funkcijom *approve* odobrava pametnom ugovoru *RealEstateEscrow* korištenje iznosa koji odgovara cijeni nekretnine.

Nakon odobrenja *RealEstateEscrow* može funkcijom *transferFrom* preuzeti odobreni iznos tokena s računa kupca tijekom izvršavanja kupoprodaje. Pametni ugovor prije toga provjerava i stvarno stanje kupca i vrijednost njegova *allowancea*, što onemogućuje kupnju ukoliko oba uvjeta nisu zadovoljena. Nakon uspješne kupoprodaje sredstva se prenose prodavatelju, dok se iskorišteni iznos odobrenja umanjuje u skladu s pravilima ERC-20 tokena. Testovima je potvrđeno ispravno odobravanje i prijenos tokena pomoću *transferFrom* funkcije.

Uloga ugovora *MockEUR* ograničena je na simulaciju financijskog aspekta sustava. Njegova je svrha omogućiti kontrolirano ispitivanje uvjeta kupnje, odobrenja sredstava, prijenos tokena i automatske isplate prodavatelju bez korištenja stvarnog novca.

5.2. Pohrana dokumentacije izvan blockchaina

Dokumentacija povezana s nekretninama predstavlja poseban dio razvijenog sustava jer izvorne datoteke nije praktično pohranjivati izravno u stanje blockchaina. Dokumenti poput zemljišnoknjižnog izvotka, katastarskog dokumenta i dokaza osnove vlasništva mogu biti spremljeni u obliku PDF-a ili slikovnih datoteka i mogu sadržavati znatno veću količinu podataka od uobičajenih podataka zapisanih u pametnom ugovoru.

Zbog toga je u razvijenom prototipu korišten kombinirani pristup. Izvorna datoteka dokumenta pohranjuje se izvan blockchain mreže, dok se u pametnom ugovoru *PropertyRegistry* evidentiraju njezina kriptografska hash vrijednost i URI adresa. Na taj način blockchain i dalje sadrži nepromjenjiv zapis kojim se dokument povezuje s određenom nekretninom, dok se sama datoteka može dohvatiti iz zasebnog spremišta.

Za potrebe lokalnog prototipa razvijen je pomoćni server nazvan *Document Storage Server*. Njegova zadaća je prihvatiti datoteku koju korisnik odabere u korisničkom sučelju, spremiti je u lokalno off-chain spremište te vratiti aplikaciji URL adresu pomoću kojeg se dokument kasnije može otvoriti i pregledati. Podržane datoteke: PDF, PNG, JPG i JPEG.

5.2.1. Document Storage Server

Document Storage Server implementiran je u datoteci *document-storage-server.mjs* pomoću platforme *Node.js* i razvojnog okruženja *Express*. Za prihvatanje datoteka koristi se biblioteka *Multer*, dok *CORS* omogućuje komunikaciju između *React* korisničkog sučelja i lokalnog servera koji se izvršavaju kao odvojeni procesi.

Primljene datoteke spremaju se u lokalni direktorij *document-storage* unutar frontend dijela projekta. Direktorij se prema potrebi automatski stvara te je isključen iz Git repozitorija. Čime se sprječava spremanje testnih dokumenata zajedno s izvornim kodom projekta.

```

JS document-storage-server.mjs X
frontend > JS document-storage-server.mjs > ...
1 import cors from "cors";
2 import express from "express";
3 import multer from "multer";
4
5 import { randomUUID } from "node:crypto";
6 import fs from "node:fs";
7 import path from "node:path";
8 import { fileURLToPath } from "node:url";
9
10 const PORT = 3001;
11 const HOST = "127.0.0.1";
12
13 const __filename = fileURLToPath(import.meta.url);
14 const __dirname = path.dirname(__filename);
15
16 const documentsDirectory = path.join(__dirname, "document-storage");
17
18 if (!fs.existsSync(documentsDirectory)) {
19   fs.mkdirSync(documentsDirectory, {
20     recursive: true,
21   });
22 }
23
24 const allowedMimeTypes = new Set([
25   "application/pdf",
26   "image/png",
27   "image/jpeg",
28 ]);
29
30 function getExtension(file) {
31   switch (file.mimetype) {
32     case "application/pdf":
33       return ".pdf";
34
35     case "image/png":
36       return ".png";
37
38     case "image/jpeg":
39       return ".jpg";
40
41     default:
42       return "";
43   }
44 }
45
46 const storage = multer.diskStorage({
47   destination: (_request, _file, callback) => {
48     callback(null, documentsDirectory);
49   },
50   filename: (_request, file, callback) => {
51     const extension = getExtension(file);
52     callback(null, `${randomUUID()}${extension}`);
53   },
54 });
55
56 const upload = multer({
57   storage,
58   limits: {
59     fileSize: 10 * 1024 * 1024,
60   },
61   fileFilter: (_request, file, callback) => {
62     if (!allowedMimeTypes.has(file.mimetype)) {
63       callback(new Error("Dovoljeni su samo PDF, PNG, JPG i JPEG dokumenti."));
64     }
65     return;
66   },
67   callback: (null, true);
68 });
69
70
71
72
73
74 const app = express();
75
76 app.use(
77   cors({
78     origin: [
79       "http://localhost:5173",
80       "http://127.0.0.1:5173",
81       "http://localhost:5174",
82       "http://127.0.0.1:5174",
83     ],
84   })
85 );
86
87 app.get("/health", (_request, response) => {
88   response.json({
89     status: "ok",
90     service: "real-estate-document-storage",
91   });
92 });
93
94 app.use(
95   "/documents",
96   express.static(documentsDirectory, {
97     fallthrough: false,
98   })
99 );
100
101 app.post("/upload", upload.single("document"), (request, response) => {
102   if (!request.file) {
103     response.status(400).json({
104       error: "Dokument nije poslan.",
105     });
106     return;
107   }
108
109   const documentURL = `http://${HOST}:${PORT}/documents/${request.file.filename}`;
110
111   response.status(201).json({
112     documentURL,
113     fileName: request.file.filename,
114     originalName: request.file.originalname,
115     mimeType: request.file.mimetype,
116     size: request.file.size,
117   });
118 });
119
120 app.use((error, _request, response, _next) => {
121   console.error("Document storage error:", error);
122
123   if (error instanceof multer.MulterError) {
124     if (error.code === "LIMIT_FILE_SIZE") {
125       response.status(400).json({
126         error: "Dokument ne smije biti veći od 10 MB.",
127       });
128       return;
129     }
130   }
131
132   response.status(400).json({
133     error:
134       error instanceof Error ? error.message : "Upload dokumenta nije uspio.",
135   });
136 });
137
138 app.listen(PORT, HOST, () => {
139   console.log("");
140   console.log("=====");
141   console.log("DOCUMENT STORAGE SERVER");
142   console.log("=====");
143   console.log("Server: http://${HOST}:${PORT}");
144   console.log("Health: http://${HOST}:${PORT}/health");
145   console.log("Dokument: http://${HOST}:${PORT}/documents/...");
146   console.log("Direktorij: documentsDirectory");
147   console.log("=====");
148 });
149
150

```

Slika 5.18. Implementacija lokalnog Document Storage Servera

Prilikom primitka datoteke *Multer* je sprema u definirani lokalni direktorij, dok servis korisničkom sučelju vraća adresu na kojoj je spremljeni dokument dostupan. Primjer takve adrese u lokalnom razvojnom okruženju je <http://127.0.0.1:3001/documents/<naziv-dokumenta>>. URI se nakon toga koristi kao podatak koji se zajedno s hash vrijednošću dokumenta zapisuje u pametni ugovor.

Servis omogućuje i dohvat prethodno spremljenih datoteka putem *HTTP* zahtjeva. Zbog toga verifikator u korisničkom sučelju može odabrati mogućnost pregleda dokumenta i otvoriti izvornu datoteku prije donošenja odluke o potvrđivanju ili odbijanju iste.

Bitno je naglasiti kako *Document Storage Server* u razvijenom prototipu nije centralna baza podataka sustava niti sadrži poslovno stanje kupoprodaje. Podaci o digitalnom vlasništvu, statusu dokumentacije, prodajama i izvršenim transakcijama i dalje se nalaze na blockchainu. Lokalni servis se koristi isključivo kao spremište izvornih datoteka dokumentacije.

Takvo rješenje služi prvenstveno za demonstraciju načina odvajanja većih datoteka od podataka spremljenih na blockchainu. Pametni ugovor nije izravno vezan za lokalni *HTTP* servis jer sprema generičku vrijednost *documentURI*. Zbog toga bi se u budućoj izvedbi lokalno spremište moglo zamijeniti drugim off-chain rješenjem bez potrebe za promjenom osnovnog modela dokumenta u pametnom ugovoru.

5.2.2. Hashiranje i povezivanje dokumenta s blockchain zapisom

Povezivanje izvorne datoteke spremljene izvan blockchaina s odgovarajućim zapisom unutar pametnog ugovora provodi se u korisničkom sučelju tijekom predaje dokumentacije. Za svaki od tri obvezna dokumenta frontend provodi isti slijed operacija: izračunava kriptografsku hash vrijednost datoteke, šalje izvornu datoteku *Document Storage Serveru*, preuzima njezinu URI adresu i oba podatka predaje pametnom ugovoru *PropertyRegistry*.

Za izračun hash vrijednosti koristi se funkcija *keccak256* iz biblioteke *ethers.js*. Sadržaj odabrane datoteke najprije se učitava kao niz bajtova, nakon čega se nad cjelokupnim sadržajem izračunava hash. Budući da rezultat ovisi o svim bajtovima datoteke, promjena njezina sadržaja rezultirala bi drugačijom hash vrijednošću.

```
RegisterPropertyForm.tsx ✕
frontend > src > components > RegisterPropertyForm > RegisterPropertyForm.tsx > ...
1 import { BrowserProvider, Contract, JsonRpcProvider, keccak256 } from "ethers";
2
3 import { useEffect, useState, type FormEvent } from "react";
4
5 import {
6     CONTRACT_ADDRESSES,
7     HARDHAT_CHAIN_ID,
8 } from "../../blockchain/contracts";
9
10 import { propertyRegistryAbi } from "../../blockchain/propertyRegistryAbi";
11 import { getPropertyStatusLabel } from "../../utils/statusLabels";
12
13 import "./RegisterPropertyForm.css";

110 async function calculateFileHash(file: File): Promise<string> {
111     const fileBytes = new Uint8Array(await file.arrayBuffer());
112
113     return keccak256(fileBytes);
114 }

457 try {
458     /* =====
459     1. IZRAČUN HASH VRIJEDNOSTI
460     ===== */
461
462     setStatusMessage(
463         "Izračunavaju se kriptografski hashovi dokumentacije...",
464     );
465
466     const [
467         landRegistryExtractHash,
468         cadastralDocumentHash,
469         ownershipDocumentHash,
470     ] = await Promise.all([
471         calculateFileHash(landRegistryExtractFile),
472         calculateFileHash(cadastralDocumentFile),
473         calculateFileHash(ownershipDocumentFile),
474     ]);
475
476     setDocumentHashes({
477         landRegistryExtract: landRegistryExtractHash,
478         cadastralDocument: cadastralDocumentHash,
479         ownershipDocument: ownershipDocumentHash,
480     });
481
482
483
484 }
```

Slika 5.19. Izračun kriptografskih hash vrijednosti dokumentacije

Hash vrijednost sama po sebi ne omogućuje pregled sadržaja dokumenta. Zbog toga se nakon izračuna izvorna datoteka šalje lokalnom *Document Storage Serveru*. Server datoteku sprema u off-chain direktorij i kao odgovor vraća *documentURI*, odnosno adresu putem koje se može pristupiti spremljenoj dokumentaciji.

Na taj način za svaki dokument postoje dvije međusobno povezane informacije: *documentHash* predstavlja kriptografski sažetak njegovog sadržaja, a *documentURI* predstavlja lokaciju na kojoj se može dohvatiti izvorna datoteka. Hash se koristi za identifikaciju i provjeru integriteta dokumenta, a URI omogućuje njegov pregled.

```

116 async function uploadDocument(file: File): Promise<UploadResponse> {
117     const formData = new FormData();
118
119     formData.append("document", file);
120
121     let response: Response;
122
123     try {
124         response = await fetch(`${DOCUMENT_STORAGE_URL}/upload`, {
125             method: "POST",
126             body: formData,
127         });
128     } catch {
129         throw new Error(
130             "Document Storage Server nije dostupan. Pokreni ga naredbom: npm run dev:storage",
131         );
132     }
133
134     let responseData: unknown;
135
136     try {
137         responseData = await response.json();
138     } catch {
139         throw new Error("Document Storage Server vratio je neispravan odgovor.");
140     }
141
142     if (!response.ok) {
143         const errorResponse = responseData as {
144             error?: unknown;
145         };
146
147         if (typeof errorResponse.error === "string") {
148             throw new Error(errorResponse.error);
149         }
150
151         throw new Error("Upload dokumenta nije uspio.");
152     }
153
154     const uploadResponse = responseData as Partial<UploadResponse>;
155
156     if (
157         typeof uploadResponse.documentURI !== "string" ||
158         !uploadResponse.documentURI.trim()
159     ) {
160         throw new Error(
161             "Document Storage Server nije vratio URI spremljenog dokumenta.",
162         );
163     }
164
165     return uploadResponse as UploadResponse;
166 }

```

```

308 /*
309  * Datoteka se prvo sprema izvan blockchajna.
310  */
311 setStatusMessage(
312     `${step} Sprema se dokument "${documentName}" u off-chain spremište...`,
313 );
314
315 const uploadResult = await uploadDocument(file);
316
317 const documentURI = uploadResult.documentURI;
318
319 setDocumentURIs((previous) => ({
320     ...previous,
321     [uriKey]: documentURI,
322 }));
323
324 /*
325  * Nakon uspješnog uploada blockchainu šaljemo
326  * i hash i URI dokumenta.
327  */
328 setStatusMessage(
329     `${step} Dokument je spremljen. Potvrdi blockchain predaju dokumenta "${documentName}" u MetaMasku...`,
330 );
331
332 const transaction = await propertyRegistryWrite.submitPropertyDocument(
333     propertyId,
334     documentType,
335     documentHash,
336     documentURI,
337 );
338
339 setDocumentTransactionHashes((previous) => ({
340     ...previous,
341     [transactionKey]: transaction.hash,
342 }));
343
344 setStatusMessage(
345     `${step} Dokument "${documentName}" je poslan. Čeka se potvrda blockchajna...`,
346 );
347
348 const receipt = await transaction.wait();

```

Slika 5.20. Predaja hash vrijednosti i URI adrese dokumenta pametnom ugovoru

Nakon uspješnog spremanja datoteke frontend poziva funkciju *submitPropertyDocument* pametnog ugovora *PropertyRegistry*. Uz identifikator nekretnine i vrstu dokumenta funkciji se predaju izračunati *documentHash* i dobiveni *documentURI*. Promjena blockchain stanja zahtijeva potvrdu korisnika putem MetaMask novčanika.

Pametni ugovor provjerava da je hash različit od nulte vrijednosti i da URI nije prazan. Nakon uspješnih provjera oba podatka spremaju se unutar strukture *PropertyDocument*, dokument dobiva status *Pending*, a vrijednost *submitted* postavlja se na *true*. Dokument je tada evidentiran na blockchainu, ali još nije potvrđen kao valjan jer postupak verifikacije provodi zasebni ovlašteni korisnik – Verifikator.

Ovakvim pristupom izbjegava se spremanje cijelih datoteka u blockchain stanje, ali se istodobno zadržava veza između blockchain zapisa i konkretne izvorne datoteke. Verifikator iz pametnog ugovora može dohvatiti hash i URI adresu dokumenta te putem URI adrese otvoriti njegov sadržaj prije potvrđivanja ili odbijanja.

Bitno je razlikovati uloge tih dvaju podataka. URI sam po sebi ne jamči da sadržaj dohvaćene datoteke nije promijenjen, dok hash vrijednost omogućuje usporedbu sadržaja s vrijednošću koja je evidentirana prilikom predaje dokumenta. Kombinacija off-chain pohrane i blockchain zapisa prototip demonstrira način na koji se veće datoteke mogu povezati s nepromjenjivim podacima spremljenima u pametnom ugovoru.

Nakon uspješnog spremanja sva tri obvezna dokumenta korisničko sučelje može nastaviti s prikazom njihova statusa, dok verifikator dobiva mogućnost pregleda izvornih datoteka i pojedinačnog potvrđivanja ili odbijanja svakog od dokumenata.

5.3. Implementacija korisničkog sučelja

Korisničko sučelje razvijeno je pomoću biblioteke *React* i programskog jezika *TypeScript*. Njegova osnovna zadaća je omogućiti jednostavnu interakciju korisnika s blockchain mrežom i razvijenim pametnim ugovorima bez potrebe za izravnim pozivanjem njihovih funkcija, te prikaz svih podataka i pregled istih.

Frontend aplikacija omogućuje povezivanje blockchain računa, registraciju i pregled nekretnina, predaju dokumentacije, verifikaciju pojedinačnih dokumenata, kreiranje i otkazivanje prodaja,

pregled dostupnih ponuda, kupnju nekretnina, upravljanje simuliranim mEUR sredstvima i pregled povijesti izvršenih transakcija.

Korisničko sučelje ne predstavlja glavni izvor podataka o stanju sustava, već ih samo prikazuje. Podaci o digitalnom vlasništvu, dokumentaciji, prodajama, simuliranim sredstvima i uvjetima kupoprodaje dohvaćaju se iz pametnih ugovora na blockchain mreži. Frontend navedene podatke obrađuje i prikazuje korisniku u razumljivom obliku.

5.3.1. Povezivanje s blockchain mrežom i MetaMaskom

Komunikacija korisničkog sučelja s blockchain mrežom ostvarena je pomoću biblioteke *ethers.js*. u implementaciji su odvojene operacije čitanja blockchain stanja od operacija kojima se stanje mijenja.

Za operacije koje samo dohvaćaju podatke koristi se objekt *JsonRpcProvider* povezan izravno s lokalnim Hardhat JSON-RPC čvorom na adresi <http://127.0.0.1:8545>. Prije stvaranja instance pametnog ugovora provjerava se identifikator mreže kako bi aplikacija potvrdila da je povezana s očekivanom lokalnom blockchain mrežom.

```
1  import { BrowserProvider, Contract, JsonRpcProvider, keccak256 } from "ethers";

19  const LOCAL_RPC_URL = "http://127.0.0.1:8545";
20  const DOCUMENT_STORAGE_URL = "http://127.0.0.1:3001";

168  async function createReadRegistry(): Promise<Contract> {
169      const provider = new JsonRpcProvider(LOCAL_RPC_URL);
170
171      const network = await provider.getNetwork();
172
173      if (network.chainId !== HARDHAT_CHAIN_ID) {
174          throw new Error(
175              `Neočekivana blockchain mreža. Chain ID: ${network.chainId.toString()}.`,
176          );
177      }
178
179      return new Contract(
180          CONTRACT_ADDRESSES.propertyRegistry,
181          propertyRegistryAbi,
182          provider,
183      );
184  }
```

Slika 5.21. Povezivanje korisničkog sučelja s lokalnom Hardhat blockchain mrežom

Na ovaj način operacije poput pregleda nekretnina, statusa dokumentacije i stanja prodaja mogu se izvršiti bez otvaranja MetaMask prozora i bez dodatnog potpisa korisnika. Takvi pozivi ne mijenjaju stanje na blockchainu, nego samo čitaju već pohranjene podatke.

Operacije kojima se mijenja stanje blockchaina zahtijevaju drugačiji način povezivanja. Za njih se koristi `BrowserProvider`, koji pristupa Ethereum provideru dostupnom putem MetaMask proširenja u internetskom pregledniku. Nakon toga se dohvaća *signer*, odnosno blockchain račun kojim korisnik potpisuje transakciju.

```
492      /* =====
493      3. METAMASK WRITE SIGNER
494      ===== */
495
496      const browserProvider = new BrowserProvider(window.ethereum);
497
498      const signer = await browserProvider.getSigner();
499
500      const signerAddress = await signer.getAddress();
501
502      if (signerAddress.toLowerCase() !== account.toLowerCase()) {
503          throw new Error("MetaMask račun se promijenio. Pokušaj ponovno.");
504      }
505
506      const propertyRegistryWrite = new Contract(
507          CONTRACT_ADDRESSES.propertyRegistry,
508          propertyRegistryAbi,
509          signer,
510      );
```

Slika 5.22. Povezivanje MetaMask računa za potpisivanje blockchain transakcija

Prije stvaranja *write* instance ugovora aplikacija dodatno provjerava odgovara li adresa dobivena iz MetaMaska adresi računa koji frontend trenutno smatra povezanim. Ako se račun u međuvremenu promijeni, operacija se prekida i korisniku se prikazuje odgovarajuća poruka. Time se smanjuje mogućnost da korisnik nenamjerno potvrdi transakciju s pogrešnog blockchain računa.

Nakon stvaranja instance ugovora povezane sa *signerom* mogu se pozivati funkcije koje mijenjaju stanje blockchaina, primjerice *registerProperty*, *submitPropertyDocument*, funkcije za verifikaciju dokumentacije, kreiranje prodaje ili izvršenje kupnje. MetaMask prije slanja takve transakcije korisniku prikazuje zahtjev za njezinu potvrdu.

Lokalna Hardhat mreža koristi *Chain ID 31337*. MetaMask je konfiguriran tako da pristupa istoj lokalnoj mreži preko RPC adrese <http://127.0.0.1:8545>. Time React aplikacija, MetaMask i pametni ugovori tijekom razvoja i ispitivanja rade nad istim lokalnim blockchain stanjem.

Odvajanje READ i WRITE operacije omogućuje jednostavnije korištenje aplikacije. Dohvat podataka ne zahtijeva nepotrebne potpise korisnika, dok svaka operacija koja mijenja stanje blockchaina ostaje eksplicitno potvrđena putem digitalnog novčanika.

5.3.2. Korisnički profili i višekorisnički način rada

Korisničko sučelje razvijeno je kao višekorisnički sustav u kojem povezani blockchain račun obrađuje dostupne funkcionalnosti aplikacije. Završna verzija razlikuje tri osnovna korisnička profila: Administratora, Verifikatora i Korisnika. Profil se određuje na temelju posebnih blockchain uloga dodijeljenih povezanoj adresi.

Ako povezani račun ima administrativnu ulogu *DEFAULT_ADMIN_ROLE*, korisničko sučelje prikazuje profil Administrator. Ako račun ima *VERIFIER_ROLE*, prikazuje se profil Verifikator. Blockchain račun koji nema nijednu od navedenih posebnih uloga tretira se kao Korisnik.

Uloga *TRANSFER_ROLE* ne predstavlja poseban korisnički profil. Ona se u razvijenom sustavu dodjeljuje pametnom ugovoru *RealEstateEscrow* kako bi upravo *escrow*, nakon zadovoljenja svih uvjeta kupoprodaje, mogao izvršiti prijenos digitalnog vlasništva. Time prijenos vlasništva nije ručna funkcionalnost administratora ili drugog korisnika.

```

168      /*
169      * TRANSFER_ROLE nije korisnička uloga.
170      *
171      * Ona pripada escrow pametnom ugovoru koji
172      * automatski izvršava prijenos digitalnog
173      * vlasništva.
174      *
175      * Za određivanje frontend profila zato
176      * provjeravamo samo Administratora
177      * i Verifikatora.
178      */
179      const [adminRole, verifierRole] = await Promise.all([
180          propertyRegistry.DEFAULT_ADMIN_ROLE(),
181          propertyRegistry.VERIFIER_ROLE(),
182      ]);
183
184      if (requestId !== accountRequestIdRef.current) {
185          return;
186      }
187
188      const [hasAdminRole, hasVerifierRole] = await Promise.all([
189          propertyRegistry.hasRole(adminRole, selectedAccount),
190          propertyRegistry.hasRole(verifierRole, selectedAccount),
191      ]);
192
193      if (requestId !== accountRequestIdRef.current) {
194          return;
195      }
196
197      const detectedRoles: string[] = [];
198
199      if (hasAdminRole) {
200          detectedRoles.push("Administrator");
201      }
202
203      if (hasVerifierRole) {
204          detectedRoles.push("Verifikator");
205      }
206
207      /*
208      * Svaki račun bez posebne administrativne
209      * ili verifikatorske uloge obični je Korisnik.
210      *
211      * Korisnik nije trajno označen kao Kupac
212      * ili Prodavatelj.
213      *
214      * Njegova uloga ovisi o konkretnoj radnji:
215      *
216      * - kada registrira i prodaje vlastitu
217      *   nekretninu nastupa kao prodavatelj
218      *
219      * - kada kupuje nekretninu drugog korisnika
220      *   nastupa kao kupac
221      */
222      if (!hasAdminRole && !hasVerifierRole) {
223          detectedRoles.push("Korisnik");

```

Slika 5.23. Određivanje korisničkog profila prema blockchain ulozi

Administrator ima nadzornu i administrativnu ulogu u sustavu. Putem korisničkog sučelja može pregledati registrirane nekretnine, aktivne i završene prodaje, globalne podatke sustava te dodjeljivati simulirana *MockEUR* sredstva drugim korisnicima. Administrator ne potvrđuje dokumentaciju i ne izvršava ručni prijenos digitalnog vlasništva.

Verifikator predstavlja poseban profil namijenjen provjeri dokumentacije. Njegovo korisničko sučelje omogućuje pregled predanih dokumenata, njihovih hash vrijednosti i URI adresa te otvaranje izvornih datoteka. Nakon pregleda svaki dokument može zasebno potvrditi ili odbiti. Nekretnina postaje spremna za prodaju tek kada su sva tri obvezna dokumenta potvrđena.

Najvažnija promjena završne verzije odnosi se na Korisnika. Korisnik nije unaprijed određen kao Kupac ili Prodavatelj. Umjesto toga, njegova uloga proizlazi iz trenutnog blockchain stanja i odnosa prema određenoj nekretnini ili prodaji.

Ako je adresa povezanog računa jednaka vrijednosti *digitalOwner* određene nekretnine korisnik je njezin trenutni digitalni vlasnik. Takvu nekretninu može pregledavati među vlastitim nekretninama te, nakon zadovoljenja svih uvjeta, ponuditi na prodaju. U toj konkretnoj kupoprodaji korisnik nastupa kao Prodavatelj.

Ako aktivnu prodaju nije kreirao povezani korisnik, ona se može prikazati među prodajama dostupnima za kupnju. Kada korisnik pokrene kupnju takve nekretnine u toj transakciji nastupa kao Kupac.

```

198     const exists = sale.exists as boolean;
199
200     const status = Number(sale.status);
201
202     const seller = sale.seller as string;
203
204     /*
205     * SaleStatus:
206     *
207     * 0 = Created
208     * 1 = Funded
209     * 2 = Completed
210     * 3 = Cancelled
211     *
212     * Funded je kod našeg ugovora
213     * samo prijelazni status unutar
214     * iste transakcije.
215     *
216     * Kupcu prikazujemo samo Created.
217     */
218     const isCreated = status === 0;
219
220     const belongsToAnotherAccount =
221         seller.toLowerCase() !== normalizedAccount;
222
223     if (!exists || !isCreated || !belongsToAnotherAccount) {
224         continue;
225     }
226
227     const propertyId = sale.propertyId as bigint;
228
229     /*
230     * Ovdje se vidi ključna stvar
231     * za diplomski:
232     *
233     * frontend NE računa sam
234     * uvjete kupoprodaje.
235     *
236     * getPurchaseConditions()
237     * ih vraća iz smart contracta.
238     */
239     const [property, blockchainConditions] = await Promise.all([
240         propertyRegistry.getProperty(propertyId),
241
242         realEstateEscrow.getPurchaseConditions(saleId, account),
243     ]);

```

Slika 5.24. Prilagodba funkcionalnosti prema blockchain stanju povezanog računa

Takvim pristupom isti blockchain račun tijekom vremena može sudjelovati u više različitih transakcija. Primjerice, ukoliko korisnik koji je u jednoj transakciji kupio nekretninu nakon prijenosa postane njezin novi *digitalOwner* te ju kasnije može ponovno ponuditi na prodaju. U sljedećoj transakciji isti korisnik tada nastupa kao Prodavatelj. Na jednak način prethodni Prodavatelj može kasnije biti Kupac neke druge nekretnine.

Višekorisnički model zato ne zahtjeva unaprijed definirane adrese za kupca i prodavatelja. Posebne uloge koriste se samo za Administratora i Verifikatora, dok se prava Korisnika nad nekretninama i prodajama određuju prema stvarnom stanju pohranjenom u pametnim ugovorima.

Prilagodba korisničkog sučelja prema profilu i blockchain stanju prvenstveno pojednostavljuje korištenje aplikacije. Ona sama po sebi ne predstavlja sigurnosnu zaštitu sustava. Ako bi korisnik i pokušao izravno pozvati funkciju koja mu nije dopuštena, odgovarajuće provjere unutar pametnih ugovora odbile bi transakciju ako definirani uvjeti nisu zadovoljeni.

5.3.3. Registracija i pregled nekretnina

Registracija nekretnine u korisničkom sučelju implementirana je komponentnom *RegisterPropertyForm*. Korisnik unosi katastarsku općinu, broj katastarske čestice i adresu nekretnine te odabire tri obvezna dokumenta. Prije pokretanja blockchain transakcije frontend provjerava jesu li unesena sva tri obvezna dokumenta. Prije pokretanja blockchain transakcije frontend provjerava jesu li unesena sva obvezna tekstualna polja i jesu li odabrane sve potrebne datoteke.

Nakon pripreme podataka stvara se *write* instanca pametnog ugovora *PropertyRegistry* povezana s MetaMask *signerom*. Registracija se zatim pokreće pozivom funkcije *registerProperty*, kojoj se proslijeđuju katastarska općina, broj čestice i adresa nekretnine. Budući da funkcija mijenja blockchain stanje korisnik transakciju mora potvrditi u MetaMasku.

```

512      /* ===== */
513      4. REGISTRACIJA NEKRETNINE
514      /* ===== */
515
516      if (createdPropertyId === null) {
517          setStatusMessage("1/4 Potvrdi registraciju nekretnine u MetaMasku...");
518
519          const registrationTransaction =
520              await propertyRegistryWrite.registerProperty(
521                  cadastralMunicipality.trim(),
522                  parcelNumber.trim(),
523                  propertyAddress.trim(),
524              );
525
526          setRegistrationTransactionHash(registrationTransaction.hash);
527
528          setStatusMessage(
529              "1/4 Registracija je poslana. Čeka se potvrda blockchaina...",
530          );
531
532          const registrationReceipt = await registrationTransaction.wait();
533
534          if (!registrationReceipt) {
535              throw new Error("Potvrda registracije nekretnine nije pronađena.");
536          }
537
538          if (registrationReceipt.status !== 1) {
539              throw new Error("Registracija nekretnine nije uspješno izvršena.");
540          }
541
542          for (const log of registrationReceipt.logs) {
543              try {
544                  const parsedLog = propertyRegistryWrite.interface.parseLog(log);
545
546                  if (parsedLog?.name === "PropertyRegistered") {
547                      createdPropertyId = parsedLog.args.propertyId as bigint;
548
549                      break;
550                  }
551              } catch {
552                  // Log pripada drugom eventu.
553              }
554          }
555
556          if (createdPropertyId === null) {
557              throw new Error("Nije moguće očitati ID registrirane nekretnine.");
558          }
559
560          setPendingPropertyId(createdPropertyId);
561
562          setRegisteredPropertyId(createdPropertyId.toString());
563
564          const registeredProperty =
565              await propertyRegistryRead.getProperty(createdPropertyId);
566
567          if (!(registeredProperty.exists as boolean)) {
568              throw new Error(
569                  "Blockchain nije evidentirao registriranu nekretninu.",
570              );
571          }
572
573          const digitalOwner = registeredProperty.digitalOwner as string;
574
575          if (digitalOwner.toLowerCase() !== account.toLowerCase()) {
576              throw new Error(
577                  "Registrirana nekretnina nema očekivanog digitalnog vlasnika.",
578              );
579          }

```

Slika 5.25. Registracija nekretnine putem korisničkog sučelja

Nakon što je transakcija potvrđena, frontend iz zapisa transakcije pronalazi događaj *PropertyRegistered* i iz njega dohvaća identifikator novostvorene nekretnine. Dobiveni identifikator sprema se u stanje komponente te se koristi za nastavak predaje dokumentacije.

Nakon registracije frontend dodatno ponovo čita stanje pametnog ugovora pomoću funkcije *getProperty*. Provjerava se postoji li upravo registrirana nekretnina te odgovara li vrijednost *digitalOwner* adresi trenutno povezanog računa korisnika. Time se rezultat ne pretpostavlja samo na temelju uspješnog MetaMask poziva, nego se nakon potvrđene transakcije provjerava stvarno stanje lokalne blockchain mreže.

Implementacija također vodi računa o mogućnosti djelomično dovršene registracije. Ako je nekretnina uspješno registrirana na blockchainu, ali je postupak predaje jednog ili više dokumenata prekinut, njezin identifikator privremeno se čuva u varijabli *pendingPropertyId*. Korisnik tada može nastaviti predaju dokumentacije bez ponovnog poziva funkcije *registerProperty*. Time se izbjegava pokušaj stvaranja duplikata nekretnine nakon djelomično uspješnog procesa.

Osim registracije, korisničko sučelje omogućuje pregled podataka o nekretninama pohranjenim u ugovoru *PropertyRegistry*. Za Korisnika relevantne su prvenstveno nekretnine čiji je trenutni *digitalOwner* jednak adresi povezanog MetaMask računa. Administrator može pregledavati sve registrirane nekretnine. Takva podjela vidljiva je već u glavnoj komponenti aplikacije, gdje se *PropertyPanel* za Administratora otvara s mogućnošću prikaza svih nekretnina, dok se za Korisnika prikazuje samo skup njegovih nekretnina.


```

195     const propertyCount =
196         (await propertyRegistry.getPropertyCount()) as bigint;
197
198     if (requestId !== requestIdRef.current) {
199         return;
200     }
201
202     const loadedProperties: PropertyData[] = [];
203
204     const normalizedAccount = account.toLowerCase();
205
206     for (let propertyId = 1n; propertyId <= propertyCount; propertyId++) {
207         const property = await propertyRegistry.getProperty(propertyId);
208
209         if (requestId !== requestIdRef.current) {
210             return;
211         }
212
213         const exists = property.exists as boolean;
214
215         const digitalOwner = property.digitalOwner as string;
216
217         const belongsToConnectedAccount =
218             digitalOwner.toLowerCase() === normalizedAccount;
219
220         if (!exists || (!showAll && !belongsToConnectedAccount)) {

```

Slika 5.26. Dohvat i filtriranje registriranih nekretnina prema digitalnom vlasniku

Na taj način prikaz nekretnina proizlazi iz stvarnog blockchain stanja, a ne iz lokalno spremljenog popisa korisnika i nekretnina. Nakon uspješne kupoprodaje i promjene vrijednosti *digitalOwner*, ista nekretnina više se ne prikazuje prethodnom vlasniku kao njegova, nego novom digitalnom vlasniku. Time se korisničko sučelje automatski prilagođava promjenama koje su izvršene unutar pametnih ugovora.

Prikaz nekretnine uključuje njezin identifikator, katastarsku općinu, broj čestice, adresu, trenutnog digitalnog vlasnika i stanje dokumentacije. Takvi podaci korisniku omogućuju pregled trenutnog blockchain stanja prije nastavka postupka, primjerice kreiranja prodaje ili ponovne prodaje nekretnine koja je prethodno kupljena.

5.3.4. Predaja i verifikacija dokumentacije

Nakon što Korisnik registrira nekretninu i preda sva tri obvezna dokumenta, dokumentacija postaje dostupna Verifikatoru putem komponente *VerifyPropertiesPanel*. Komponenta dohvaća

registrirane nekretnine i podatke o svakom pojedinačnom dokumentu iz pametnog ugovora *PropertyRegistry*.

Za svaku nekretninu dohvaćaju se osnovni podaci, tri obvezna dokumenta te rezultati funkcija *hasAllRequiredDocuments* i *hasValidDocuments*. Time korisničko sučelje može prikazati koliko je dokumenata predano, koliko ih je potvrđeno i je li nekretnina spremna za prodaju. Frontend pritom ne određuje sam valjanost dokumentacije, nego prikazuje rezultate koje prima s blockchaina.

```

213 ✓      const [
214      property,
215      landRegistryDocument,
216      cadastralDocument,
217      ownershipDocument,
218      hasAllRequiredDocuments,
219      hasValidDocuments,
220 ✓    ] = await Promise.all([
221      propertyRegistry.getProperty(propertyId),
222
223 ✓      propertyRegistry.getPropertyDocument(
224      propertyId,
225      DOCUMENT_TYPE.LAND_REGISTRY_EXTRACT,
226      ),
227
228 ✓      propertyRegistry.getPropertyDocument(
229      propertyId,
230      DOCUMENT_TYPE.CADASTRAL_DOCUMENT,
231      ),
232
233 ✓      propertyRegistry.getPropertyDocument(
234      propertyId,
235      DOCUMENT_TYPE.OWNERSHIP_DOCUMENT,
236      ),
237
238      propertyRegistry.hasAllRequiredDocuments(propertyId),
239
240      propertyRegistry.hasValidDocuments(propertyId),
241    ]);
242
243 ✓    const documents: PropertyDocumentData[] = [
244 ✓    {
245      documentType: DOCUMENT_TYPE.LAND_REGISTRY_EXTRACT,
246
247      name: DOCUMENT_DEFINITIONS[0].name,
248
249      documentHash: landRegistryDocument.documentHash as string,
250
251      documentURI: landRegistryDocument.documentURI as string,
252
253 ✓      verificationStatus: Number(
254      landRegistryDocument.verificationStatus,
255      ),
256
257      submitted: landRegistryDocument.submitted as boolean,
258    },
259
260 ✓    {
261      documentType: DOCUMENT_TYPE.CADASTRAL_DOCUMENT,
262
263      name: DOCUMENT_DEFINITIONS[1].name,
264
265      documentHash: cadastralDocument.documentHash as string,
266
267      documentURI: cadastralDocument.documentURI as string,
268
269 ✓      verificationStatus: Number(
270      cadastralDocument.verificationStatus,
271      ),
272
273      submitted: cadastralDocument.submitted as boolean,
274    },
275
276 ✓    {
277      documentType: DOCUMENT_TYPE.OWNERSHIP_DOCUMENT,
278
279      name: DOCUMENT_DEFINITIONS[2].name,
280
281      documentHash: ownershipDocument.documentHash as string,
282
283      documentURI: ownershipDocument.documentURI as string,
284
285      verificationStatus: Number(
286      ownershipDocument.verificationStatus,
287      ),
288
289      submitted: ownershipDocument.submitted as boolean,
290    },
291    ];

```

Slika 5.27. Dohvat dokumentacije i statusa nekretnine za potrebe verifikacije

Za svaki dokument korisničko sučelje prikazuje njegov naziv, trenutni status, hash vrijednost i adresu izvorne datoteke. Ako je dokument ima evidentiran *documentURI*, Verifikator može otvoriti datoteku pomoću mogućnosti Pregledaj dokument. Time je moguće vizualno pregledati stvarni PDF ili slikovnu datoteku koja je spremljena u off-chain spremištu prije potvrđivanja ili odbijanja dokumenta.

Promjena statusa dokumenta provodi se funkcijom *updateDocumentVerificationStatus*. Funkcija prima identifikator nekretnine, vrstu dokumenta, njegov naziv i željenu radnju koja može biti *verify* ili *reject*.

Prije pokretanja WRITE transakcije frontend ponovno čita trenutno blockchain stanje dokumenta. Provjerava se postoji li nekretnina, je li dokument predan te nalazi li se još uvijek u statusu *Pending*. Time se izbjegava slanje transakcije nad dokumentom čije je stanje u međuvremenu promijenjeno.

Nakon toga se pomoću *BrowserProvider* objekta dohvaća MetaMask *signer*. Ako je odabrana radnja potvrđivanje poziva se funkcija *verifyPropertyDocument*, a ako je odabrana radnja odbijanje poziva se funkcija *rejectPropertyDocument*. Budući da obje funkcije mijenjaju blockchain stanje, Verifikator mora potvrditi transakciju u MetaMasku.

```

392 const readProvider = new JsonRpcProvider(LOCAL_RPC_URL);
393
394 const readNetwork = await readProvider.getNetwork();
395
396 if (readNetwork.chainId !== HARDHAT_CHAIN_ID) {
397   throw new Error("Hardhat local mreža nije dostupna.");
398 }
399
400 const propertyRegistryRead = new Contract(
401   CONTRACT_ADDRESSES.propertyRegistry,
402   propertyRegistryAbi,
403   readProvider,
404 );
405
406 const property = await propertyRegistryRead.getProperty(propertyId);
407
408 if (!property.exists as boolean) {
409   throw new Error("Nekretnina više ne postoji u registru.");
410 }
411
412 const documentBefore = await propertyRegistryRead.getPropertyDocument(
413   propertyId,
414   documentType,
415 );
416
417 const submittedBefore = documentBefore.submitted as boolean;
418
419 const statusBefore = Number(documentBefore.verificationStatus);
420
421 const documentHashBefore = documentBefore.documentHash as string;
422
423 const documentURIBefore = documentBefore.documentURI as string;
424
425 if (!submittedBefore) {
426   throw new Error("Dokument nije predan i nije ga moguće verificirati.");
427 }
428
429 if (!documentHashBefore) {
430   throw new Error("Dokument nema evidentiran blockchain hash.");
431 }
432
433 if (!documentURIBefore.trim()) {
434   throw new Error("Dokument nema evidentiranu adresu za pregled.");
435 }
436
437 if (statusBefore !== 0) {
438   throw new Error("Dokument nije u statusu Na čekanju.");
439 }
440
441 /*
442  * BrowserProvider koristimo isključivo
443  * za MetaMask potpis WRITE operacije.
444  */
445 const browserProvider = new BrowserProvider(window.ethereum);
446
447 const signer = await browserProvider.getSigner();
448
449 const signerAddress = await signer.getAddress();
450
451 if (signerAddress.toLowerCase() !== account.toLowerCase()) {
452   throw new Error("MetaMask račun se promijenio. Pokušaj ponovno.");
453 }
454
455 const propertyRegistryWrite = new Contract(
456   CONTRACT_ADDRESSES.propertyRegistry,
457   propertyRegistryAbi,
458   signer,
459 );
460
461 const actionText = action === "verify" ? "potvrdu" : "odbijanje";
462
463
464 setStatusMessage(
465   Potvrdi ${actionText} dokumenta "${documentName}" u MetaMasku...
466 );
467
468 const transaction =
469   action === "verify"
470     ? await propertyRegistryWrite.verifyPropertyDocument(
471       propertyId,
472       documentType,
473     )
474     : await propertyRegistryWrite.rejectPropertyDocument(
475       propertyId,
476       documentType,
477     );
478
479 setStatusMessage(
480   "Transakcija je poslana. Čeka se potvrda blockchaine...",
481 );
482
483 const receipt = await transaction.wait();

```

```

466 const postTransactionProvider = new JsonRpcProvider(LOCAL_RPC_URL);
467
468 const propertyRegistryPost = new Contract(
469   CONTRACT_ADDRESSES.propertyRegistry,
470   propertyRegistryAbi,
471   postTransactionProvider,
472 );
473
474 const documentAfter = await propertyRegistryPost.getPropertyDocument(
475   propertyId,
476   documentType,
477 );
478
479 const submittedAfter = documentAfter.submitted as boolean;
480
481 const statusAfter = Number(documentAfter.verificationStatus);
482
483 const documentHashAfter = documentAfter.documentHash as string;
484
485 const documentURIAfter = documentAfter.documentURI as string;
486
487 if (!submittedAfter) {
488   throw new Error("Blockchain više ne evidencira dokument kao predan.");
489 }
490
491 /*
492  * Verifikacija ili odbijanje omoguće promijeniti
493  * samo status. Hash i URI moraju ostati isti.
494  */
495 if (
496   documentHashAfter.toLowerCase() !== documentHashBefore.toLowerCase()
497 ) {
498   throw new Error(
499     "Hash dokumenta neočekivano se promijenio tijekom verifikacije.",
500 );
501 }
502
503 if (documentURIAfter !== documentURIBefore) {
504   throw new Error(
505     "URI dokumenta neočekivano se promijenio tijekom verifikacije.",
506 );
507 }
508
509 const expectedStatus = action === "verify" ? 1 : 2;
510
511 if (statusAfter !== expectedStatus) {
512   throw new Error(
513     action === "verify"
514       ? "Blockchain nije evidentirao dokument kao potvrđen."
515       : "Blockchain nije evidentirao dokument kao odbijen.",
516 );
517 }
518
519 /*
520  * Ako se upravo potvrđuje treći dokument,
521  * PropertyRegistry sam mora automatski
522  * ovrnuti cijelu dokumentaciju na članom.
523  */
524 const [hasAllRequiredDocuments, hasValidDocuments] = await Promise.all([
525   propertyRegistryPost.hasAllRequiredDocuments(
526     propertyId,
527   ) as Promise-boolean,
528   propertyRegistryPost.hasValidDocuments(propertyId) as Promise-boolean,
529 ]);
530
531 setStatusMessage("");
532
533 if (action === "verify" && hasAllRequiredDocuments && hasValidDocuments) {
534   setSuccessMessage(
535     `${documentName} za nekretninu ID ${propertyId.toString()} uspješno je potvrđen. Sva 3 obvezna dokumenta sada su potvrđena i nekretnina je spremna za prodaju.`,
536 );
537 } else {
538   setSuccessMessage(
539     action === "verify"
540       ? `${documentName} za nekretninu ID ${propertyId.toString()} uspješno je potvrđen. : ${documentName} za nekretninu ID ${propertyId.toString()} uspješno je odbijen.`,
541       : `${documentName} za nekretninu ID ${propertyId.toString()} uspješno je odbijen.`,
542 );
543 }
544
545 await loadProperties();

```

Slika 5.28. Potvrđivanje i odbijanje dokumentacije putem korisničkog sučelja

Nakon potvrđene MetaMask transakcije frontend ponovno dohvaća dokument izravno s lokalne Hardhat mreže. Provjerava se odgovara li novi status očekivanoj vrijednosti *Verified* ili *Rejected*. Time korisničko sučelje ne pretpostavlja da je operacija uspješna samo zato što je transakcija poslana, već provjerava i konačno stanje zapisano u pametnom ugovoru.

Nakon svake promjene ponovno se dohvaćaju i rezultati funkcija *hasAllRequiredDocuments* i *hasValidDocuments*. Ako je upravo potvrđen treći obvezni dokument te obje funkcije vraćaju pozitivnu vrijednost, korisniku se prikazuje poruka da su sva tri dokumenta potvrđena i da je nekretnina spremna za prodaju.

Na taj način odluka o valjanosti dokumentacije ostaje pod kontrolom pametnog ugovora, dok React komponenta Verifikatoru pruža pregledno sučelje za čitanje dokumentacije i pokretanje odgovarajućih blockchain transakcija.

5.3.5. Kreiranje prodaje i kupnja nekretnine

Kreiranje nove prodaje omogućeno je komponentom *CreateSaleForm*. Prije prikaza nekretnina koje se mogu ponuditi na prodaju komponenta dohvaća trenutno stanje blockchaina i automatski izdvaja samo one nekretnine koje zadovoljavaju potrebne uvjete. U ponuđeni popis ulaze nekretnine koje postoje u registru, čiji je povezani račun trenutni digitalni vlasnik, čija je dokumentacija u potpunosti potvrđena i koje već nemaju aktivnu drugu prodaju.

Za potvrđivanje postoji li aktivna prodaja najprije se prolazi kroz postojeće zapise u ugovoru *RealEstateEscrow*. Prodaje u statusu *Created*, odnosno aktivne prodaje, povezuju se s pripadajućim nekretninama kako ih Korisnik ne bi mogao ponovo ponuditi. Nakon toga se za svaku nekretninu provjeravaju njezin *digitalOwner* i rezultat funkcije *hasValidDocuments*. Time korisničko sučelje korisniku prikazuje samo nekretnine koje su prema trenutnom blockchain stanju prikladne za kreiranje nove prodaje.

```

146      /*
147      * Prvo određujemo koje nekretnine
148      * već imaju aktivnu prodaju.
149      */
150      const propertiesWithActiveSale = new Set<string>();
151
152      for (let saleId = 1n; saleId <= saleCount; saleId++) {
153        const sale = await realEstateEscrow.getSale(saleId);
154
155        if (requestId !== requestIdRef.current) {
156          return;
157        }
158
159        const saleExists = sale.exists as boolean;
160
161        const saleStatus = Number(sale.status);
162
163        const salePropertyId = sale.propertyId as bigint;
164
165        /*
166        * SaleStatus:
167        *
168        * 0 = Created
169        * 1 = Funded
170        * 2 = Completed
171        * 3 = Cancelled
172        *
173        * Funded je u našem escrow ugovoru
174        * samo prijelazni status unutar iste
175        * blockchain transakcije.
176        */
177        const isActive = saleStatus === 0 || saleStatus === 1;
178
179        if (saleExists && isActive) {
180          propertiesWithActiveSale.add(salePropertyId.toString());
181        }
182      }
183
184      const eligibleProperties: PropertyOption[] = [];
185
186      const normalizedAccount = account.toLowerCase();
187
188      for (let propertyId = 1n; propertyId <= propertyCount; propertyId++) {
189        /*
190        * Blockchain sam određuje jesu li
191        * sva tri obvezna dokumenta valjana.
192        */
193        const [property, hasValidDocuments] = await Promise.all([
194          propertyRegistry.getProperty(propertyId),
195          propertyRegistry.hasValidDocuments(propertyId),
196        ]);
197
198        if (requestId !== requestIdRef.current) {
199          return;
200        }
201
202        const exists = property.exists as boolean;
203
204        const digitalOwner = property.digitalOwner as string;
205
206        const belongsToConnectedAccount =
207          digitalOwner.toLowerCase() === normalizedAccount;
208
209        const documentsValid = hasValidDocuments as boolean;
210
211        const hasNoActiveSale = !propertiesWithActiveSale.has(
212          propertyId.toString(),
213        );
214
215        /*
216        * U dropdown ulazi samo nekretnina:
217        *
218        * - koja postoji
219        * - kojoj je povezani račun vlasnik
220        * - kojoj su svi dokumenti potvrđeni
221        * - koja nema drugu aktivnu prodaju
222        */
223        if (
224          exists &&
225          belongsToConnectedAccount &&
226          documentsValid &&
227          hasNoActiveSale
228        ) {
229          eligibleProperties.push({
230            id: propertyId as bigint,
231            cadastralMunicipality: property.cadastralMunicipality as string,
232            parcelNumber: property.parcelNumber as string,
233            propertyAddress: property.propertyAddress as string,
234            hasValidDocuments: documentsValid,
235          });
236        }
237      }

```

Slika 5.29. Određivanje nekretnina dostupnih za kreiranje prodaje

Nakon odabira nekretnine Korisnik unosi prodajnu cijenu. Iznos se pomoću funkcije *parseUnits* pretvara u najmanje jedinice *MockEUR* tokena, uz dvije decimalne znamenke. Neposredno prije slanja WRITE transakcije frontend ponovo provjerava valjanost dokumentacije i trenutnog digitalnog vlasnika. Time se izbjegava oslanjanje na podatke koji su možda bili aktualni u trenutku prvog učitavanja obrasca, ali su se u međuvremenu promijenili.

Ako su provjere uspješne, pomoću MetaMask *signera* stvara se write instanca ugovora *RealEstateEscrow*, a prodaja se kreira pozivom funkcije *createSale*. Korisnik zatim potvrđuje transakciju u MetaMasku. Pametni ugovor, unatoč provjerama u frontendu, ponovno provodi vlastite provjere vlasništva, dokumentacije, cijene i postojanja druge aktivne prodaje.

```

360     const priceInSmallestUnits = parseUnits(normalizedPrice, 2);
361
362     if (priceInSmallestUnits <= 0n) {
363         throw new Error("Cijena mora biti veća od nule.");
364     }
365
366     const propertyId = BigInt(selectedPropertyId);
367
368     /*
369     * Neposredno prije MetaMask WRITE transakcije
370     * još jednom čitamo aktualno blockchain stanje
371     * izravno s Hardhat nodea.
372     */
373     const readProvider = new JsonRpcProvider(LOCAL_RPC_URL);
374
375     const readNetwork = await readProvider.getNetwork();
376
377     if (readNetwork.chainId !== HARDHAT_CHAIN_ID) {
378         throw new Error("Hardhat local mreža nije dostupna.");
379     }
380
381     const propertyRegistryRead = new Contract(
382         CONTRACT_ADDRESSES.propertyRegistry,
383         propertyRegistryAbi,
384         readProvider,
385     );
386
387     const realEstateEscrowRead = new Contract(
388         CONTRACT_ADDRESSES.realEstateEscrow,
389         realEstateEscrowAbi,
390         readProvider,
391     );
392
393     const [isValidDocuments, digitalOwner, saleCountBefore] =
394         await Promise.all([
395             propertyRegistryRead.isValidDocuments(
396                 propertyId,
397             ) as Promise<boolean>,
398             propertyRegistryRead.getDigitalOwner(propertyId) as Promise<string>,
399             realEstateEscrowRead.getSaleCount() as Promise<bigint>,
400         ]);
401
402     if (!isValidDocuments) {
403         throw new Error("Nekretnina nema potpuno potvrđenu dokumentaciju.");
404     }
405
406     if (digitalOwner.toLowerCase() !== account.toLowerCase()) {
407         throw new Error(
408             "Povezani račun više nije digitalni vlasnik nekretnine.",
409         );
410     }
411 }

```

```

414     /*
415     * MetaMask koristimo samo za WRITE
416     * operaciju koju prodavatelj mora potpisati.
417     */
418     const browserProvider = new BrowserProvider(window.ethereum);
419
420     const signer = await browserProvider.getSigner();
421
422     const signerAddress = await signer.getAddress();
423
424     if (signerAddress.toLowerCase() !== account.toLowerCase()) {
425         throw new Error("MetaMask račun se promijenio. Pokušaj ponovno.");
426     }
427
428     const realEstateEscrowWrite = new Contract(
429         CONTRACT_ADDRESSES.realEstateEscrow,
430         realEstateEscrowAbi,
431         signer,
432     );
433
434     setStatusMessage("Potvrdi kreiranje prodaje u MetaMasku...");
435
436     /*
437     * Iako smo na frontendu provjerili
438     * dokumentaciju i vlasništvo,
439     * RealEstateEscrow.createSale()
440     * ponovno provjerava uvjete.
441     *
442     * Frontend zato ne može zaobići
443     * sigurnosna pravila smart contracta.
444     */
445     const transaction = await realEstateEscrowWrite.createSale(
446         propertyId,
447         priceInSmallestUnits,
448     );
449
450     setTransactionHash(transaction.hash);
451
452     setStatusMessage(
453         "Transakcija je poslana. Čeka se potvrda blockchaine...",
454     );
455
456     const receipt = await transaction.wait();

```

Slika 5.30. Kreiranje prodaje putem pametnog ugovora *RealEstateEscrow*

Nakon potvrđene transakcije prodaja se pojavljuje u popisu aktivnih prodaja. Korisnik vidi svoje aktivne prodaje, dok Administrator može pregledavati sve aktivne prodaje u sustavu. Otkazivanje prodaje omogućeno je samo stvarnom prodavatelju. Komponenta *ActiveSalesPanel* prije slanja transakcije provjerava podudara li se adresa prodavatelja s povezanim računom te zatim poziva funkciju *cancelSale*. Nakon potvrde ponovno se čita blockchain stanje i provjerava je li prodaja prešla u status *Cancelled*.

Kupoprodaja iz perspektive drugog Korisnika implementirana je komponentom *PurchaseSalePanel*. Komponenta prikazuje samo aktivne prodaje drugih korisnika te rezultate funkcije *getPurchaseConditions*. Uz svaku prodaju korisniku se prikazuju stanje njegovih mEUR sredstava i trenutni iznos odobren escrow ugovoru.

Prije same kupnje potrebno je odobriti escrow ugovoru korištenje potrebnog iznosa *MockEUR* tokena. Funkcija *approveSale* najprije provjerava raspolaže li Korisnik dovoljnim stanjem, a zatim poziva ERC-20 funkciju *approve*. Escrow ugovoru odobrava se točno iznos koji odgovara cijeni

nekretnine koja se prodaje. Nakon potvrđene transakcije stanje se ponovno učitava kako bi rezultat funkcije `getPurchaseConditions` pokazao da je uvjet dovoljnog *allowance*-a zadovoljen.

```
388     async function approveSale(sale: ActiveSale): Promise<void> {
389         setProcessingAction(`approve-${sale.id.toString()}`);
390
391         setStatusMessage("");
392         setSuccessMessage("");
393         setErrorMessage("");
394         setTransactionHash("");
395
396         try {
397             /*
398              * Ovo je samo pomoćna frontend
399              * zaštita.
400              *
401              * Stvarni uvjeti ostaju pod
402              * kontrolom smart contracta.
403              */
404             if (!sale.conditions.buyerHasSufficientBalance) {
405                 throw new Error(
406                     "Kupac nema dovoljno MockEUR sredstava za ovu prodaju.",
407                 );
408             }
409
410             const { mockEUR } = await getSignerContracts();
411
412             setStatusMessage("Potvrdi odobrenje MockEUR tokena u MetaMasku...");
413
414             /*
415              * Kupac odobrava samo točan iznos
416              * potreban za ovu kupoprodaju.
417              */
418             const transaction = await mockEUR.approve(
419                 CONTRACT_ADDRESSES.realEstateEscrow,
420                 sale.price,
421             );
422
423             setTransactionHash(transaction.hash);
424
425             setStatusMessage(
426                 "Transakcija je poslana. Čeka se potvrda blockchaina...",
427             );
428
429             const receipt = await transaction.wait();
430
431             if (!receipt) {
432                 throw new Error("Potvrda transakcije nije pronađena.");
433             }
434
435             setStatusMessage("");
436
437             setSuccessMessage(
438                 `Escrow ugovoru odobreno je korištenje ${formatUnits(
439                     sale.price,
440                     2,
441                 )} mEUR za prodaju ID ${sale.id.toString()}.`,
442             );
443
444             /*
445              * Sada ponovno čitamo stanje
446              * direktno s Hardhat RPC-a.
447              *
448              * getPurchaseConditions()
449              * mora vratiti allowance = true.
450              */
451             await loadPurchaseData();
```

Slika 5.31. Odobravanje *MockEUR* sredstava escrow pametnom ugovoru

Kada su svi uvjeti zadovoljeni kupnja se pokreće funkcijom *purchaseSale*. Neposredno prije WRITE transakcije frontend ponovno poziva *getPurchaseConditions* izravno preko lokalnog Hardhat RPC-a. Ako vrijednost *readyForPurchase* nije istinita, kupnja se prekida prije slanja transakcije. Time korisničko sučelje smanjuje mogućnost nepotrebnog pokušaja kupnje na temelju zastarjelog prikaza.

Ako pametni ugovor potvrdi da su uvjeti još uvijek zadovoljeni putem MetaMask *signera* poziva se funkcija *fundSale*. Kao što je prethodno opisano ta funkcija ponovno provodi sve sigurnosne provjere te u slučaju uspjeha automatski izvršava prijenos sredstava i digitalnog vlasništva.

Nakon potvrde kupoprodajne transakcije frontend ponovno čita stanje s blockchaina. Provjerava se novi digitalni vlasnik nekretnine, novo stanje mEUR tokena Kupca i Prodavatelja te završni status prodaje. Uspjeh se prihvaća tek kada je status prodaje *Completed* i kada se adresa novog *digitalOwner* podudara s adresom kupca.

```

469     try {
470         /*
471          * Neposredno prije WRITE transakcije
472          * ponovno pitamo smart contract jesu li
473          * svi uvjeti još uvijek zadovoljeni.
474          *
475          * Čitanje ide direktno preko Hardhat RPC-a.
476          */
477         const readProvider = new JsonRpcProvider(LOCAL_RPC_URL);
478
479         const readEscrow = new Contract(
480             CONTRACT_ADDRESSES.realEstateEscrow,
481             realEstateEscrowAbi,
482             readProvider,
483         );
484
485         const latestConditions = await readEscrow.getPurchaseConditions(
486             sale.id,
487             account,
488         );
489
490         if (!latestConditions.readyForPurchase as boolean) {
491             throw new Error(
492                 "Smart contract potvrđuje da još nisu ispunjeni svi uvjeti za kupoproduju.",
493             );
494         }
495
496         const { realEstateEscrow } = await getSignerContracts();
497
498         setStatusMessage(
499             "Svi uvjeti su zadovoljeni. Potvrdi kupnju nekretnine u MetaMasku...",
500         );
501
502         /*
503          * fundSale je ključna WRITE operacija.
504          *
505          * Smart contract ponovno provjerava:
506          *
507          * - status prodaje
508          * - dokumentaciju
509          * - vlasništvo prodavatelja
510          * - da kupac nije prodavatelj
511          * - saldo kupca
512          * - allowance
513          *
514          * Ako je sve zadovoljeno, ugovor
515          * automatski:
516          *
517          * 1. uzima sredstva kupca
518          * 2. prenosi digitalno vlasništvo
519          * 3. isplaćuje prodavatelja
520          * 4. završava prodaju
521          */
522         const transaction = await realEstateEscrow.fundSale(sale.id);
523
524         setTransactionHash(transaction.hash);
525
526         setStatusMessage(
527             "Pametni ugovor automatski izvršava kupoproduju. Čeka se potvrda blockchaina...",
528         );
529
530         const receipt = await transaction.wait();
531
532         if (!receipt) {
533             throw new Error("Potvrda kupoprodajne transakcije nije pronađena.");
534         }
535
536         /*
537          * Nakon potvrđene transakcije konačno
538          * stanje NE čitamo preko MetaMaska,
539          * nego direktno s Hardhat nodea.
540          */
541         const postTransactionProvider = new JsonRpcProvider(LOCAL_RPC_URL);
542
543         const propertyRegistryRead = new Contract(
544             CONTRACT_ADDRESSES.propertyRegistry,
545             propertyRegistryAbi,
546             postTransactionProvider,
547         );
548
549         const mockEURRead = new Contract(
550             CONTRACT_ADDRESSES.mockEUR,
551             mockEURAbi,
552             postTransactionProvider,
553         );
554
555         const realEstateEscrowRead = new Contract(
556             CONTRACT_ADDRESSES.realEstateEscrow,
557             realEstateEscrowAbi,
558             postTransactionProvider,
559         );
560
561         const [newDigitalOwner, newBuyerBalance, sellerBalance, completedSale] =
562             await Promise.all([
563                 propertyRegistryRead.getDigitalOwner(
564                     sale.propertyId,
565                 ) as Promise<string>,
566                 mockEURRead.balanceOf(account) as Promise<bigint>,
567                 mockEURRead.balanceOf(sale.seller) as Promise<bigint>,
568                 realEstateEscrowRead.getSale(sale.id),
569             ]);
570
571         const completedStatus = Number(completedSale.status);
572
573         if (completedStatus !== 2) {
574             throw new Error(
575                 "Prodaja nije dobila očekivani status ${getSaleStatusLabel(2)}.",
576             );
577         }
578
579         if (newDigitalOwner.toLowerCase() !== account.toLowerCase()) {
580             throw new Error("Digitalno vlasništvo nije preneseno na kupca.");
581         }
582     }

```

Slika 5.32. Izvršenje kupoprodaje i provjera konačnog blockchain stanja

Nakon uspješne kupnje prodaja više nije prikazana među aktivnim ponudama, dok kupljena nekretnina postaje vidljiva među nekretninama novog digitalnog vlasnika. Time višekorisnički način rada omogućuje da isti račun koji je u ovoj transakciji nastupao kao Kupac kasnije kupljenu nekretninu može ponovno ponuditi na prodaju i u novoj transakciji tada nastupa kao Prodavatelj.

5.3.6. Prikaz povijesti transakcija

Završni dio korisničkog sučelja predstavlja komponenta *TransactionHistoryPanel* koja omogućuje pregled prethodno provedenih kupoprodaja. Prikaz povijesti prilagođen je povezanom korisničkom profilu gdje Administrator pomoću funkcije *showAll* može pregledavati cjelokupnu povijest sustava, dok Korisnik vidi transakcije u kojima je njegova Ethereum adresa sudjelovala kao Kupac ili Prodavatelj.

Takav način prikaza odgovara višekorisničkom modelu jer isti Korisnik tijekom vremena može biti i Prodavatelj i Kupac. Povijest transakcija zato omogućuje pregled vlastitih završenih prodaja i kupnji, dok Administratorski profil pruža širi pregled aktivnosti sustava.

```
134     for (let saleId = 1n; saleId <= saleCount; saleId++) {
135         const sale = await realEstateEscrow.getSale(saleId);
136
137         if (requestId !== requestIdRef.current) {
138             return;
139         }
140
141         const exists = sale.exists as boolean;
142         const status = Number(sale.status);
143         const seller = sale.seller as string;
144         const buyer = sale.buyer as string;
145
146         /*
147          * SaleStatus:
148          *
149          * 0 = Created
150          * 1 = Funded
151          * 2 = Completed
152          * 3 = Cancelled
153          */
154
155         const isCompleted = status === 2;
156         const isCancelled = status === 3;
157
158         const belongsToConnectedAccount =
159             seller.toLowerCase() === normalizedAccount ||
160             buyer.toLowerCase() === normalizedAccount;
161
162         if (
163             !exists ||
164             (!isCompleted && !isCancelled) ||
165             (!showAll && !belongsToConnectedAccount)
166         ) {
167             continue;
168         }
169
170         const propertyId = sale.propertyId as bigint;
171
172         const property = await propertyRegistry.getProperty(propertyId);
173
174         if (requestId !== requestIdRef.current) {
175             return;
176         }
177
178         loadedSales.push({
179             id: sale.id as bigint,
180             propertyId,
181             seller,
182             buyer,
183             price: sale.price as bigint,
184             status,
185
186             propertyAddress: property.propertyAddress as string,
187
188             cadastralMunicipality: property.cadastralMunicipality as string,
189
190             parcelNumber: property.parcelNumber as string,
191
192             currentDigitalOwner: property.digitalOwner as string,
193         });
194     }
```

Slika 5.33. Dohvat i filtriranje povijesti kupoprodajnih transakcija

Implementacijom navedenih React komponenti korisničko sučelje povezuje funkcionalnosti pametnih ugovora u jedinstven višekorisnički sustav. Frontend Korisnicima omogućuje registraciju nekretnina, predaju i provjeru dokumentacije, kreiranje prodaje, upravljanje

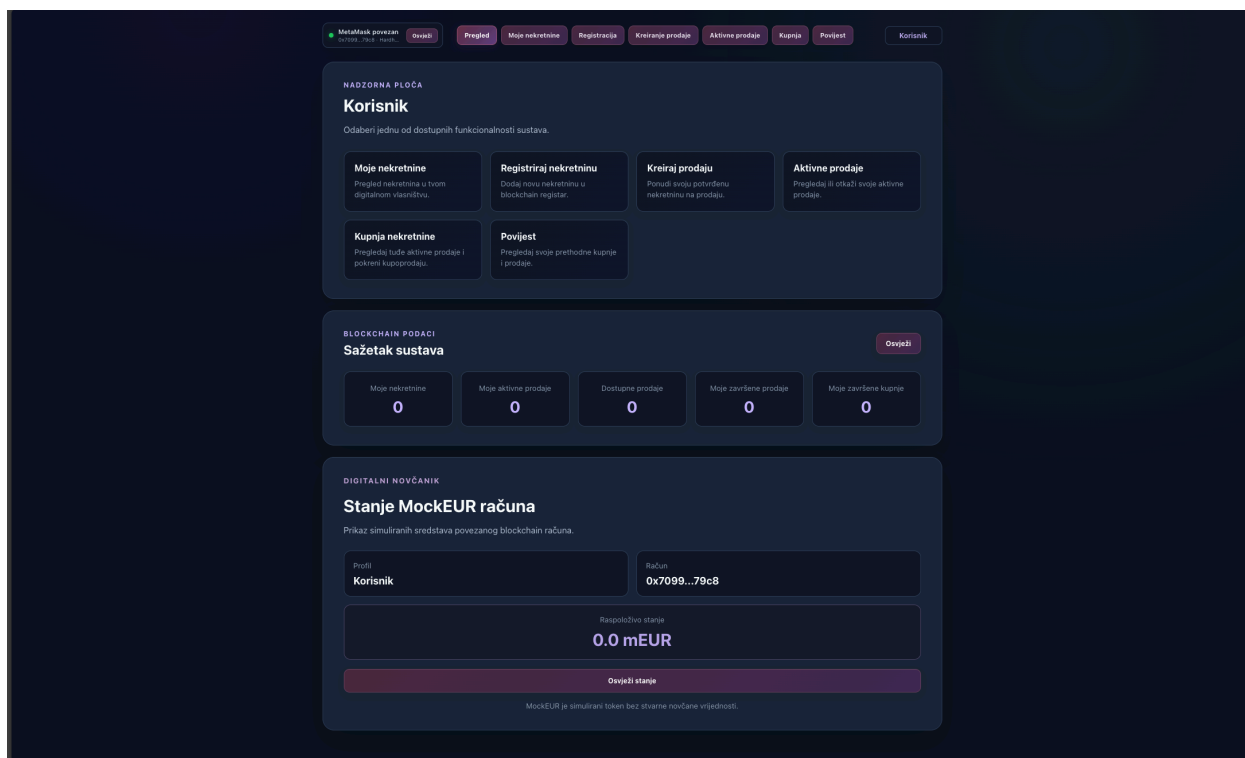
simuliranim sredstvima, izvršenje kupoprodaje i pregled prethodnih transakcija. Poslovna pravila i konačno stanje pritom ostaju definirani i pohranjeni u pametnim ugovorima, dok korisničko sučelje služi kao pristupačan način njihove uporabe.

6. PRIKAZ I ISPITIVANJE RADA SUSTAVA

U ovom poglavlju prikazan je rad razvijenog sustava kroz osnovne scenarije korištenja. Naglasak je na praktičnom tijeku rada Korisnika, od povezivanja blockchain računa i registracije nekretnina, do provjere dokumentacije, kreiranja prodaje i završetka kupoprodaje. Prikazani rezultati dobiveni su korištenjem lokalne Hardhat blockchain mreže, MetaMask novčanika i razvijenog React korisničkog sučelja.

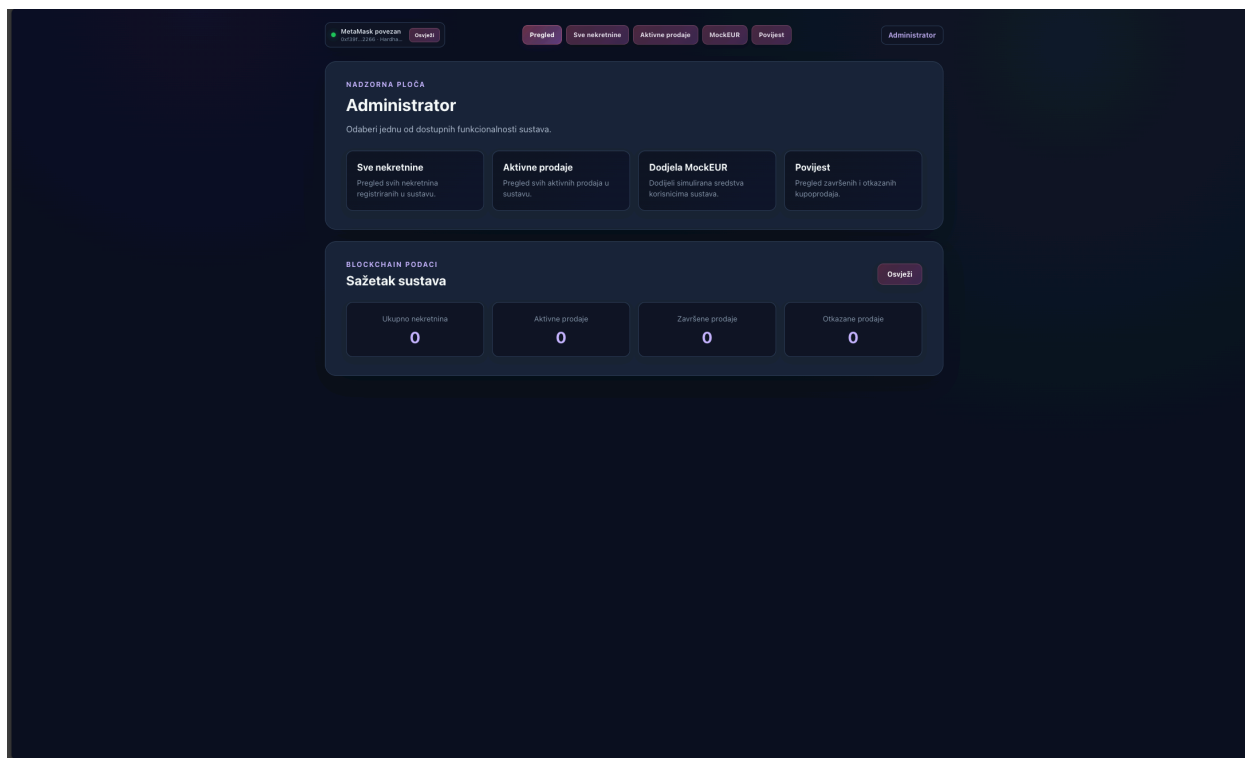
6.1. Povezivanje korisnika i prikaz korisničkog profila

Prije korištenja funkcionalnosti sustava Korisnik povezuje MetaMask novčanik s aplikacijom. Nakon povezivanja aplikacija dohvaća adresu aktivnog računa i njegove blockchain uloge te na temelju njih prikazuje odgovarajući korisnički profil.

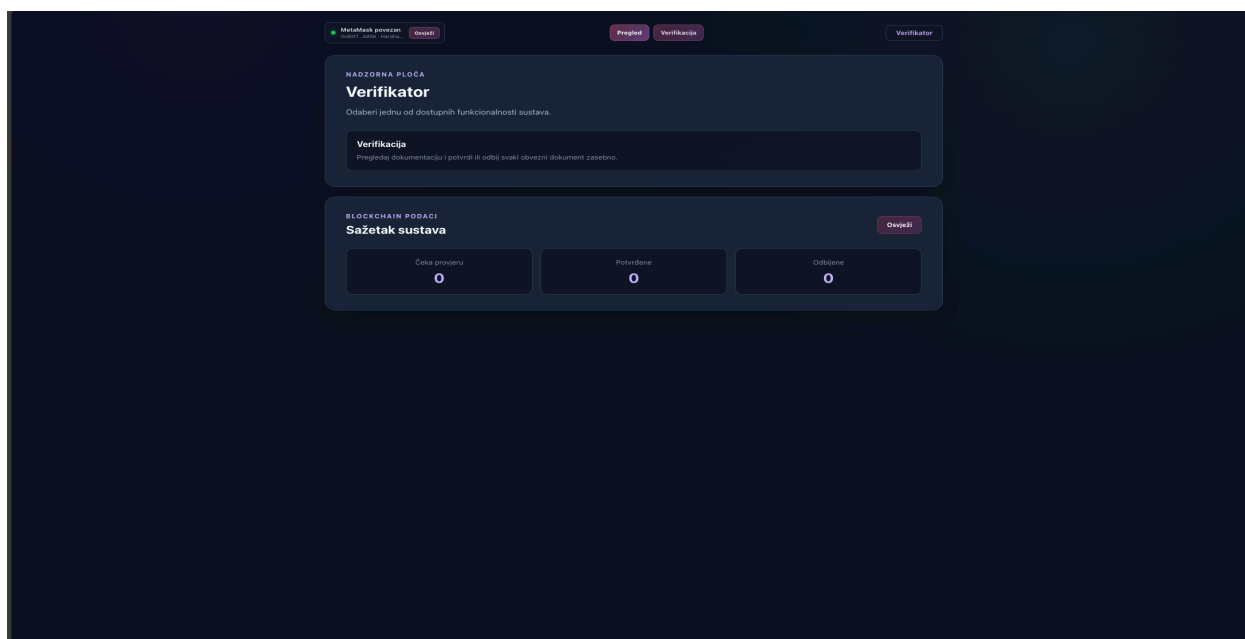


Slika 6.1. Početni zaslon aplikacije za povezani Korisnički račun

Povezani račun na prikazanom primjeru nema administratorsku niti verifikacijsku ulogu, zbog čega mu aplikacija dodjeljuje profil Korisnik. Ovom profilu dostupne su funkcionalnosti registracije i pregleda vlastitih nekretnina, kreiranje prodaje, kupnja nekretnina od drugih korisnika i pregled povijesti vlastitih transakcija.



Slika 6.2. Početni zaslon aplikacije za povezani Administratorski račun



Slika 6.3. Početni zaslon aplikacije za povezani Verifikatorski račun

Promjenom aktivnog MetaMask računa mijenja se i profil prikazan u aplikaciji. Administrator ima pristup nadzornim i administrativnim funkcionalnostima sustava, dok Verifikator ima pristup pregledu i potvrđivanju dokumentacije registriranih nekretnina.

6.2. Registracija nekretnine i predaja dokumentacije

Korisnik novu nekretninu registrira unosom osnovnih katastarskih i adresnih podataka te dodavanjem tri obvezna dokumenta. Nakon potvrde blockchain transakcije nekretnina dobiva vlastiti identifikator i povezuje se s adresom korisnika koji ju je registrirao.

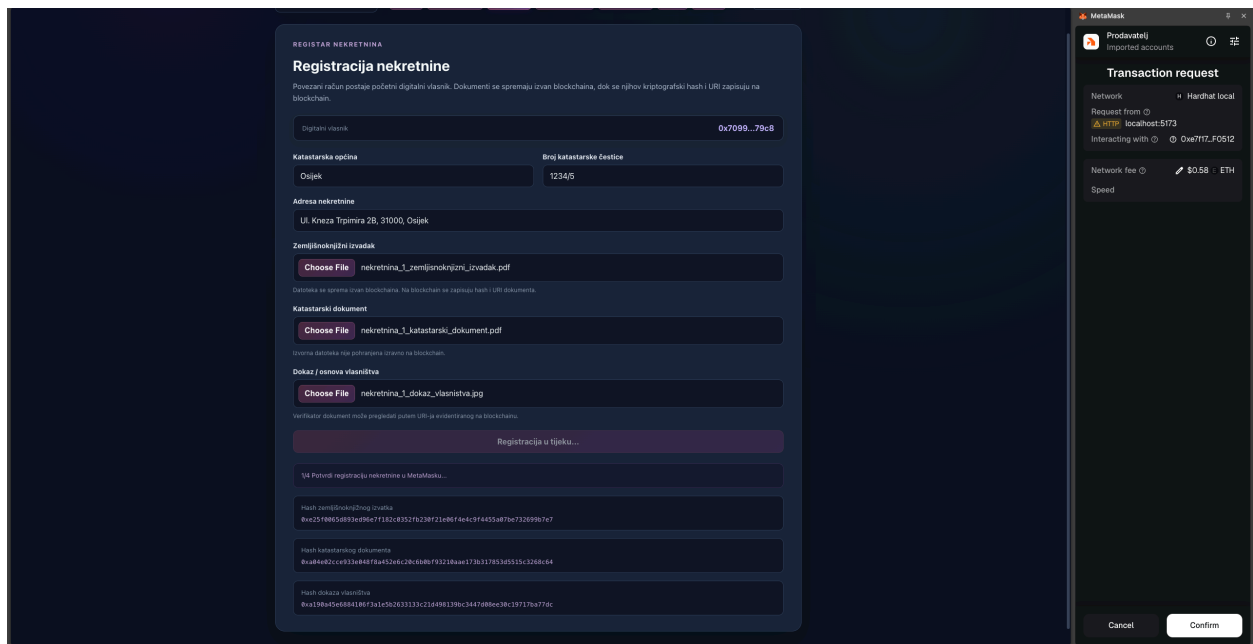
The screenshot shows a web application interface for registering real estate. At the top, there's a navigation bar with buttons like 'Ogledi', 'Pregled', 'Moje nekretnine', 'Registracija', 'Kreiranje prodaje', 'Aktivna prodaja', 'Kupnja', 'Povijest', and 'Korisnik'. The main content area is titled 'REGISTAR NEKRETNINA' and 'Registracija nekretnine'. It contains a form with the following fields and sections:

- Digitelni vlasnik:** A text field with the value '0x7099...79c8'.
- Katastarska općina:** A text field with the value 'Osijek'.
- Broj katastarske čestice:** A text field with the value '1234/5'.
- Adresa nekretnine:** A text field with the value 'Ul. Kneza Trpimira 2B, 31000, Osijek'.
- Zemljišnoknjižni izvadak:** A section with a 'Choose File' button and the filename 'nekretnina_1_zemljišnoknjižni_izvadak.pdf'.
- Katastarski dokument:** A section with a 'Choose File' button and the filename 'nekretnina_1_katastarski_dokument.pdf'.
- Dokaz / osnova vlasništva:** A section with a 'Choose File' button and the filename 'nekretnina_1_dokaz_vlasnistva.jpg'.

Below the document sections, there are small informational notes: 'Datoteka se sprema izvan blockchaine. Na blockchain se zapisuju hash i URI dokumenta.' and 'Verifikator dokument može pregledati putem URI-ja evidentiranog na blockchainu.' At the bottom of the form is a large purple button labeled 'Registriraj nekretninu i predaj dokumente'.

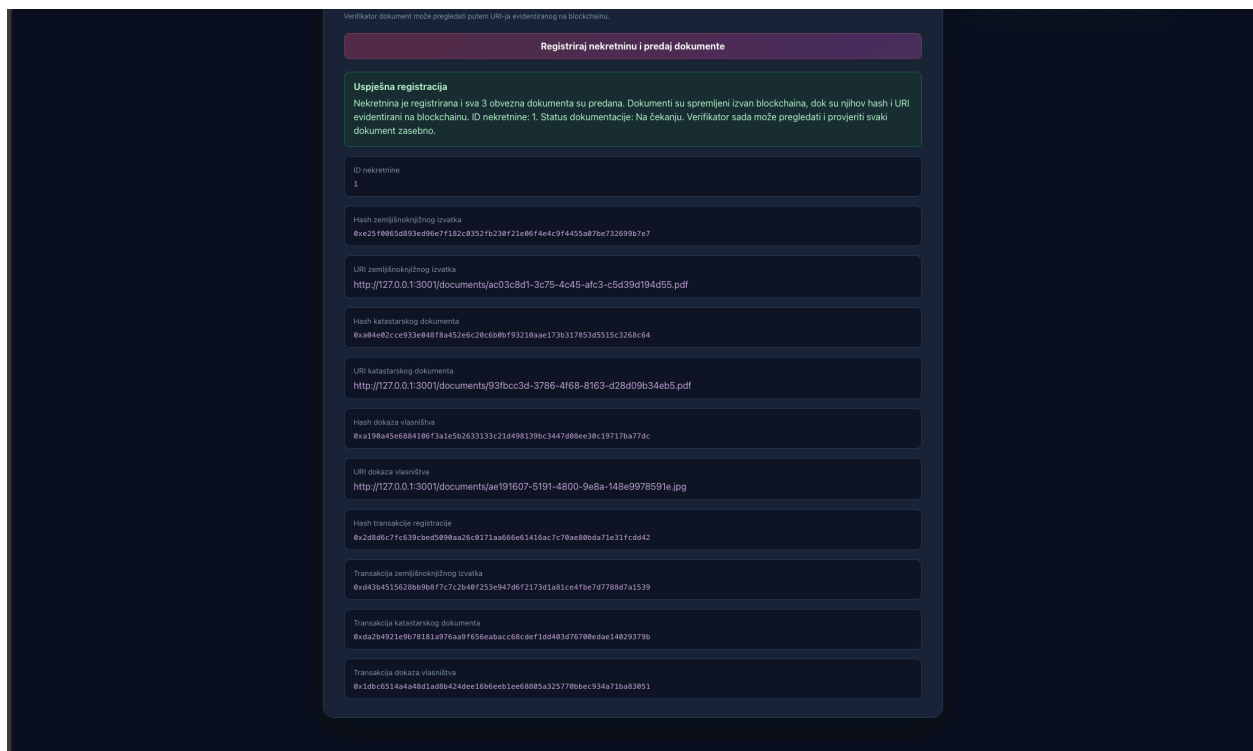
Slika 6.4. Unos podataka i dokumentacije prilikom registracije nekretnine

Na slici 6.4. prikazan je obrazac za registraciju nekretnine prije slanja blockchain transakcije. Korisnik unosi osnovne podatke o nekretnini i dodaje tri obvezna dokumenta: zemljišnoknjižni izvadak, katastarski dokument i dokaz vlasništva.



Slika 6.5. Potvrda transakcije registracije nekretnine u MetaMasku

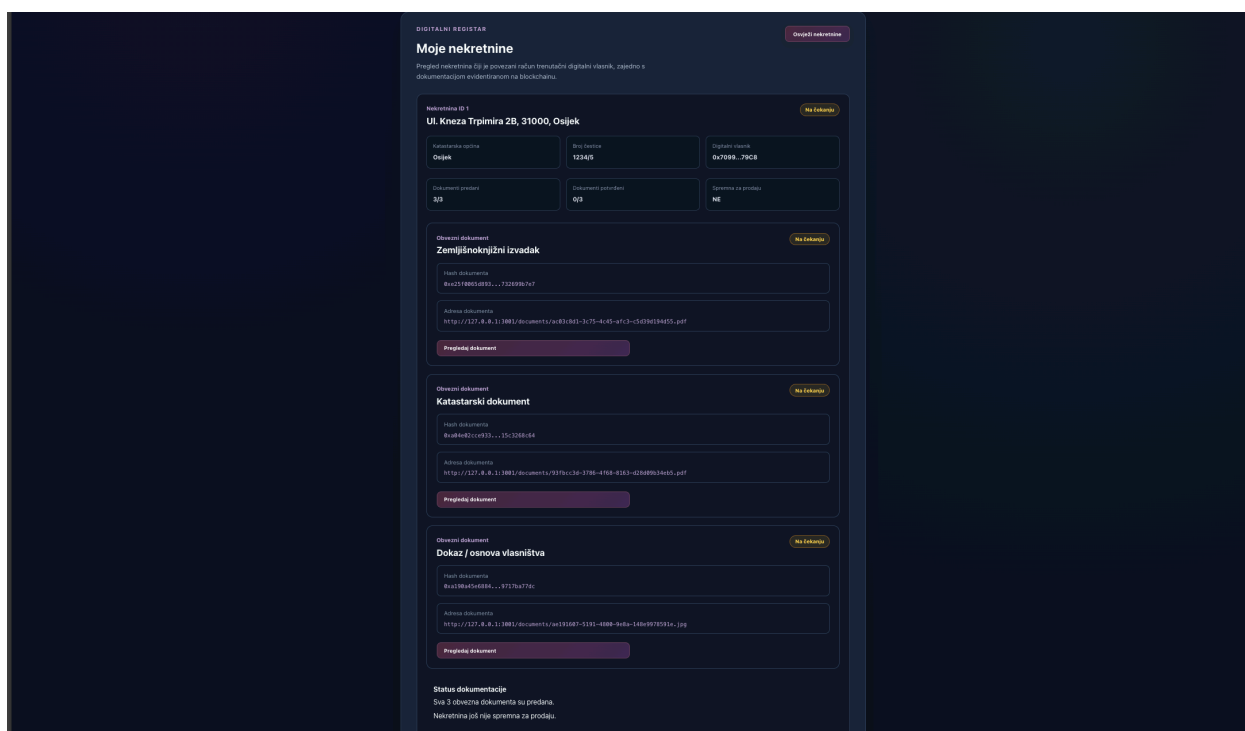
Nakon unosa podataka i odabira dokumentacije Korisnik pokreće registraciju nekretnine. Budući da se radi o WRITE operaciji koja mijenja stanje blockchaina, transakciju je potrebno potvrditi u MetaMask novčaniku. U prikazanom primjeru MetaMask prikazuje zahtjev za potvrdu transakcije prije njezina slanja na lokalnu Hardhat mrežu.



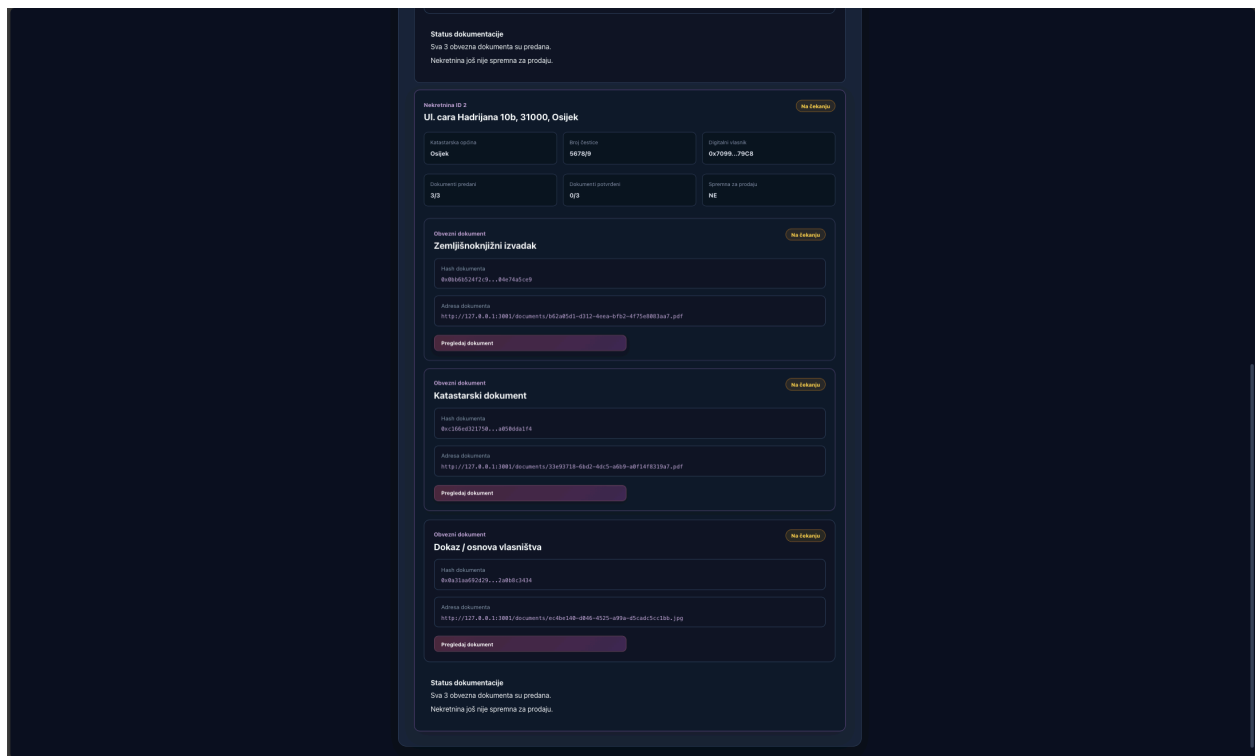
Slika 6.6. Uspješno evidentiranje nekretnine i dokumentacije

Nakon potvrde transakcije aplikacija prikazuje poruku o uspješnoj registraciji nekretnine. Uz osnovne podatke o registraciji prikazane su hash vrijednosti i URI adrese sva tri predana dokumenta čime se potvrđuje njihova povezanost s blockchain zapisom i off-chain spremištem.

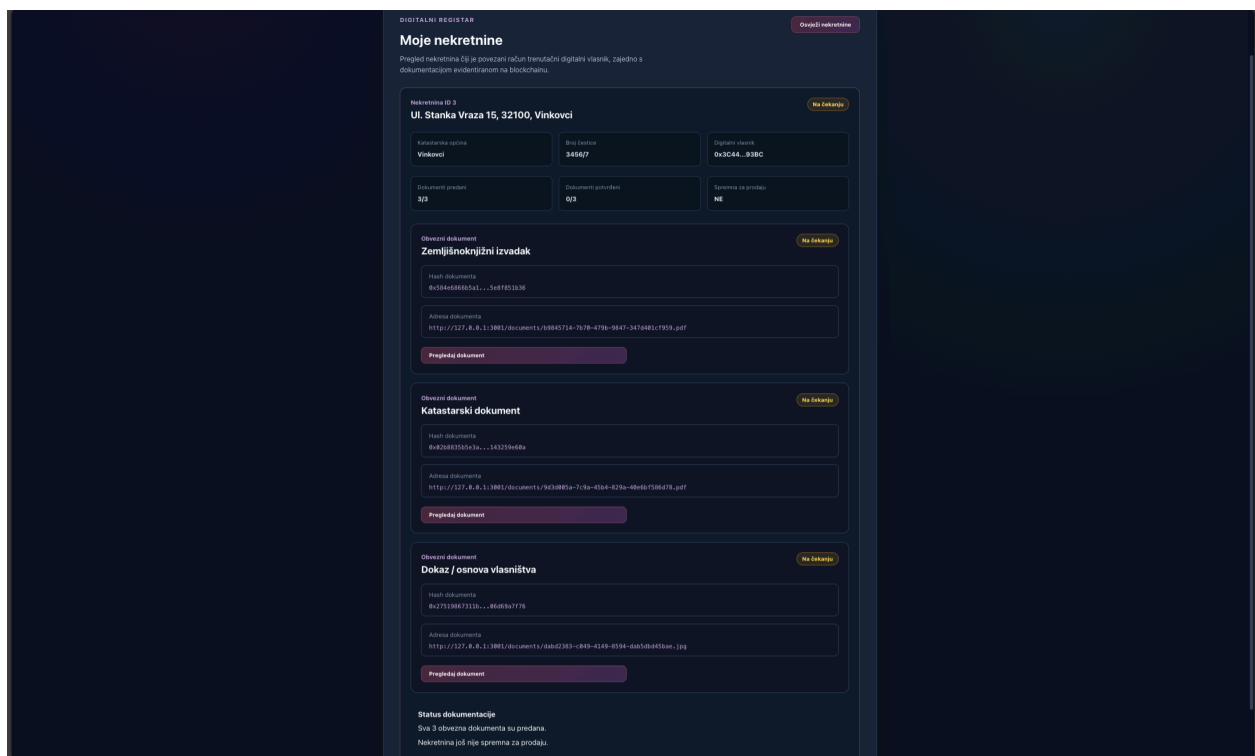
Isti postupak registracije i predaje dokumentacije proveden je za još dvije testne nekretnine. Za potrebe ispitivanja korištena su dva Korisnička računa s različitim adresama, pri čemu su na jednom računu registrirane dvije nekretnine, dok je na drugom registrirana jedna nekretnina. Takva raspodjela omogućuje ispitivanje višekorisničkog rada sustava, odnosno prodaju nekretnine s jednog računa i njezinu kupnju s drugog računa.



Slika 6.7. Registrirana prva nekretnina Korisnika1



Slika 6.8. Registrirana druga nekretnina Korisnika1

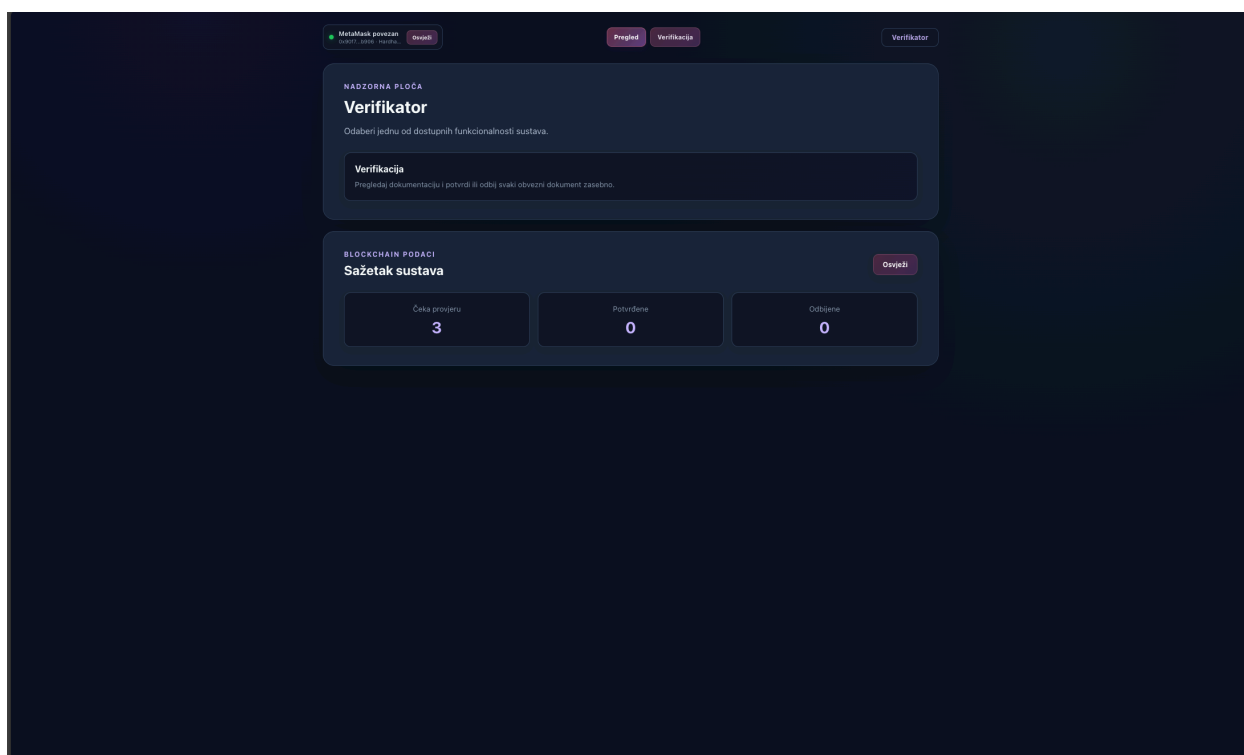


Slika 6.9. Registrirana prva nekretnina Korisnika2

Na slikama 6.7., 6.8. i 6.9. prikazano je početno stanje testnog scenarija nakon registracije svih nekretnina, pri čemu na slikama 6.7. i 6.8. su nekretnine Korisničkog računa koji posjeduje dvije nekretnine (Korisnik1), a na slici 6.9. su nekretnine drugog Korisničkog računa (Korisnik2) koji posjeduje jednu nekretninu.

6.3. Verifikacija dokumentacije

Nakon registracije nekretnina njihova dokumentacija još nema status potvrđene dokumentacije. Verifikator se zato povezuje svojim MetaMask računom i kroz odgovarajući prikaz pregledava predane dokumente i zasebno odlučuje o njihovu potvrđivanju ili odbijanju.



Slika 6.10. Početno stanje verifikacijskog profila prije provjere dokumentacije

Nakon registracije svih nekretnina Verifikatoru su one dostupne za pregled. Na početnoj nadzornoj ploči prikazan je broj nekretnina koje čekaju provjeru te broj potvrđenih ili odbijenih nekretnina. Budući da u prikazanom trenutku dokumentacija još nije provjerena, sve tri registrirane nekretnine nalaze se u početnom stanju verifikacije.

Otvaranjem prikaza za verifikaciju moguće je pregledati osnovne podatke svake nekretnine i sva tri povezana dokumenta. Uz svaki dokument prikazani su njegova hash vrijednost, URI adresa, trenutni status te mogućnost pregleda, potvrđivanja ili odbijanja dokumenta.

The screenshot displays the 'Verifikator' interface for a property located at 'Ul. cara Hadrijana 10b, 31000, Osijek'. The property details include the Katastarska općina (Osijek), Broj čestice (5678/9), and Digitalni vlasnik (0x7099...79C8). The status shows 'Dokumenti predani' (3/3), 'Dokumenti potvrđeni' (0/3), and 'Sprema za prodaju' (NE). Below the property details, there are three document sections, each with a 'Na čekanju' (On hold) status:

- Zemljišnoknjižni izvadak:** Hash dokumenta (0x066052472c9...04e74a5ce9), Adresa dokumenta (http://127.0.0.1:3001/documents/062a5d1-0312-4eea-bfb2-4f75e0083aa7.pdf). Buttons: Pregledaj dokument, Potvrdi dokument, Odbij dokument.
- Katastarski dokument:** Hash dokumenta (0xc166e0321750...a0580da1f4), Adresa dokumenta (http://127.0.0.1:3001/documents/33e93718-60d2-4dc5-a6b9-a0f14f0313a7.pdf). Buttons: Pregledaj dokument, Potvrdi dokument, Odbij dokument.
- Dokaz / osnova vlasništva:** Hash dokumenta (0x0a31aa692d29...2a0b0c3434), Adresa dokumenta (http://127.0.0.1:3001/documents/ec4be140-d046-4525-a99a-d5cad3cc1bb.jpg). Buttons: Pregledaj dokument, Potvrdi dokument, Odbij dokument.

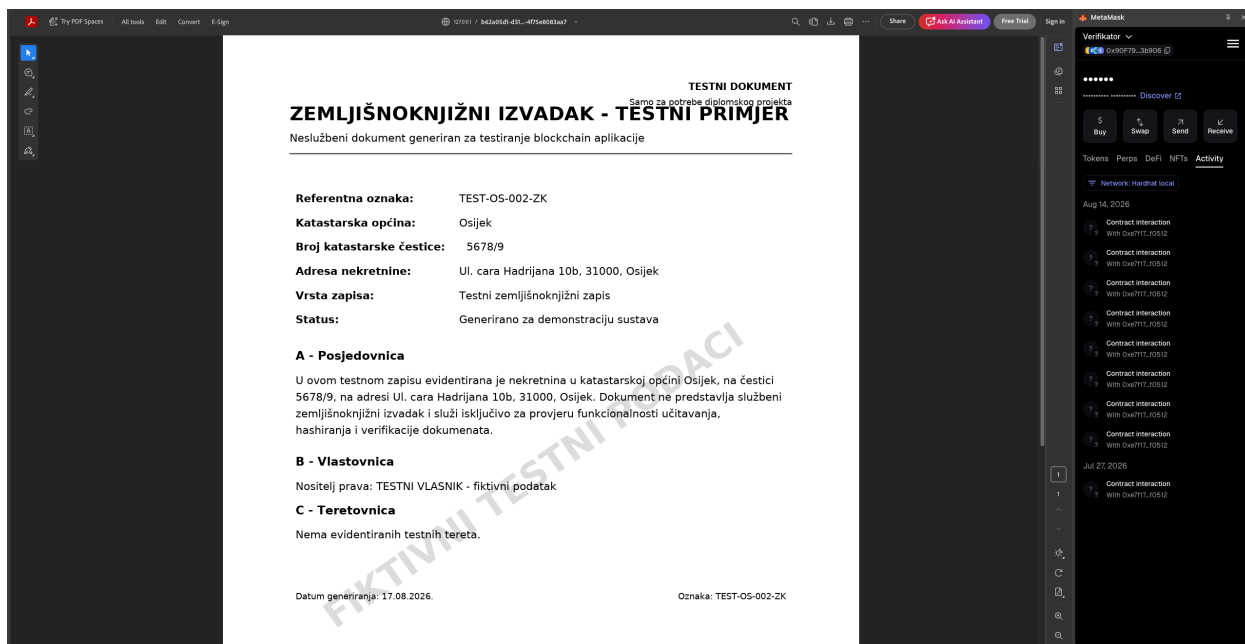
Slika 6.11. Podaci o nekretnini i dokumentaciji dostupnoj Verifikatoru

This screenshot shows the 'Status dokumentacije' (Documentation status) section of the Verifikator interface. It displays the same three documents as the previous screenshot, but with a summary status at the bottom:

Status dokumentacije
Sva 3 obvezna dokumenta su predana.
Nekretnina još nije spremna za prodaju.

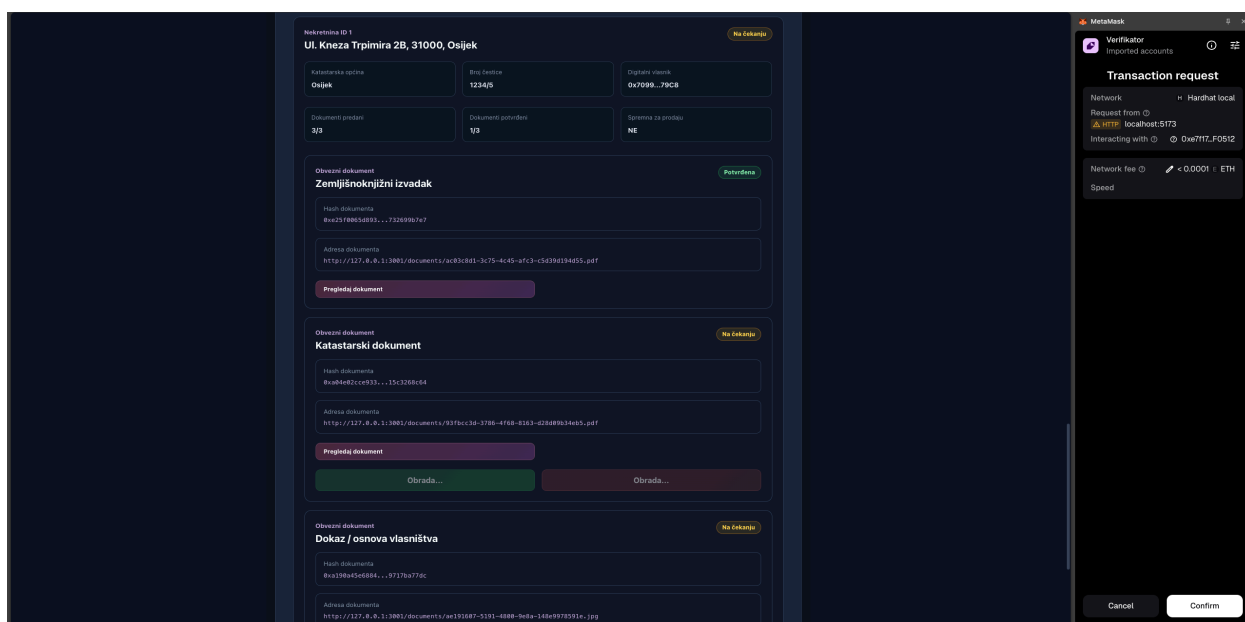
Slika 6.12. Mogućnosti potvrđivanja i odbijanja dokumentacije nekretnine

U početnom stanju sva tri dokumenta imaju status da čekaju provjeru i potvrdu ili odbijanje. Verifikator svaki dokument obrađuje zasebno, pri čemu prije donošenja odluke može otvoriti izvornu datoteku spremljenu



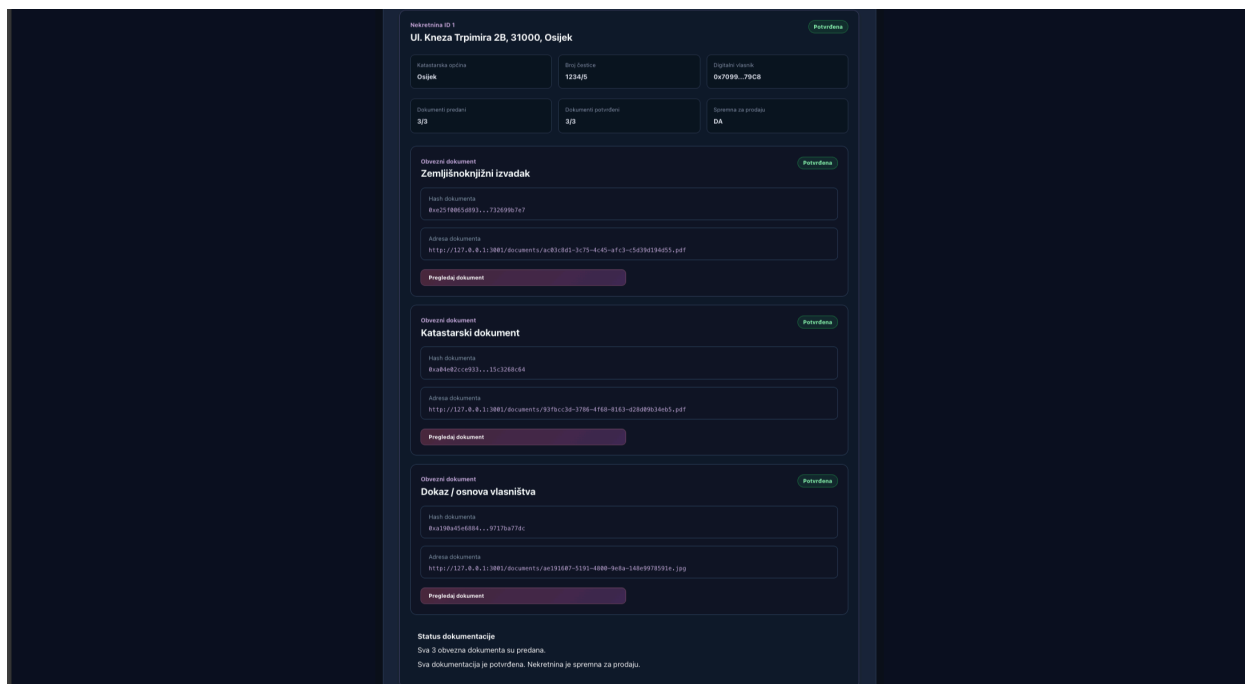
Slika 6.13. Pregled izvornog dokumenta prije verifikacije

Na slici 6.13. prikazan je primjer otvaranja zemljišnoknjižnog izvotka pomoću funkcionalnosti „Pregledaj dokument“. Na taj način Verifikator prije potvrđivanja ili odbijanja može pregledati sadržaj izvorne datoteke povezan s blockchain zapisom.



Slika 6.14. Potvrda verifikacije dokumenta putem MetaMask

Nakon pregleda dokumenta Verifikator pokreće njegovo potvrđivanje. Budući da promjena statusa dokumenta predstavlja WRITE operaciju, transakciju je potrebno potvrditi putem MetaMask novčanika.



Slika 6.15. Nekretnina s potpuno potvrđenom dokumentacijom

Nakon potvrđivanja sva tri obvezna dokumenta sustav evidentira da je dokumentacija nekretnine u potpunosti valjana. Nekretnina tada zadovoljava dokumentacijski uvjet za kreiranje prodaje.

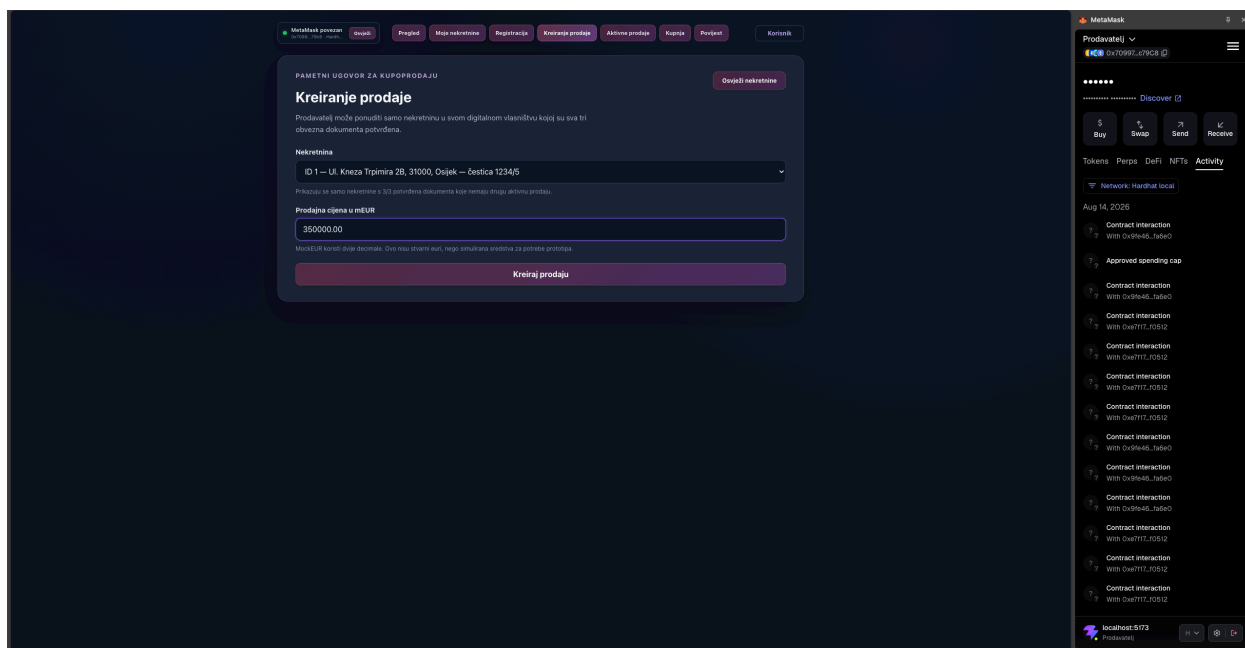
Isti postupak verifikacije proveden je i nad dokumentacijom preostalih testnih nekretnina, bez dodatnog prikazivanja jednakih koraka.

Sustav podržava i negativni scenarij verifikacije. Ako Verifikator utvrdi da pojedini dokument nije valjan, može ga odbiti, nakon čega se njegov status mijenja u odbijen. U tom slučaju dokumentacija nekretnine ne može se smatrati potpuno potvrđenom te takva nekretnina ne zadovoljava uvjete za kreiranje prodaje. Na taj način sustav osigurava da se u postupak kupoprodaje mogu uključiti samo nekretnine s valjanom i u potpunosti potvrđenom dokumentacijom.

U ispitivanju je provjeren i scenarij odbijanja dokumentacije, pri čemu odbijena nekretnina nije bila dostupna za daljnje kreiranje ponude.

6.4. Kreiranje prodaje

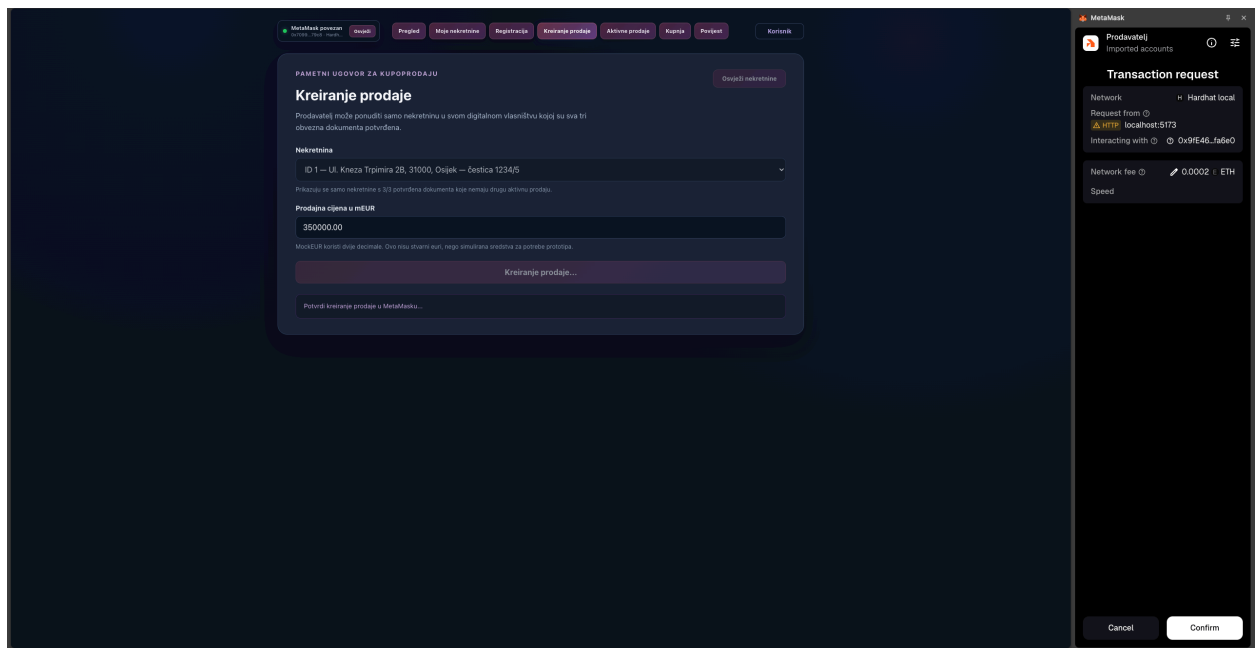
Nakon uspješne verifikacije dokumentacije nekretnina zadovoljava uvjete za kreiranje prodaje. Korisnik koji je evidentiran kao digitalni vlasnik može odabrati jednu od svojih potvrđenih nekretnina, unijeti prodajnu cijenu i pokrenuti postupak kreiranja ponude. Time se u pametnom ugovoru *RealEstateEscrow* evidentira nova aktivna prodaja dostupna drugim korisnicima sustava.



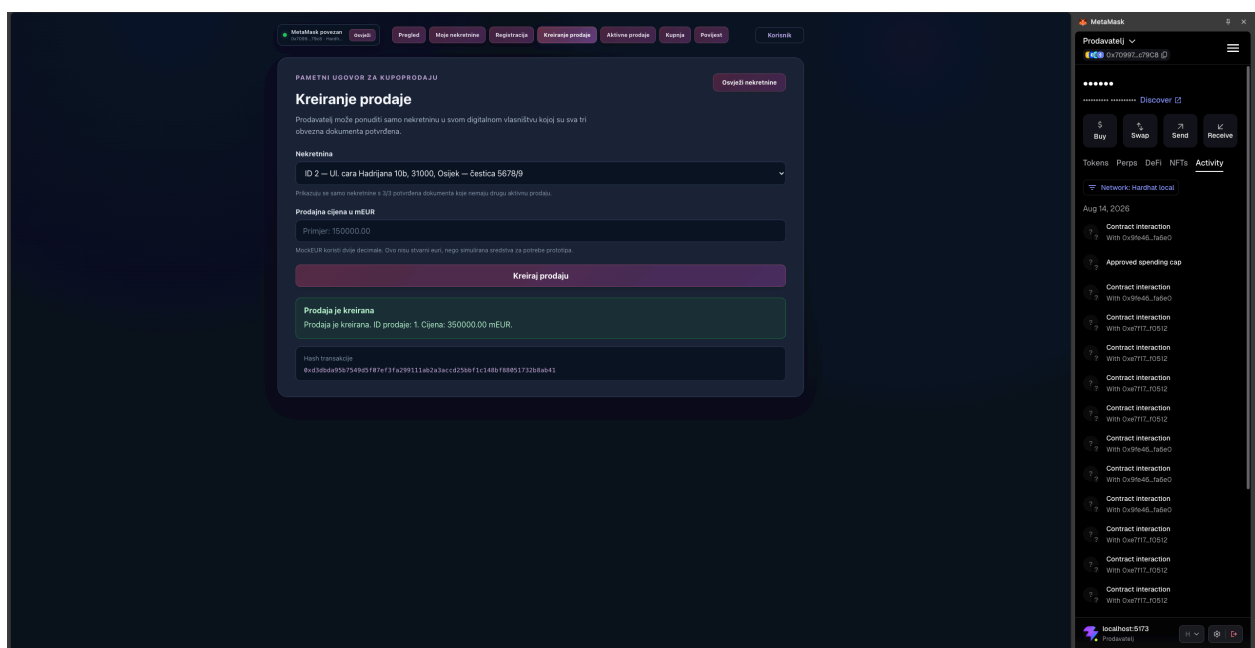
Slika 6.16. Unos podataka za kreiranje prodaje nekretnine

Na slici 6.16. prikazan je obrazac za kreiranje prodaje potvrđene nekretnine. Korisnik odabire jednu od svojih nekretnina spremnih za prodaju i unosi iznos prodajne cijene izražen u mEUR tokenima.

Zbog toga što se radi o WRITE operaciji, potrebna je potvrda na MetaMasku.

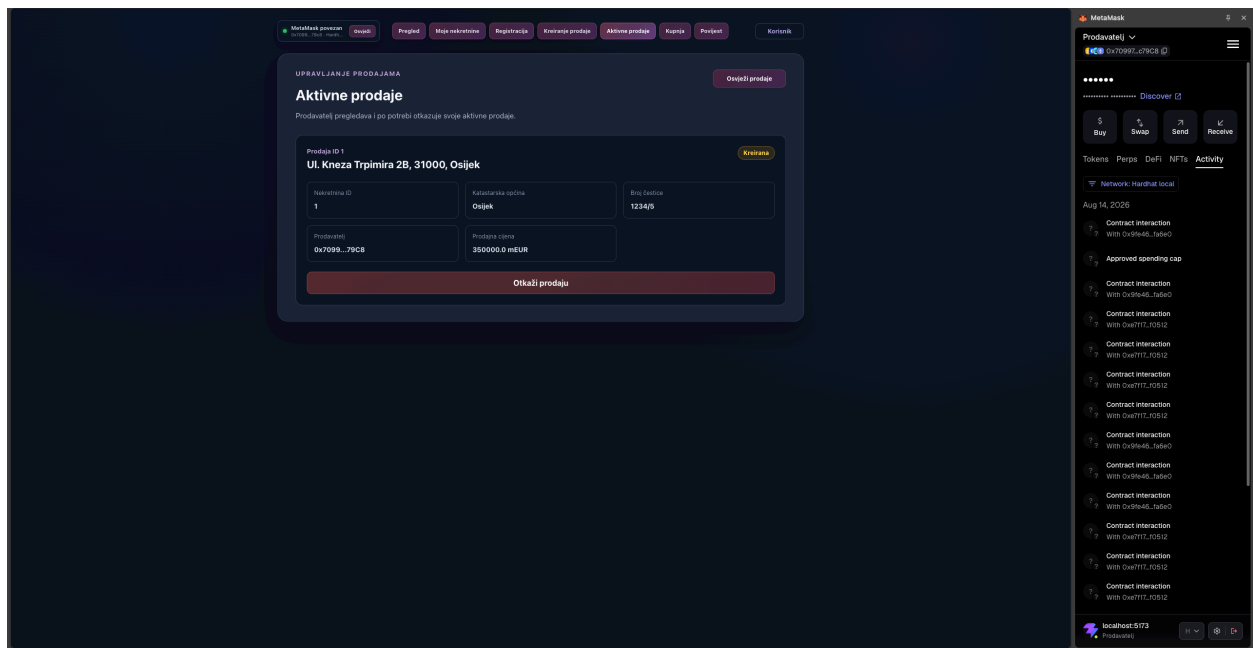


Slika 6.17. Potvrda kreiranja prodaje putem MetaMask



Slika 6.18. Uspješno kreirana aktivna prodaja nekretnine

Nakon potvrde transakcije prodaja se evidentira u sustavu kao aktivna ponuda. Takva prodaja postaje vidljiva vlasniku u prikazu aktivnih prodaja, a drugim korisnicima u prikazu dostupnih nekretnina za kupnju.

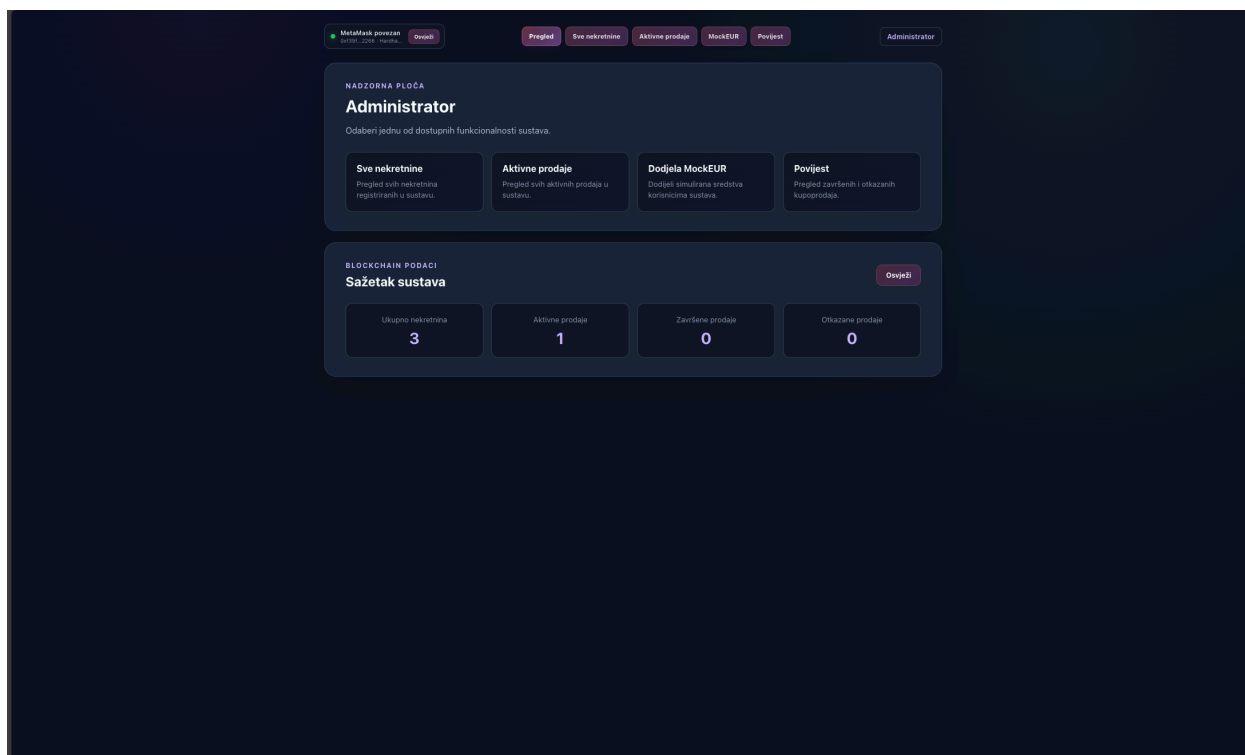


Slika 6.19. Prikaz aktivnih prodaja Korisnika

U ispitivanju je kreirana prodaja za jednu od nekretnina korisničkog računa koji raspolaže s dvije registrirane nekretnine. Na taj način pripremljen je scenarij u kojem drugi korisnički račun može pristupiti postupku kupnje iste nekretnine.

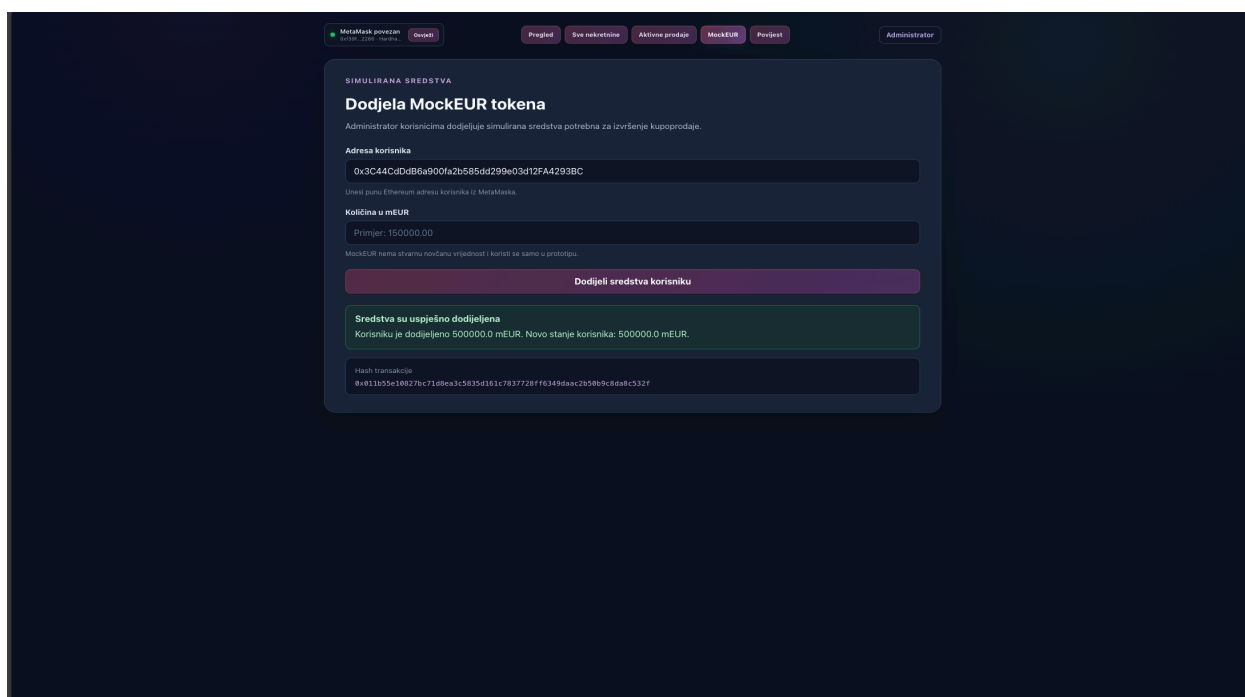
6.5. Dodjela simuliranih sredstava kupcu

Za izvršavanje kupnje drugi korisnički račun mora raspolagati dovoljnim iznosom simuliranih sredstava. Budući da se u prototipu koristi testni ERC-20 token *MockEUR*, potrebna sredstva kupcu dodjeljuje administratorski račun.



Slika 6.20. Dodjela *MockEUR* sredstava kupcu putem administratorskoga računa

Na slici 6.20. prikazan je administratorski prikaz za dodjelu simuliranih *MockEUR* sredstava korisničkom računu. Administrator unosi adresu primatelja i iznos tokena potreban za izvršavanje planirane kupoprodaje.

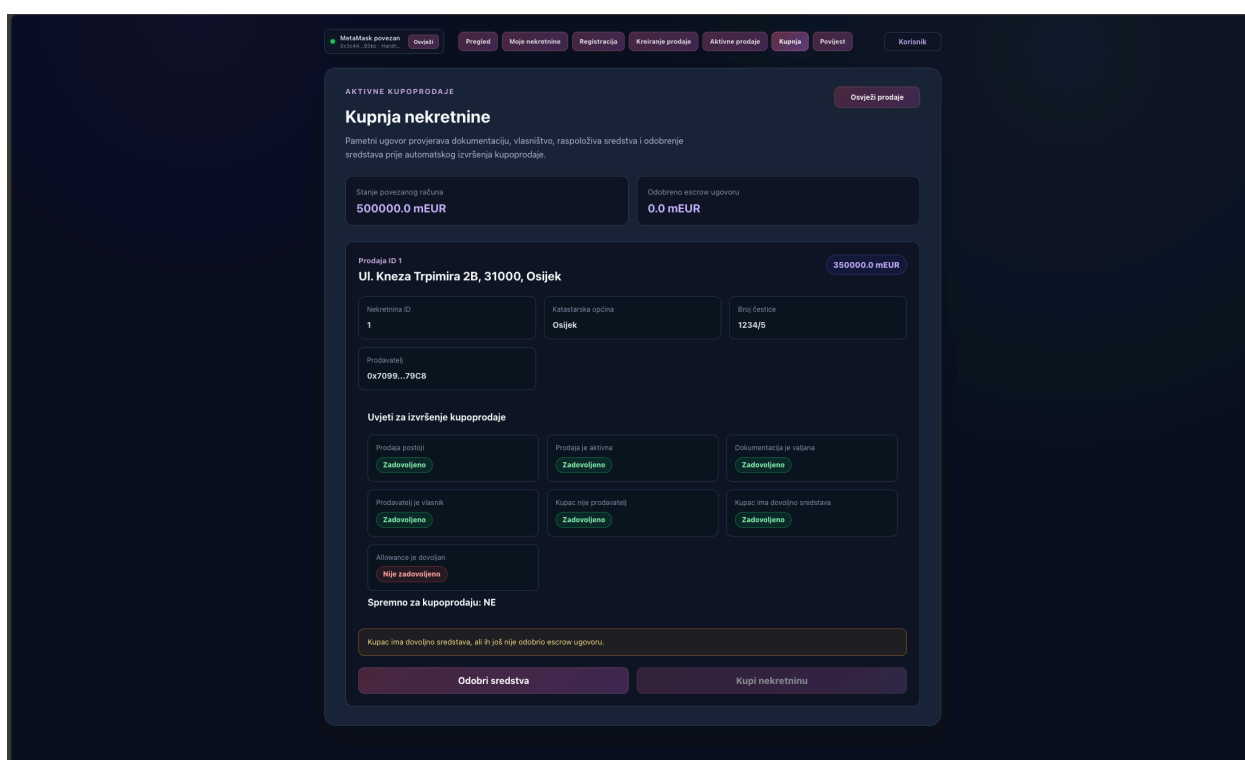


Slika 6.21. Dodjela simuliranih sredstava kupcu

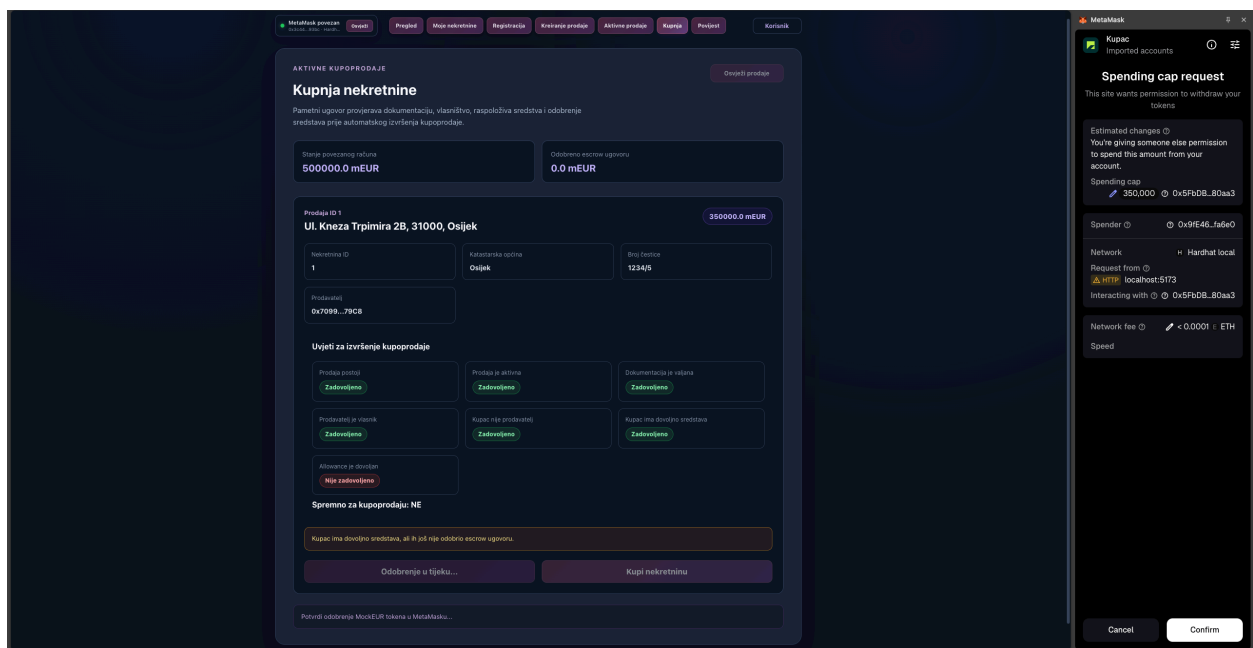
Nakon potvrde transakcije u MetaMasku, sustav evidentira uspješnu dodjelu *MockEUR* tokena kupcu. Na taj način su zadovoljeni početni financijski preduvjeti potrebni za nastavak postupka kupnje nekretnine.

6.6. Kupnja nekretnine

Nakon što kupac raspolaže dovoljnim iznosom *MockEUR* tokena, potrebno je escrow pametnom ugovoru odobriti korištenje potrebnog iznosa sredstava. Tek nakon toga mogu se zadovoljiti svi uvjeti za izvršenje kupoprodaje i pokrenuti završna blockchain transakcija.

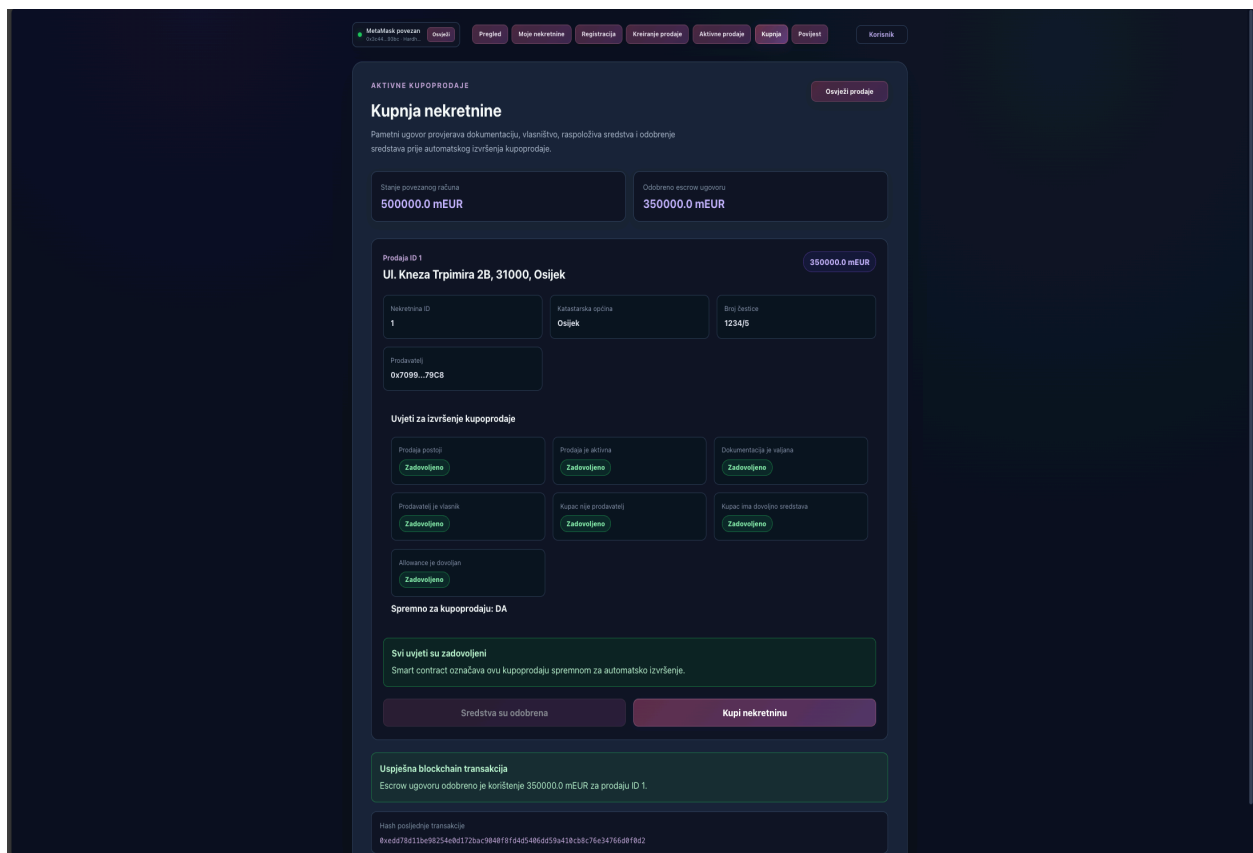


Slika 6.22. pregled aktivne prodaje iz perspektive Kupca

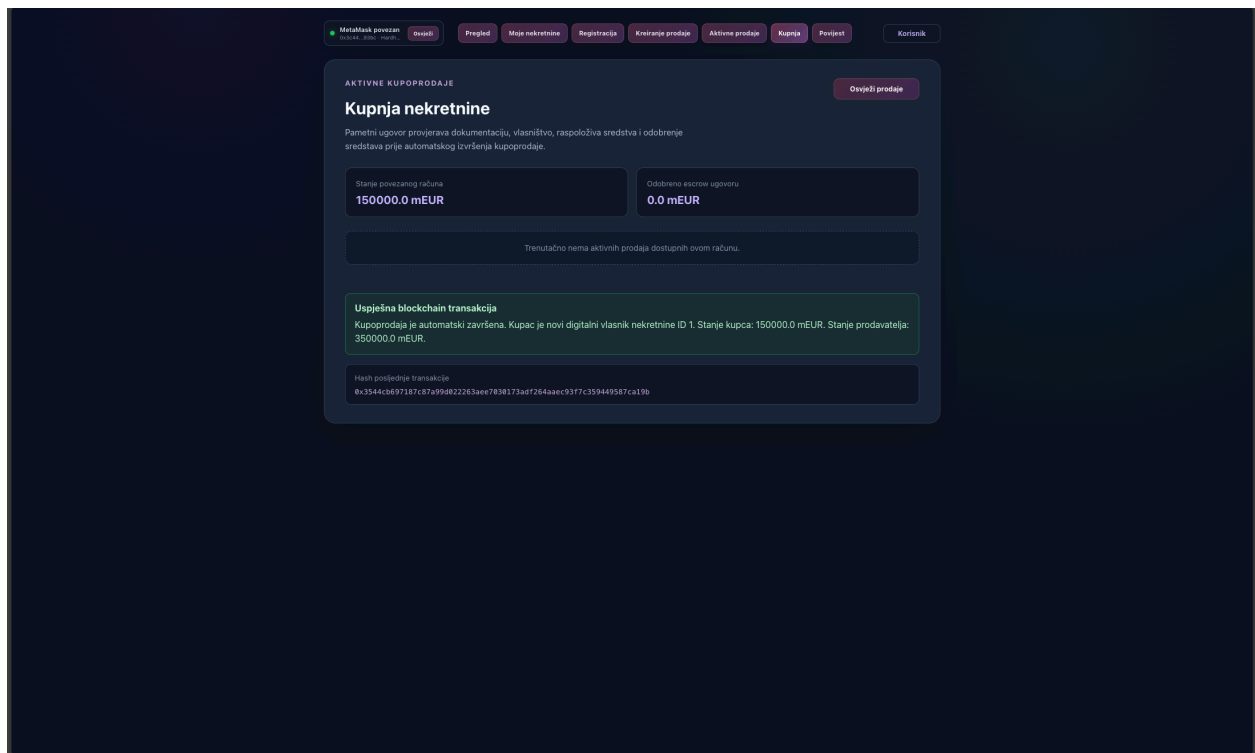


Slika 6.23. Odobravanje *MockEUR* sredstava escrow pametnom ugovoru

Prije pokretanja kupnje kupac escrow ugovoru mora odobriti raspolaganje potrebnim iznosom *MockEUR* tokena. Ovim korakom omogućuje se izvršavanje financijskog dijela kupoprodaje.



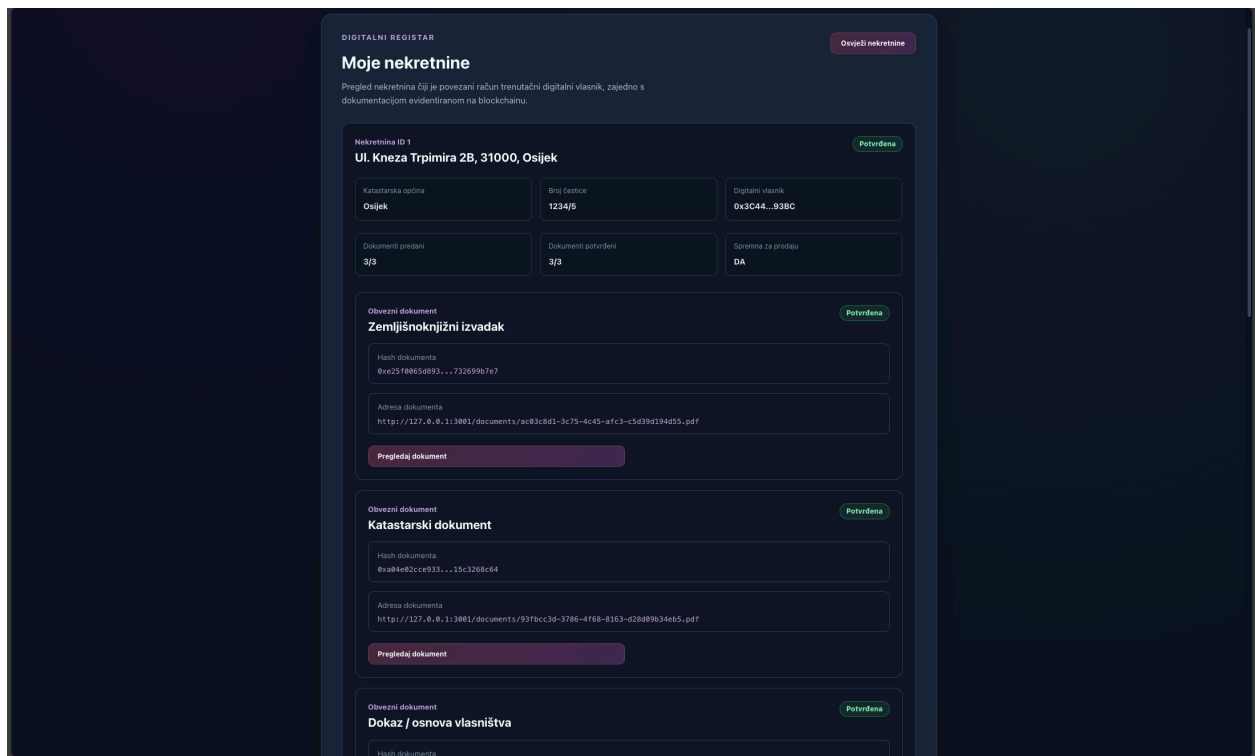
Slika 6.24. Odobrenje sredstava escrow ugovoru



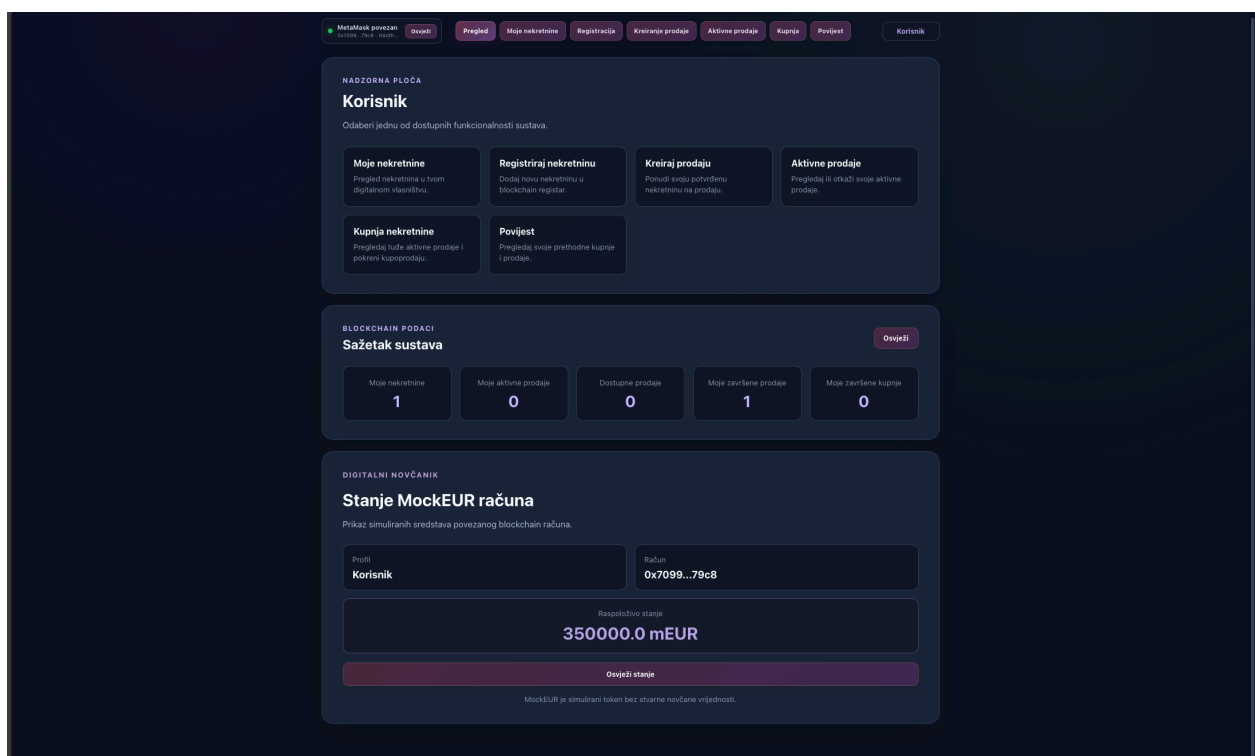
Slika 6.25. Uspješno izvršena kupoprodaja nekretnine

6.7. Rezultat kupoprodaje i povijest transakcija

Nakon završetka kupoprodaje moguće je provjeriti konačno stanje sustava kroz prikaz nekretnina i povijesti transakcija. Na taj se način potvrđuje da je digitalno vlasništvo preneseno na kupca, a da je transakcija ispravno evidentirana u sustavu.

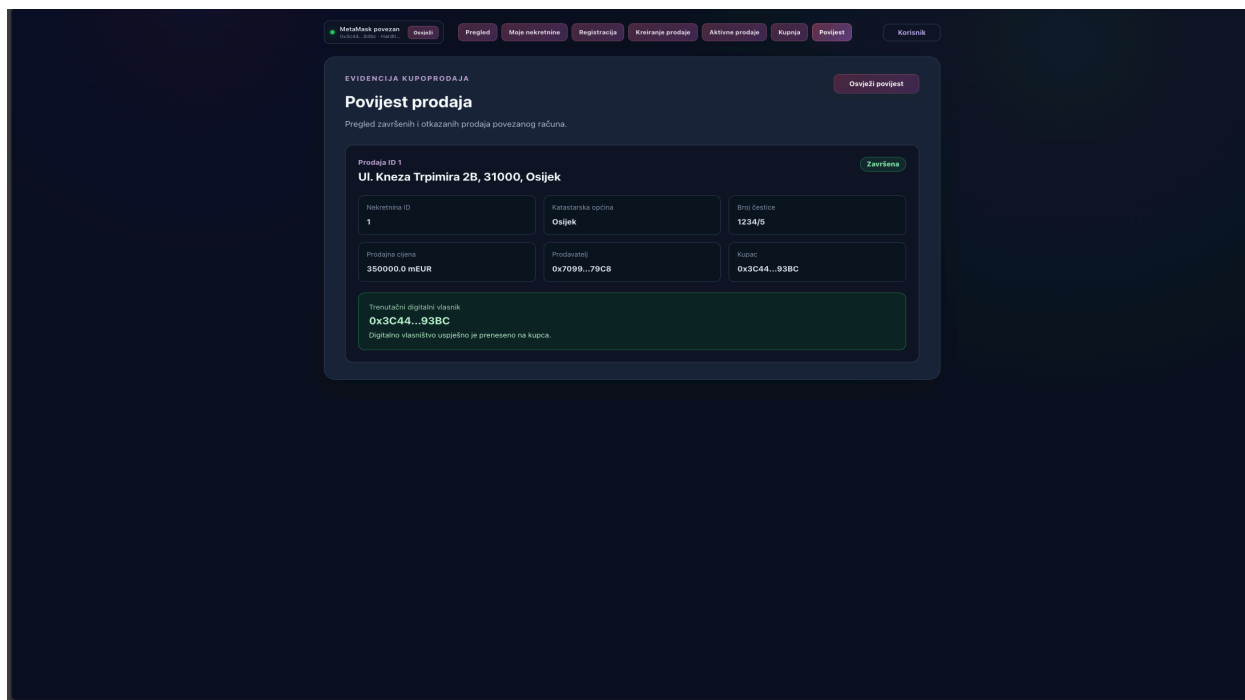


Slika 6.26. Nekretnina evidentirana kod novog digitalnog vlasnika



Slika 6.27. Prebačena sredstva na račun Prodavatelja

Nakon uspješne kupnje nekretnina više nije prikazana među aktivnim ponudama, nego se pojavljuje među nekretninama Korisnika koji ju je kupio. Time je potvrđen prijenos digitalnog vlasništva.



Slika 6.28. Prikaz evidentirane kupoprodaje u povijesti transakcija

Završena kupoprodaja ostaje evidentirana u povijesti transakcija sustava. Korisnik tako može pregledati prethodne kupnje i prodaje, dok administratorski profil ima mogućnost šireg nadzora svih evidentiranih transakcija.

6.8. Ispitivanje sustava

Uz praktično ispitivanje rada kroz korisničko sučelje provedeno je i ispitivanje funkcionalnosti pametnih ugovora pomoću automatiziranih Hardhat testova. Testovima su provjereni osnovni i rubni scenariji registracije nekretnina, verifikacije dokumentacije, kreiranja prodaje, financiranja kupoprodaje i završetka prijenosa digitalnog vlasništva.

```
PROBLEMS  TERMINAL  OUTPUT  DEBUG CONSOLE  PORTS  POSTMAN CONSOLE

● [ ] ~>doc\code\99\bitbank\BRD\Diplonski_rad\blockchain\real-estate\blockchain on [ ] multi-user :1 14 72 > npx hardhat test

Compiled 4 Solidity files with solc 0.8.28 (evm target: Cancun)

Running Solidity tests

Running node:test tests

--- MOCK EUR TOKEN ---
Adresa ugovora: 0x5fbd2315678afecb367f032d93f642f64180aa3
Naziv tokena: Mock Euro
Oznaka tokena: mEUR
Broj decimala: 2
Početna količina tokena: 0

MockEUR
  ✓ Postavlja token i vraća ispravne osnovne podatke

--- STVARANJE MOCK EUR TOKENA ---
Adresa kupca: 0x70997970c51812dc3a018c7d01b50e0d17dc79c8
Stvorena količina u najmanjim jedinicama: 20000000
Stanje kupca u najmanjim jedinicama: 20000000
Stanje kupca u mEUR: 200000
Ukupna količina tokena: 20000000

MockEUR
  ✓ Stvara mEUR tokene i dodjeljuje ih kupcu

--- NEOVLAŠTENO STVARANJE TOKENA ---
Adresa neovlaštenog korisnika: 0x70997970c51812dc3a018c7d01b50e0d17dc79c8
Stanje kupca nakon pokušaja: 0
Ukupna količina tokena nakon pokušaja: 0

MockEUR
  ✓ Ne dopušta korisniku bez nulte uloge stvaranje tokena

--- ODOBRENJE I PRIJENOS TOKENA ---
Adresa kupca: 0x70997970c51812dc3a018c7d01b50e0d17dc79c8
Adresa prodavatelja: 0x3c44cd0db6a300fa2b585dd299e03d12fa4293bc
Simulirana escrow adresa: 0x9079b6eb2c4f87036e785902ef1f01e3b906
Odobreni iznos: 1500000
Stanje kupca nakon prijenosa: 5000000
Stanje prodavatelja nakon prijenosa: 15000000
Preostalo odobrenje: 0

MockEUR
  ✓ Odobrenje i prijenos tokena putem transferFrom funkcije

--- PROPERTY REGISTRY ULOGE ---
Administrator: 0x3b06d51aad8b6f4ce6ab8827279cfff0b2266
DEFAULT_ADMIN_ROLE: true
VERIFIER_ROLE: false
TRANSFER_ROLE: false

PropertyRegistry
  ✓ Postavlja ugovor i dodjeljuje samo administratorsku ulogu deployeru

--- OBVEZNI DOKUMENTI ---
Broj obveznih dokumenata: 3

MockEUR
  ✓ Definira tri obvezne vrste dokumenata

--- REGISTRIRANA NEKRETNOST ---
ID: 1
Katastarska općina: Osijek
Broj čestice: 1234/5
Adresa: Europska avenija 1, Osijek
Dijelovni vlasnik: 0x70997970c51812dc3a018c7d01b50e0d17dc79c8
Status dokumentacije: 0
Svi dokumenti predani: false
Svi dokumenti valjani: false

MockEUR
  ✓ Registrira nekretnost bez automatske potvrde dokumentacije

--- PREDAJA DOKUMENTACIJE ---
Svi dokumenti predani: true
Svi dokumenti potvrđeni: false
URI zemljišnoknjižnog izvataka: ipfs://test/0823a2b46b9df6cb343c71009c57938255f8cfa573a9ebb0c0915f9f45ed5e41
URI katastarskog dokumenta: ipfs://test/782ebac0b19114685aa58099e237f664c31d188f0ba0ad454ed753881efdb384
URI dokaza vlasništva: ipfs://test/18a7794e0b4d1c51f421e455f1cbab807566adf99e27e531d4c14fb9b29e4a1

MockEUR
  ✓ Vlasnik predaje svi tri obvezna dokumenta s hashom i URI adresom
  ✓ Ne dopušta korisniku koji nije vlasnik predaju dokumenta
  ✓ Ne dopušta predaju dokumenta s praznim hashom
  ✓ Ne dopušta predaju dokumenta s praznim URI-jem
  ✓ Ne dopušta potvrdu dokumenta koji nije predan

--- DIELOVITA VERIFIKACIJA ---
Predano: 3/3
Potvrđeno: 2/3
Ukupni status: 0
Sprema za prodaju: false

MockEUR
  ✓ Djelotvorno potvrđena dokumentacija ne omogućuje prodaju

--- POTPUNA DOKUMENTACIJA ---
Svi dokumenti predani: true
Svi dokumenti potvrđeni: true
Status nekretnosti: 2
Sprema za prodaju: true

MockEUR
  ✓ Svi tri potvrđena dokumenta automatski potvrđuju nekretnost

--- OOBIZJENI DOKUMENT ---
Status dokumenta: 2
URI dokumenta: ipfs://test/18a7794e0b4d1c51f421e455f1cbab807566adf99e27e531d4c14fb9b29e4a1
Dokumentacija valjana: false

MockEUR
  ✓ Dodijena dokument automatski označava dokumentaciju nekretnosti dodijeljen
  ✓ Dodijeni dokument može se potpuno predati s novim hashom i URI-jem te zatim potvrditi
  ✓ Ne dopušta zamjenu već potvrđenog dokumenta
  ✓ Ne dopušta administratoru bez VERIFIER_ROLE potvrdu dokumenta
  ✓ Ne dopušta dodatnu registraciju isto katastarske čestice
  ✓ Vraća točan broj registriranih nekretnosti
  ✓ Administrator dodjeljuje VERIFIER_ROLE posebnom racunu
  ✓ Ne dopušta prijenos vlasništva ako dokumentacija nije potpuno potvrđena

--- USPJEŠAN PRIJENOS VLASNIŠTVA ---
Prodavatelj: 0x70997970c51812dc3a018c7d01b50e0d17dc79c8
Kupac: 0x3c44cd0db6a300fa2b585dd299e03d12fa4293bc
Vlasnik prije: 0x70997970c51812dc3a018c7d01b50e0d17dc79c8
Vlasnik nakon: 0x3c44cd0db6a300fa2b585dd299e03d12fa4293bc
Status dokumentacije: 1

MockEUR
  ✓ Racun s TRANSFER_ROLE prenosi vlasništvo kada su svi tri dokumenta potvrđena
  ✓ Ne dopušta korisniku bez TRANSFER_ROLE prijenos potvrđene nekretnosti

--- REAL ESTATE ESCROW ---
PropertyRegistry: 0x5fbd2315678afecb367f032d93f642f64180aa3
MockEUR: 0xe7f1725e7734ce288f8367e1bb143e90b3f0512
RealEstateEscrow: 0xf4e4673667362d9065f0992f22720e9f3c7fae0

RealEstateEscrow
  ✓ Postavlja escrow i povezuje ga s registrom i tokenom

--- KREIRANA PRODAJA ---
ID prodaje: 1
ID nekretnosti: 1
Prodavatelj: 0x70997970c51812dc3a018c7d01b50e0d17dc79c8
Kupac: 0x0000000000000000000000000000000000000000000000000000000000000000
Cijena: 15000000
Status: 0

MockEUR
  ✓ Vlasnik nekretnosti s potpuno potvrđenom dokumentacijom kreira prodaju

--- NEPOTPUNA VERIFIKACIJA ---
Svi dokumenti predani: true
Potvrđeno: 2/3
Dokumentacija valjana: false

MockEUR
  ✓ Ne dopušta kreiranje prodaje ako nisu potvrđena svi tri dokumenta
  ✓ Ne dopušta korisniku koji nije vlasnik nekretnosti kreiranje prodaje
  ✓ Ne dopušta ovakve aktivne prodaje za istu nekretnost

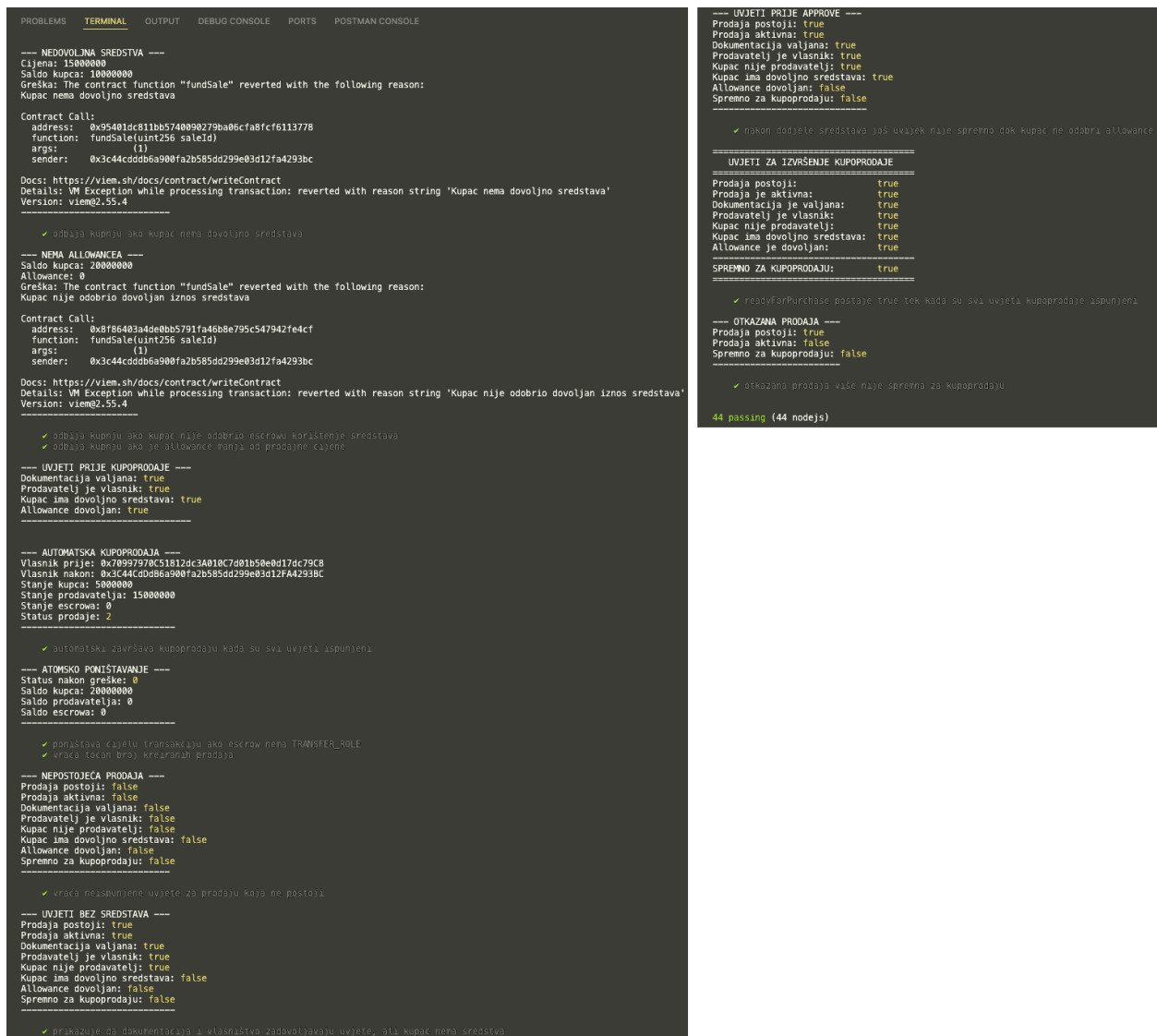
--- CIJENA NULA ---
Greška: The contract function "createSale" reverted with the following reason:
Cijena mora biti veća od nule

Contract Call:
address: 0x596b70e3f0db447751af701d576719a970857b
function: createSale(uint256 propertyId, uint256 price)
args: (1, 0)
sender: 0x70997970c51812dc3a018c7d01b50e0d17dc79c8

Docs: https://viem.sh/docs/contract/writeContract
Details: VM Exception while processing transaction: reverted with reason string 'Cijena mora biti veća od nule'
Version: viem@2.55.4

MockEUR
  ✓ Ne dopušta kreiranje prodaje s cijenom nula
  ✓ Prodavatelj otkazuje prodaju prije uplate i zatim može kreirati novu
  ✓ Ne dopušta drugom korisniku otkazivanje prodaje
  ✓ Ne dopušta prodavatelju novu vlastitu nekretnost
```

Slika 6.29. Rezultati uspješnog izvođenja automatiziranih testova – 1. dio



Slika 6.30. Rezultati uspješnog izvođenja automatiziranih testova – 2. dio

Na slikama 6.29. i 6.30. prikazani su rezultati izvođenja automatiziranih testova, čime je dodatno potvrđena ispravnost implementiranih pametnih ugovora i povezanih poslovnih pravila sustava.

7. ZAKLJUČAK

Cilj ovog diplomskog rada bio je istražiti mogućnosti primjene blockchain tehnologije i pametnih ugovora u procesu kupoprodaje nekretnina te na temelju provedene analize razviti funkcionalan prototip sustava koji demonstrira automatizaciju pojedinih koraka tog procesa. U početnom dijelu rada provedena je usporedba tradicionalnih, digitalnih i blockchainom temeljenih sustava za praćenje vlasništva nad nekretninama. Tradicionalni sustavi, poput zemljišnih knjiga i katastra, pružaju visoku razinu pravne sigurnosti i institucionalnog povjerenja, ali uključuju veliki broj formalnih postupaka i sudionika. Digitalizacijom se pojedini postupci ubrzavaju i podaci postaju dostupniji, no temeljna centralizirana struktura sustava uglavnom ostaje nepromjenjiva.

Analiza blockchain pristupa pokazala je da svojstva poput nepromjenjivosti zapisa, transparentnosti i mogućnosti automatizacije poslovnih pravila pomoću pametnih ugovora mogu imati značajnu primjenu u području evidencije i prijenosa vlasništva. Istodobno, analizom pravnog okvira Republike Hrvatske utvrđeno je da se stvarno pravo vlasništva nad nekretninom ne može prenijeti isključivo promjenom blockchain zapisa. Za pravno valjan prijenos i dalje je potrebno zadovoljiti uvjete propisane važećim zakonodavstvom i provesti odgovarajući upis u zemljišne knjige. Zbog toga blockchain u ovom području treba promatrati kao moguću tehnološku nadogradnju i sredstvo automatizacije, a ne kao neposrednu zamjenu postojećih državnih registara i institucija.

U praktičnom dijelu rada razvijen je prototip sustava temeljen na Ethereum kompatibilnoj blockchain platformi. Poslovna logika raspoređena je između pametnih ugovora *PropertyRegistry*, *RealEstateEscrow* i *MockEUR*. Ugovor *PropertyRegistry* omogućuje registraciju nekretnina, evidenciju trenutnog digitalnog vlasnika i upravljanje obveznom dokumentacijom. Za svaku nekretninu definirana su tri obvezna dokumenta: zemljišnoknjižni izvadak, katastarski dokument i dokaz odnosno osnova vlasništva. Svaki dokument prolazi zaseban postupak verifikacije te može biti u stanju *Pending*, *Verified* ili *Rejected*. Nekretnina može sudjelovati u daljnjem postupku prodaje tek kada je sva potrebna dokumentacija predana i potvrđena.

Budući da pohrana cijelih PDF ili slikovnih datoteka izravno na blockchain nije praktična, implementiran je kombinirani model pohrane dokumentacije. Izvorne datoteke spremaju se u pomoćno off-chain spremište, dok se na blockchain zapisuju njihove *keccak256* hash vrijednosti i URI adrese. Time je omogućeno povezivanje konkretnog dokumenta s nepromjenjivim blockchain zapisom, a Verifikator istodobno može otvoriti izvornu datoteku i pregledati njezin sadržaj prije

potvrđivanja ili odbijanja. Takav pristup pokazuje kako se blockchain može koristiti za čuvanje integriteta podataka bez potrebe za pohranom velikih datoteka u samom blockchain stanju.

Proces kupoprodaje implementiran je pomoću pametnog ugovora *RealEstateEscrow*. Prije izvršenja kupnje ugovor provjerava postoji li prodaja i je li aktivna, je li dokumentacija nekretnine potpuno potvrđena, je li prodavatelj trenutni digitalni vlasnik nekretnine, razlikuju li se kupac i prodavatelj te raspolaže li kupac dovoljnim iznosom simuliranih sredstava i potrebnim odobrenjem za njihovo korištenje. Financijski dio prototipa simuliran je ERC-20 tokenom *MockEUR*. Kada su svi definirani uvjeti zadovoljeni, prijenos sredstava, prijenos digitalnog vlasništva i završetak prodaje izvršavaju se kao dio iste blockchain transakcije. Time se postiže automatsko izvršavanje postupka, odnosno sprječava se situacija u kojoj bi samo dio kupoprodaje bio uspješno proveden.

Za korištenje sustava razvijeno je React i TypeScript korisničko sučelje povezano s blockchain mrežom pomoću biblioteke *ethers.js* i MetaMask novčanika. Implementiran je višekorisnički način rada s profilima Administrator, Verifikator i Korisnik. Korisnik nije trajno određen kao kupac ili prodavatelj, nego njegova uloga ovisi o konkretnoj transakciji i trenutnom blockchain stanju. Time isti korisnik može prodavati vlastitu nekretninu, kupiti nekretninu drugog korisnika te naknadno ponovno ponuditi kupljenu nekretninu na prodaju. Administrator upravlja simuliranim financijskim sredstvima i nadzornim funkcionalnostima, dok verifikator provodi provjeru dokumentacije.

Rad sustava ispitan je kroz praktične scenarije u korisničkom sučelju i automatizirane Hardhat testove. Provjereni su registracija više nekretnina s različitih računa, predaja i pregled dokumentacije, potvrđivanje i odbijanje pojedinačnih dokumenata, kreiranje prodaje, dodjela simuliranih sredstava kupcu, odobravanje sredstava escrow ugovoru, izvršenje kupoprodaje, promjena digitalnog vlasnika i prikaz završene transakcije u povijesti. Ispitani su i negativni scenariji, poput pokušaja kupnje bez dovoljnog sredstva, prodaja nekretnine s nepotvrđenom dokumentacijom i drugih neispunjenih uvjeta. Rezultati ispitivanja pokazali su da pametni ugovori provode definirana poslovna pravila neovisno o korisničkom sučelju te odbijaju nedopuštene operacije.

Za primjenu sličnog sustava u stvarnom okruženju u Republici Hrvatskoj, bilo bi potrebno ostvariti integraciju s postojećim zemljišnoknjižnim i katastarskim sustavima, riješiti pravna pitanja valjanosti blockchain zapisa i pametnih ugovora, pouzdanu identifikaciju sudionika, zaštitu osobnih podataka i način korištenja stvarnih financijskih sredstava.

Na temelju provedenog istraživanja, implementacije i ispitivanja može se zaključiti da blockchain tehnologija i pametni ugovori imaju potencijal za povećanje transparentnosti i automatizacije pojedinih dijelova procesa kupoprodaje nekretnina. Razvijeni prototip pokazao je da je tehnički moguće u jedinstvenom sustavu povezati registraciju nekretnina, provjeru dokumentacije, upravljanje ponudama, provjeru financijskih uvjeta, prijenos sredstava i promjenu digitalnog vlasništva. Međutim, stvarna primjena takvog sustava ne ovisi samo o tehničkoj izvedivosti, nego prije svega o njegovoj integraciji s postojećim pravnim i institucionalnim okvirom.

LITERATURA

- [1] Zakon o vlasništvu i drugim stvarnim pravima, Narodne novine 91/96, 68/98, 137/99, 22/00, 73/00, 114/01, 79/06, 141/06, 146/08, 38/09, 153/09, 143/12, 152/14, 81/15, 94/17. Dostupno na: <https://www.zakon.hr/z/241/zakon-o-vlasnistvu-i-drugim-stvarnim-pravima> [08.02.2026.]
- [2] Državna geodetska uprava, Katastar nekretnina. Dostupno na: <https://dgu.gov.hr/katastar-nekretnina/152> [08.02.2026.]
- [3] Strategija e-Hrvatska 2020, Narodne novine 85/2015, Dostupno na: https://mpudt.gov.hr/UserDocsImages//RDD/dokumenti//Strategija_e-Hrvatska_2020.pdf [08.02.2026.]
- [4] Vlada Republike Hrvatske, Nacionalni plan oporavka i otpornosti 2021.–2026. Dostupno na: <https://planoporavka.gov.hr/> [08.02.2026.]
- [5] Državna geodetska uprava, Sektor za katastarski sustav. Dostupno na: <https://dgu.gov.hr/o-nama/ustrojstvo/sektor-za-katastarski-sustav/111> [08.02.2026.]
- [6] S. Verma, S. K. Pathak, P. Chaudhary, P. Yadav, S. Yousuf, „Land Registry System Using Blockchain Technology“, 2024 International Conference on Emerging Innovations and Advanced Computing (INNOCOMP), str. 170–177, 2024.
- [7] M. Kempe, „The Land Registry in the Blockchain – Testbed“, razvojni projekt Lantmäteriet, Landshypotek Bank, SBAB, Telia Company, ChromaWay i Kairos Future, 2017. [09.02.2026.]
- [8] BlockchainX, *Blockchain-Based Land Registry System*. Dostupno na: <https://www.blockchainx.tech/blockchain-based-land-registry-system/> [09.02.2026.]
- [9] B. Makala, A. Anand, „Blockchain and Land Administration“, The Legal Aspects of Blockchain, UNOPS / World Bank, 2018. [09.02.2026.]
- [10] Debut Infotech, *How Blockchain Land Registry System Works*. Dostupno na: <https://www.debutinfotech.com/blog/how-blockchain-land-registry-system-works> [09.02.2026.]
- [11] Zakon o zemljišnim knjigama, NN **63/19, 128/22, 155/23, 127/24**, Dostupno na: <https://www.zakon.hr/z/103/zakon-o-zemljisnim-knjigama> [01.04.2026.]
- [12] Sustav E-Građani Republike Hrvatske, Dostupno na: <https://gov.hr/hr/upis-prava-vlasnistva/1306> [01.04.2026.]
- [13] Katastar - sve što trebate znati, Dostupno na: <https://www.zavasnekretnine.hr/katastar> [01.04.2026.]

- [14] Online na jednom mjestu - katastar i zemljišne knjige, Dostupno na: <https://oss.uredjenazemlja.hr/> [01.04.2026.]
- [15] Vodič namijenjen građanima vezano uz oporezivanje stjecanja nekretnina, Dostupno na: <https://porezna-uprava.gov.hr/hr/stjecatelj-nekretnine/4700> [01.04.2026]
- [16] I. Bashir, „Blockchain Consensus: An Introduction to Classical, Blockchain, and Quantum Consensus Protocols“, 1st Edition, Apress, 2022.
- [17] C. Nguyen, H. Thai, D. Nguyen „Proof-of-Stake Consensus Mechanisms for Future Blockchain Networks: Fundamentals, Applications and Opportunities“, IEEE Access PP(99):1-1, 2019.
- [18] A. Yakovenko, „Solana: A New Architecture for a High Performance Blockchain“, Solana Whitepaper. Dostupno na: <https://solana.com/solana-whitepaper.pdf> [17.08.2026.]

POPIS KRATICA

CORS – Cross-Origin Resource Sharing

DLT – Distributed Ledger Technology

ERC-20 – Ethereum Request for Comments 20

HTTP – Hypertext Transfer Protocol

JPEG – Joint Photographic Experts Group

JPG – Joint Photographic Experts Group

JSON-RPC – JavaScript Object Notation – Remote Procedure Call

mEUR – oznaka simuliranog tokena MockEUR

PDF – Portable Document Format

PDV – porez na dodanu vrijednost

PNG – Portable Network Graphics

RPC – Remote Procedure Call

URI – Uniform Resource Identifier

URL – Uniform Resource Locator

ZIS – Zajednički informacijski sustav zemljišnih knjiga i katastra

SAŽETAK

U ovom diplomskom radu istražena je primjena blockchain tehnologije i pametnih ugovora u postupku kupoprodaje nekretnina. Provedena je usporedba tradicionalnih, digitalnih i blockchainom temeljenih sustava za praćenje vlasništva te je analiziran pravni okvir prijenosa vlasništva nad nekretninama u Republici Hrvatskoj. Na temelju provedene analize razvijen je prototip višekorisničkog blockchain sustava za registraciju nekretnina, upravljanje dokumentacijom i automatizirano izvršenje kupoprodaje.

Sustav je implementiran pomoću pametnih ugovora *PropertyRegistry*, *RealEstateEscrow* i *MockEUR*, dok je korisničko sučelje razvijeno u Reactu i TypeScriptu. Za svaku nekretninu evidentiraju se osnovni podaci, digitalni vlasnik i tri obvezna dokumenta. Izvorne datoteke dokumentacije pohranjuju se izvan blockchaina, dok se njihove hash vrijednosti i URI adrese zapisuju u blockchain. Verifikator svaki dokument može zasebno potvrditi ili odbiti, a prodaja je dopuštena tek nakon potvrde cjelokupne dokumentacije.

Kupoprodaja se izvršava pomoću escrow pametnog ugovora koji provjerava vlasništvo, valjanost dokumentacije, stanje i odobrenje simuliranih *MockEUR* sredstava te druge definirane uvjete. Nakon njihova ispunjenja prijenos sredstava i digitalnog vlasništva izvršavaju se automatski unutar iste blockchain transakcije. Razvijeni sustav ispitan je kroz korisničko sučelje i automatizirane Hardhat testove. Rezultati pokazuju mogućnost automatizacije i povećanja transparentnosti pojedinih dijelova kupoprodajnog procesa.

Ključne riječi: blockchain, digitalno vlasništvo, Ethereum, nekretnine, pametni ugovori

IMPLEMENTATION OF A BLOCKCHAIN SYSTEM FOR REAL ESTATE TRANSACTIONS USING SMART CONTRACTS

ABSTRACT

This master's thesis investigates the application of blockchain technology and smart contracts in the real estate purchase and sale process. A comparison of traditional, digital, and blockchain-based property ownership tracking systems was conducted, and the legal framework governing the transfer of real estate ownership in the Republic of Croatia was analyzed. Based on this analysis, a prototype multi-user blockchain system for property registration, document management, and automated execution of real estate transactions was developed.

The system was implemented using the *PropertyRegistry*, *RealEstateEscrow*, and *MockEUR* smart contracts, while the user interface was developed using React and TypeScript. For each property, basic information, the digital owner, and three mandatory documents are recorded. The original document files are stored off-chain, while their hash values and URI addresses are recorded on the blockchain. A verifier can individually approve or reject each document, and a property can only be offered for sale after all required documentation has been approved.

The purchase and sale process is executed through an escrow smart contract that verifies ownership, document validity, the balance and allowance of simulated *MockEUR* funds, and other predefined conditions. Once all conditions are satisfied, the transfer of funds and digital ownership is executed automatically within the same blockchain transaction. The developed system was tested through the user interface and automated Hardhat tests. The results demonstrate the potential for automating and increasing the transparency of individual parts of the real estate transaction process.

Keywords: blockchain, digital ownership, Ethereum, real estate, smart contracts

ŽIVOTOPIS

Mislav Čačić rođen je 1. siječnja 1999. godine u Osijeku. Nakon završetka Osnovne škole Bartola Kašića u Vinkovcima 2014. godine upisuje smjer Tehničar za mehatroniku u Tehničkoj školi Ruđera Boškovića u Vinkovcima, koju završava 2018. godine. Iste godine upisuje sveučilišni prijediplomski studij Računarstva na Fakultetu elektrotehnike, računarstva i informacijskih tehnologija Osijek, koji završava 2024. godine, nakon čega upisuje sveučilišni diplomski studij Računarstva, smjer Podatkovna znanost. Tijekom studija stječe praktično iskustvo kroz stručne prakse i rad na poslovima razvoja softvera, prvenstveno u području frontend i web-razvoja, koristeći tehnologije React, Angular, JavaScript, TypeScript, C# i baze podataka.

Potpis autora

PRILOZI

Izvorni kod razvijenog blockchain sustava za kupoprodaju nekretnina dostupan je u GitHub repozitoriju: <https://github.com/MislavCacic/Masters-Thesis>